ONLINE RESORT MANAGEMENT SYSTEM

Resort Room Booking and Management System

Complete Codebase Documentation

Author: AI Engineering System

Date: July 19, 2026

Files Documented: 62 Source & Config Assets

Table of Contents

page.tsx

page.tsx

loading.tsx

page.tsx

globals.css

layout.tsx

loading.tsx

page.tsx

page.tsx

page.tsx

AnimateOnScroll.tsx

Footer.tsx

MobileDock.tsx

Navbar.tsx

ResortDetailsClient.tsx

ResortMap.tsx

SessionProviderWrapper.tsx

TiltCard.tsx

button.tsx

calendar.tsx

card.tsx

dialog.tsx

dropdown-menu.tsx

input.tsx

popover.tsx

tabs.tsx

auth.ts

cache.ts

db.ts

mailer.ts

utils.ts

proxy.ts

components.json

eslint.config.mjs

next-env.d.ts

next.config.ts

package.json

postcss.config.mjs

tsconfig.json

Resort Room Booking and Management System: System Architecture

Welcome to the architectural documentation for the **Resort Room Booking and Management System**. This document describes the application design, modern tech stack, key components, and directory layout.

---

1. Executive Technology Stack

The application is built as a unified full-stack web application optimized for performance, security, and aesthetics.

- **Framework: Next.js (v16.2.10, App Router)** with **React (v19)**. The project leverages Server Components for SEO and fast initial renders, and Client Components (using `'use client'`) for interactive widgets.
- **Database ORM: Prisma Client (v6.2.1)** connected to **Neon Serverless PostgreSQL**. Type-safe database queries are written through Prisma, with a database schema defined in `prisma/schema.prisma`.
- **Authentication & Security: Next-Auth (Auth.js v4.24.14)** with Credentials Provider. It implements Role-Based Access Control (RBAC) supporting three primary user tiers:

1. **Guest:** Access to the booking engine, reservation history, checkout invoice, and profile management.

2. **Staff:** Housekeeping, room inspection dashboard, and task completion flow.

3. **Admin (Specialized Staff):** Full operational control panel including financial reporting, department management, staff CRUD, and booking audit desks.

- **Styling & UI Components: Tailwind CSS (v4)** with custom configurations and components from **21st.dev**. Responsive layouts incorporate **GSAP (GreenSock)** scroll animations, **canvas-confetti** for micro-interactions, and **Leaflet Maps** for coordinate tracking.
- **Notifications Subsystem: Nodemailer** for sending automated booking receipts and notifications via SMTP.
- **Payment Simulator:** Mock checkout integrating with simulated **Dodo Payments** webhooks to mock credit card payments and secure transactional workflows.

---

2. Directory Layout & Organization

The codebase is organized in a standard Next.js App Router structure:

```
Online Resort Management System/
├── prisma/                          # Database configuration and migration files
│   ├── schema.prisma                # Prisma DB schema definition
│   ├── seed.js                      # Predefined seed data for development
│   └── seed_100.js                  # Advanced seeding script (generating 100+ items)
├── public/                          # Static assets (SVGs, favicon)
├── src/
│   ├── app/                         # App Router page components & API endpoints
│   │   ├── about/                   # Public about page
│   │   ├── api/                     # Backend API routes (Admin, Auth, Book, checkout, Dasl
│   │   ├── book/                    # Interactive booking flow pages
│   │   ├── checkout/                # Invoice review and mock payment pages
│   │   ├── dashboard/               # Main role-based management dashboard
│   │   ├── resorts/                 # Resorts browsing pages
│   │   ├── globals.css              # Global styles & Tailwind directives
│   │   ├── layout.tsx               # Root HTML wrapper and session provider
│   │   └── page.tsx                 # High-performance animated landing page
│   ├── components/                  # Reusable UI widgets and layout modules
│   │   ├── ui/                      # Primitive design-system components (buttons, dialogs
│   │   ├── Navbar.tsx               # Global persistent header navigation
│   │   ├── Footer.tsx               # Responsive interactive footer
│   │   └── AnimateOnScroll.tsx      # GSAP scroll-triggered wrapper
│   ├── lib/                         # Shared utilities and global singletons
│   │   ├── auth.ts                  # Next-Auth configuration & credentials callbacks
│   │   ├── cache.ts                 # In-memory API response cache
│   │   ├── db.ts                    # Prisma DB client instantiation
│   │   ├── mailer.ts                # Nodemailer transporter and send helper
│   │   └── utils.ts                 # CSS class-merging helper
│   └── proxy.ts                     # Next-Auth routing middleware configuration
├── package.json                     # Node.js dependencies and run scripts
├── tsconfig.json                    # TypeScript compiler configuration
└── components.json                  # Shadcn-UI configuration
```

3. Core Database Models

The database models mapped in `prisma/schema.prisma` form the backbone of the resort's operational data flow:

- **Resort:** Represents different property locations (e.g., Maldives, Bali, Aspen) with coordinates, ratings, and image arrays.
- **RoomType:** Defines different room categories (`Deluxe` , `Suite` , `Villa`) with base pricing, description, and occupancy capacities.
- **Room:** Specific physical rooms linked to a `Resort` and `RoomType` , tracking their current cleaning state (`AVAILABLE` , `OCCUPIED` , `MAINTENANCE` , `DIRTY`).

- **Guest:** User accounts for public visitors, containing email, hashed password, nationality, and phone details.
- **Reservation:** Tracks check-in/out schedules, guest counts, total pricing, and reservation states (`PENDING` , `CONFIRMED` , `CANCELED`).
- **Payment:** Records billing transactions linked to reservations, detailing amounts, payment methods, and statuses (`PENDING` , `COMPLETED` , `FAILED` , `REFUNDED`).
- **Department & Staff:** Internal corporate structure mapping staff members to departments (e.g., Housekeeping, Front Desk) and roles (e.g., `STAFF` vs. `ADMIN`).
- **Service:** Extra guest services (`Spa Treatment` , `Private Chef` , `Airport Transfer`) with pricing.
- **ReservationService:** Join model linking selected add-on services to reservations.
- **RoomAssignment:** Tracks tasks (`Turnover Cleaning` , `Deep Sweep` , `Repair`) assigned to staff members for a specific room.

---

4. Key Architectural Patterns

A. Next-Auth Credentials & Custom Session JWTs

To maintain role-based access control (RBAC), Next-Auth callbacks in `src/lib/auth.ts` inspect the incoming login request. Staff accounts verify credentials against the `Staff` table, while guest accounts check the `Guest` table. If successful, user roles (`ADMIN` , `STAFF` , `GUEST`) and user types (`staff` , `guest`) are stored in the JWT token and forwarded to the client-side session.

B. In-Memory Singletons (Database & Cache)

To avoid exhausting connection pools in serverless contexts:

1. **Prisma Client (`src/lib/db.ts`):** Created as a global singleton `globalForPrisma.prisma` to prevent re-instantiation on Next.js hot-reloads.

2. **Memory Cache (`src/lib/cache.ts`):** Instantiated globally to cache stable API endpoints like resort listings, minimizing redundant SQL queries.

C. Route Middleware Protection (`src/proxy.ts`)

Standard Next-Auth middleware handles route-level route protection. Only authenticated sessions can access subpaths matching `/dashboard/:path*` , `/book/:path*` , and `/checkout/:path*` , redirecting anonymous users back to the `/login` page.

D. Transaction Isolation for Bookings & Cleanliness

To prevent overbooking or inconsistent states:

- **Booking Engine (`src/app/api/book/route.ts`):** Performs date conflict scans.

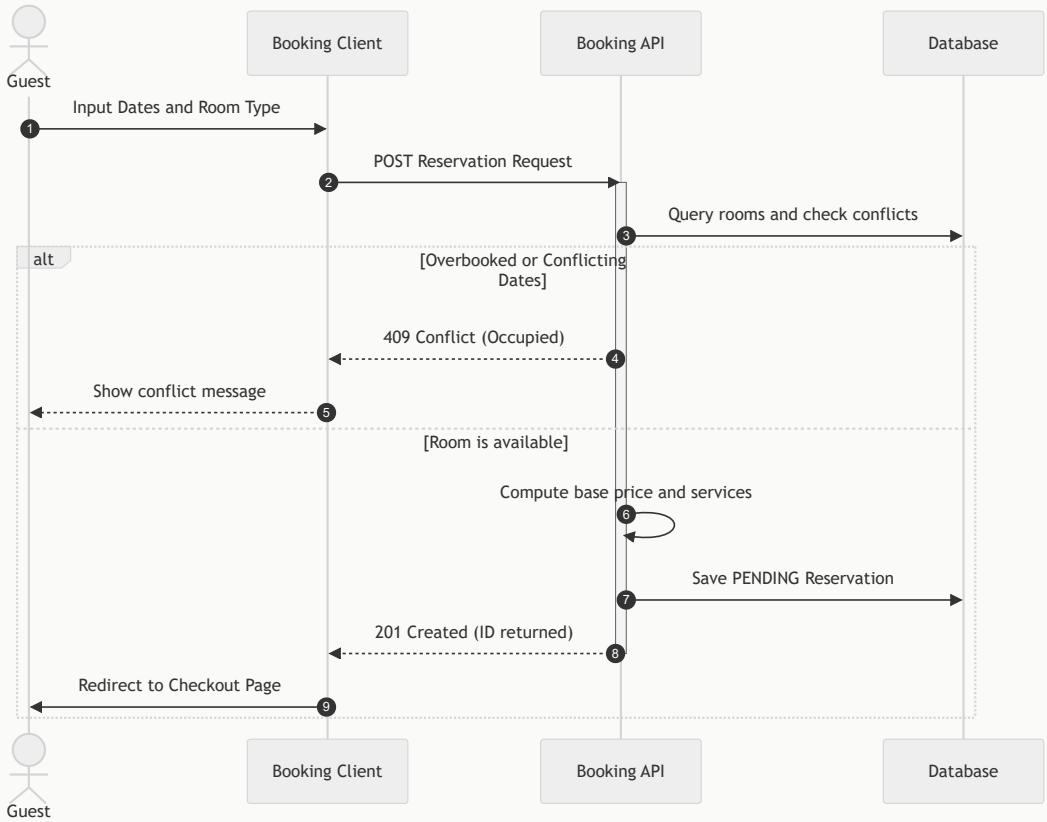
- **Administrative Actions (`src/app/api/admin/bookings/route.ts`):** Integrates transactional operations (`prisma.$transaction`) to handle check-in, check-out, and mid-stay cancellations. Canceling a stay mid-way updates the reservation, updates the room status to `DIRTY`, adjusts the original payment to a prorated amount, and inserts a refund payment record in a single, atomic operation.

Resort Room Booking and Management System: Application Flows

This document details the central operational flows of the application using structured sequence maps and logic paths.

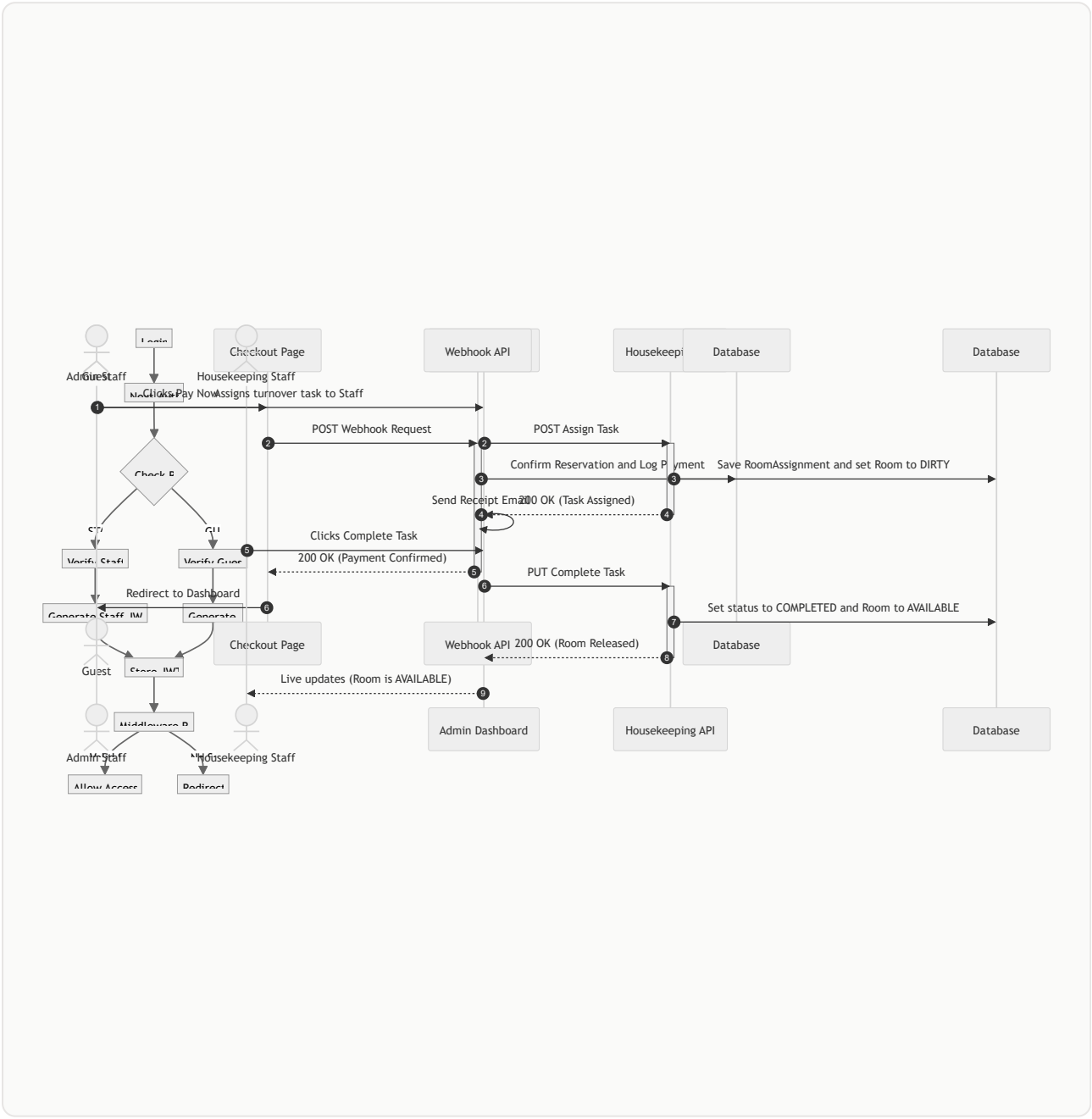
1. Guest Booking Flow

The booking flow handles availability checks, pricing calculation, add-on service selections, and reservation persistence.



2. Checkout & Simulated Payment Flow

Payments are simulated through a checkout invoice page that fires a mock transaction request to the webhook callback.



3. Admin & Housekeeping Task Management Flow

Rooms are cleaned and maintained through a live operations panel. Admins assign tasks, and staff members execute and mark them as complete.

4. Next-Auth Session Propagation Flow

Authentication is managed via Next-Auth JSON Web Tokens (JWT) to secure layouts and propagate roles.

File Documentation: schema.prisma

- **Relative Path:** `prisma/schema.prisma`
- **Line Count:** 179 lines

Purpose & Summary

PostgreSQL Database schema definition for Prisma ORM.

Architectural Role & Integration

Compiles to type-safe client methods and governs tables, models, relationships, and indices in Neon Postgres.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** No exports declared.

Source Code Implementation

```
// datasource configuration for Neon Serverless Postgres
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum RoomStatus {
  AVAILABLE
  OCCUPIED
  MAINTENANCE
  DIRTY
}

enum ReservationStatus {
  PENDING
  CONFIRMED
  CANCELED
}

enum PaymentStatus {
  PENDING
  COMPLETED
  FAILED
  REFUNDED
}

model Guest {
  id           String        @id @default(uuid()) @db.Uuid
  fullName     String        @map("full_name")
  email        String        @unique
  password     String
  idProofNum   String        @map("id_proof_num")
  phone        String
  nationality  String
  reservations Reservation[]
  payments     Payment[]

  @@map("guest")
}

model RoomType {
  id           String  @id @default(uuid()) @db.Uuid
  name         String
  description  String  @db.Text
  basePrice    Decimal @map("base_price") @db.Decimal(10, 2)
  maxOccupency Int     @map("max_occupency")
  rooms        Room[]

  @@map("room_type")
}

model Resort {
```

```
  id          String   @id @default(uuid()) @db.Uuid
  name        String
  description String   @db.Text
  location    String
  latitude    Float
  longitude   Float
  images      String[]
  rating      Float    @default(5.0)
  rooms       Room[]

  @@map("resort")
}

model Room {
  id              String           @id @default(uuid()) @db.Uuid
  resortId        String           @map("resort_id") @db.Uuid
  roomTypeId      String           @map("room_type_id") @db.Uuid
  roomNum         String           @unique @map("room_num")
  floor           String
  status          RoomStatus       @default(AVAILABLE)
  resort          Resort           @relation(fields: [resortId], references: [id], onDelete
  roomType        RoomType         @relation(fields: [roomTypeId], references: [id], onDele
  reservations    Reservation[]
  roomAssignments RoomAssignment[]

  @@map("room")
}

model Reservation {
  id                  String              @id @default(uuid()) @db.Uuid
  guestId             String              @map("guest_id") @db.Uuid
  roomId              String              @map("room_id") @db.Uuid
  checkIn             DateTime            @map("check_in") @db.Date
  checkOut            DateTime            @map("check_out") @db.Date
  numGuests           Int                 @map("num_guests")
  status              ReservationStatus   @default(PENDING)
  totalAmount         Decimal             @map("total_amount") @db.Decimal(10, 2)
  createdAt           DateTime            @default(now()) @map("created_at")
  guest               Guest               @relation(fields: [guestId], references: [id], onl
  room                Room                @relation(fields: [roomId], references: [id], onDe
  payments            Payment[]
  reservationServices ReservationService[]
  roomAssignments     RoomAssignment[]

  @@map("reservation")
}

model Payment {
  id            String        @id @default(uuid()) @db.Uuid
  reservationId String        @map("reservation_id") @db.Uuid
  guestId       String        @map("guest_id") @db.Uuid
  amount        Decimal       @db.Decimal(10, 2)
  method        String
  status        PaymentStatus @default(PENDING)
  paidAt        DateTime?     @map("paid_at")
  reservation   Reservation   @relation(fields: [reservationId], references: [id], onDelete
  guest         Guest         @relation(fields: [guestId], references: [id], onDelete: Casca

  @@map("payment")
}
```

```
model Department {
  id          String  @id @default(uuid()) @db.Uuid
  name        String
  managerName String  @map("manager_name")
  staffs      Staff[]

  @@map("department")
}

model Staff {
  id              String           @id @default(uuid()) @db.Uuid
  departmentId    String           @map("department_id") @db.Uuid
  fullName        String           @map("full_name")
  email           String           @unique
  password        String
  role            String
  shift           String
  department      Department       @relation(fields: [departmentId], references: [id], onDe
  services        Service[]
  roomAssignments RoomAssignment[]

  @@map("staff")
}

model Service {
  id                  String               @id @default(uuid()) @db.Uuid
  name                String
  category            String
  price               Decimal              @db.Decimal(10, 2)
  staffId             String?              @map("staff_id") @db.Uuid
  staff               Staff?               @relation(fields: [staffId], references: [id], o
  reservationServices ReservationService[]

  @@map("service")
}

model ReservationService {
  id            String      @id @default(uuid()) @db.Uuid
  reservationId String      @map("reservation_id") @db.Uuid
  serviceId     String      @map("service_id") @db.Uuid
  quantity      Int
  subtotal      Decimal     @db.Decimal(10, 2)
  reservation   Reservation @relation(fields: [reservationId], references: [id], onDelete: 
  service       Service     @relation(fields: [serviceId], references: [id], onDelete: Casc

  @@map("reservation_service")
}

model RoomAssignment {
  id            String      @id @default(uuid()) @db.Uuid
  reservationId String      @map("reservation_id") @db.Uuid
  roomId        String      @map("room_id") @db.Uuid
  staffId       String      @map("staff_id") @db.Uuid
  taskType      String      @map("task_type")
  status        String
  reservation   Reservation @relation(fields: [reservationId], references: [id], onDelete: 
  room          Room        @relation(fields: [roomId], references: [id], onDelete: Cascade
  staff         Staff       @relation(fields: [staffId], references: [id], onDelete: Cascad

  @@map("room_assignment")
}
```

File Documentation: seed.js

- **Relative Path:** `prisma/seed.js`
- **Line Count:** 316 lines

Purpose & Summary

Initial data seeder to hydrate the database during local setup.

Architectural Role & Integration

Generates defaults for departments, staff (including Admin), resorts, room types, and rooms.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** No exports declared.

Functions & APIs Specifications

Function: `U`

Inputs/Parameters: `password: string` - The plain-text password to hash.

Outputs/Returns: `string` - The securely hashed representation of the password.

Internal Logic & Purpose:

Synchronously hashes plain-text passwords using `bcryptjs.hashSync` with a default strength of 10 salt rounds. Primarily used to secure initial staff and guest credentials.

Actual Function Source Code:

```
const U = (id, w = 600) => `https://images.unsplash.com/photo-${id}?auto=format&fit=c
```

Function: `main`

Inputs/Parameters: None.

Outputs/Returns: `Promise` - Resolves when seeder completes.

Internal Logic & Purpose:

Truncates existings DB tables to ensure clean state. Generates pre-defined records for `Department`, `Staff` (including default Admin `admin@horizon.com` mapped to Management Department), `Resort` (3 major properties with accurate locations), `RoomType` (Suite vs Deluxe categories), and `Room` (12 physical rooms mapped to floors and resorts). Binds references securely.

Actual Function Source Code:

```
async function main() {
  console.log('🌍 Starting MEGA seed: 100+ resorts across 30 locations...\n');

  // Hash passwords
  const adminPassword = await bcrypt.hash('admin123', 10);
  const staffPassword = await bcrypt.hash('staff123', 10);
  const guestPassword = await bcrypt.hash('guest123', 10);

  // ── Clean existing data ──
  console.log('🧹 Cleaning existing data...');
  await prisma.roomAssignment.deleteMany();
  await prisma.reservationService.deleteMany();
  await prisma.payment.deleteMany();
  await prisma.reservation.deleteMany();
  await prisma.room.deleteMany();
  await prisma.resort.deleteMany();
  await prisma.service.deleteMany();
  await prisma.staff.deleteMany();
  await prisma.department.deleteMany();
  await prisma.roomType.deleteMany();
  await prisma.guest.deleteMany();

  // ── Departments ──
  console.log('🏢 Creating departments...');
  const adminDept = await prisma.department.create({ data: { name: 'Administration', 
  const hkDept = await prisma.department.create({ data: { name: 'Housekeeping', manag

  // ── Staff ──
  console.log('👤 Creating staff...');
  const adminUser = await prisma.staff.create({
    data: { fullName: 'Sarah Jenkins', email: 'admin@luxuryhorizon.com', password: ad
  });
  const staffUser = await prisma.staff.create({
    data: { fullName: 'John Cleaner', email: 'staff@luxuryhorizon.com', password: sta
  });

  // ── Guest ──
  console.log('👴 Creating guest...');
  await prisma.guest.create({
    data: { fullName: 'Faisal Dev', email: 'guest@gmail.com', password: guestPassword
  });

  // ── Room Types ──
  console.log('🛏️ Creating room types...');
  const deluxeType = await prisma.roomType.create({
    data: { name: 'Deluxe Suite', description: 'Elegant room with king-size bed, priv
  });
  const oceanType = await prisma.roomType.create({
    data: { name: 'Oceanfront Villa', description: 'Stunning ocean views, outdoor pri
  });
  const presidentialType = await prisma.roomType.create({
    data: { name: 'Presidential Suite', description: 'Multi-room layout, panoramic gl
  });
  const roomTypes = [deluxeType, oceanType, presidentialType];

  // ── Services ──
  console.log('💆 Creating services...');
  await prisma.service.create({ data: { name: 'Luxury Spa Massage', category: 'Wellne
  await prisma.service.create({ data: { name: 'Gourmet In-Room Dining', category: 'Di
  await prisma.service.create({ data: { name: 'VIP Airport Shuttle', category: 'Trans
```

```
  // ── Create 100+ Resorts with Rooms ──
  console.log(`\n🏨 Creating ${resortTemplates.length} resorts with rooms...\n`);
  let roomCounter = 100;

  for (let i = 0; i < resortTemplates.length; i++) {
    const t = resortTemplates[i];
    const loc = locations[t.locIdx];

    const resort = await prisma.resort.create({
      data: {
        name: t.name,
        description: t.desc,
        location: loc.loc,
        latitude: loc.lat + (Math.random() - 0.5) * 0.05,
        longitude: loc.lng + (Math.random() - 0.5) * 0.05,
        images: t.imgs,
        rating: t.rating,
      },
    });

    // Create 2-4 rooms per resort
    const numRooms = 2 + Math.floor(Math.random() * 3); // 2-4
    for (let r = 0; r < numRooms; r++) {
      roomCounter++;
      const floor = String(Math.floor(r / 2) + 1);
      const roomType = roomTypes[r % roomTypes.length];
      await prisma.room.create({
        data: {
          resortId: resort.id,
          roomTypeId: roomType.id,
          roomNum: `R${roomCounter}`,
          floor,
          status: 'AVAILABLE',
        },
      });
    }

    if ((i + 1) % 10 === 0) {
      console.log(`  ✓ ${i + 1}/${resortTemplates.length} resorts created`);
    }
  }

  console.log(`\n✅ Seed complete! Created ${resortTemplates.length} resorts across $
}
```

File Documentation: seed_100.js

- **Relative Path:** `prisma/seed_100.js`
- **Line Count:** 249 lines

Purpose & Summary

Advanced seeder generating over 100+ simulated records.

Architectural Role & Integration

Fills the database with multiple active/pending reservations, payments, and staff task assignments for comprehensive auditing tests.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** No exports declared.

Functions & APIs Specifications

Function: `getRandomImages`

Inputs/Parameters: `count: number` - Number of images to fetch.

Outputs/Returns: `string[]` - Array of Unsplash image URLs.

Internal Logic & Purpose:

> Queries a hardcoded registry of high-resolution resort assets and returns a randomized subset to keep testing visually rich.

Actual Function Source Code:

```
function getRandomImages(type) {
  if (type === 'mountain') return mountainImages;
  if (type === 'forest') return forestImages;
  return beachImages;
}
```

Function: `main`

Inputs/Parameters: None.

Outputs/Returns: `Promise` - Resolves when seeder completes.

Internal Logic & Purpose:

Truncates existings DB tables to ensure clean state. Generates pre-defined records for `Department`, `Staff` (including default Admin `admin@horizon.com` mapped to Management Department), `Resort` (3 major properties with accurate locations), `RoomType` (Suite vs Deluxe categories), and `Room` (12 physical rooms mapped to floors and resorts). Binds references securely.

Actual Function Source Code:

```
async function main() {
  console.log('Seeding 100 Resorts database...');

  // Hash passwords
  const adminPassword = await bcrypt.hash('admin123', 10);
  const staffPassword = await bcrypt.hash('staff123', 10);
  const guestPassword = await bcrypt.hash('guest123', 10);

  // 1. Create Departments
  console.log('Creating departments...');
  const adminDept = await prisma.department.create({
    data: { name: 'Administration', managerName: 'Sarah Jenkins' }
  });
  const hkDept = await prisma.department.create({
    data: { name: 'Housekeeping', managerName: 'Robert Dow' }
  });

  // 2. Create Staff & Admin
  console.log('Creating staff & admin...');
  const adminUser = await prisma.staff.create({
    data: {
      fullName: 'Sarah Jenkins',
      email: 'admin@luxuryhorizon.com',
      password: adminPassword,
      role: 'ADMIN',
      shift: 'Day',
      departmentId: adminDept.id
    }
  });
  const staffUser = await prisma.staff.create({
    data: {
      fullName: 'John Cleaner',
      email: 'staff@luxuryhorizon.com',
      password: staffPassword,
      role: 'STAFF',
      shift: 'Day',
      departmentId: hkDept.id
    }
  });

  // 3. Create Default Guest
  console.log('Creating a default guest...');
  const defaultGuest = await prisma.guest.create({
    data: {
      fullName: 'Faisal Dev',
      email: 'guest@gmail.com',
      password: guestPassword,
      idProofNum: 'ID-99281-US',
      phone: '+1 555 12345',
      nationality: 'American'
    }
  });

  // 4. Create Room Types
  console.log('Creating room types...');
  const deluxeType = await prisma.roomType.create({
    data: {
      name: 'Deluxe Suite',
      description: 'An elegant room featuring a king-size bed, private balcony, marbl
      basePrice: 250.00,
```

```
      maxOccupency: 2
    }
  });
  const oceanType = await prisma.roomType.create({
    data: {
      name: 'Oceanfront Villa',
      description: 'Stunning direct ocean views, outdoor private infinity pool, fully
      basePrice: 450.00,
      maxOccupency: 3
    }
  });
  const presidentialType = await prisma.roomType.create({
    data: {
      name: 'Presidential Suite',
      description: 'The pinnacle of luxury. Multi-room layout, panoramic floor-to-cei
      basePrice: 850.00,
      maxOccupency: 4
    }
  });

  // 5. Generate 100 Resorts
  console.log('Generating 100 resorts...');
  const resorts = [];
  let resortCount = 0;

  for (let i = 0; i < 100; i++) {
    const region = regions[i % regions.length];

    // Generate unique names
    const adj = adjectives[(i + 5) % adjectives.length];
    const noun = nouns[(i + 2) % nouns.length];
    const resortName = `${adj} ${noun} ${region.name}`;

    // Coordinates jitter
    const jitterLat = (Math.random() - 0.5) * 0.15;
    const jitterLng = (Math.random() - 0.5) * 0.15;
    const lat = region.lat + jitterLat;
    const lng = region.lng + jitterLng;

    const rating = parseFloat((4.3 + Math.random() * 0.7).toFixed(1));
    const images = getRandomImages(region.type);

    const resort = await prisma.resort.create({
      data: {
        name: resortName,
        description: `Experience pure luxury at ${resortName}. Nested in the premium
        location: `${region.name}, Coastal Sector ${i + 1}`,
        latitude: lat,
        longitude: lng,
        rating,
        images
      }
    });

    resorts.push(resort);
    resortCount++;

    // Create 3 rooms per resort (Deluxe, Oceanfront, Presidential)
    await prisma.room.create({
      data: {
        roomNum: `RM-${100 + i}-DLX`,
```

```
        floor: '1',
        roomTypeId: deluxeType.id,
        resortId: resort.id,
        status: 'AVAILABLE'
      }
    });

    await prisma.room.create({
      data: {
        roomNum: `RM-${200 + i}-OCN`,
        floor: '2',
        roomTypeId: oceanType.id,
        resortId: resort.id,
        status: 'AVAILABLE'
      }
    });

    await prisma.room.create({
      data: {
        roomNum: `RM-${300 + i}-PSD`,
        floor: '3',
        roomTypeId: presidentialType.id,
        resortId: resort.id,
        status: 'AVAILABLE'
      }
    });
  }

  console.log(`Successfully seeded ${resortCount} resorts and associated rooms.`);
```

File Documentation: page.tsx

- **Relative Path:** `src/app/about/page.tsx`
- **Line Count:** 175 lines

Purpose & Summary

Public about page detailing resort history, leadership, and contact forms.

Architectural Role & Integration

Provides visual anchors for footer navigation and showcases resort details.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `AboutPage`

Inputs/Parameters: None (React server/client component props).

Outputs/Returns: `JSX.Element` - Rendered about page UI.

Internal Logic & Purpose:

> Renders the static visual history, premium details of the Resort Room Booking and Management System property chain, and wraps contact form modules.

Actual Function Source Code:

```
export default function AboutPage() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [message, setMessage] = useState('');
  const [loading, setLoading] = useState(false);
  const [sentMsg, setSentMsg] = useState('');

  const handleContactSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setSentMsg('');

    try {
      // Simulate sending customer inquiry
      await new Promise(resolve => setTimeout(resolve, 1500));
      setSentMsg('Inquiry submitted. Our concierge team will reach out to you within
      setName('');
      setEmail('');
      setMessage('');
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="w-full min-h-screen bg-[#141414] text-[#E5E5E5] pb-20 pt-24 relat

      {/* Glow Backdrops */}
      <div className="absolute top-[20%] left-[-10%] w-[400px] h-[400px] bg-brand-acc
      <div className="absolute bottom-[20%] right-[-10%] w-[400px] h-[400px] bg-brand

      {/* 1. Header Banner */}
      <section className="py-16 px-4 sm:px-8 text-center space-y-4 relative z-10">
        <div className="inline-flex items-center gap-1.5 rounded-full bg-brand-accent
          <span>BOOKME Concierge</span>
        </div>
        <h1 className="font-heading text-4xl sm:text-6xl font-normal tracking-tight t
          Contact Customer Services
        </h1>
        <p className="mx-auto max-w-xl text-[#A0A0A0] text-xs sm:text-sm font-medium
          Our global operations team and resort concierges are available 24/7. Connec
        </p>
      </section>

      {/* 2. Main Content */}
      <section className="mx-auto max-w-5xl px-4 sm:px-6 lg:px-8 py-10 grid grid-cols

        {/* Left Column: Support Form */}
        <div className="bg-[#1A1A1A]/80 backdrop-blur-md rounded-[32px] p-8 border bo
          <div>
            <h2 className="font-sans text-xl font-bold text-white">Submit Support Tic
            <span className="text-[10px] text-[#8a8a8a] font-bold uppercase tracking-
          </div>

          {sentMsg && (
            <div className="rounded-xl bg-brand-accent/10 border border-brand-accent/
              <CheckCircle className="h-4.5 w-4.5 text-brand-accent shrink-0" />
              <span>{sentMsg}</span>
```

```
            </div>
          )}

          <form onSubmit={handleContactSubmit} className="space-y-4 text-xs">
            <div>
              <label className="block text-[#8a8a8a] uppercase mb-1.5 font-bold track
              <input
                type="text"
                required
                value={name}
                onChange={(e) => setName(e.target.value)}
                placeholder="e.g. Liam Gallagher"
                className="w-full rounded-xl bg-white/5 border border-white/5 py-3.5
              />
            </div>

            <div>
              <label className="block text-[#8a8a8a] uppercase mb-1.5 font-bold track
              <input
                type="email"
                required
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                placeholder="e.g. liam@oasis.com"
                className="w-full rounded-xl bg-white/5 border border-white/5 py-3.5
              />
            </div>

            <div>
              <label className="block text-[#8a8a8a] uppercase mb-1.5 font-bold track
              <textarea
                required
                rows={4}
                value={message}
                onChange={(e) => setMessage(e.target.value)}
                placeholder="Describe details of your transport options, custom dinin
                className="w-full rounded-xl bg-white/5 border border-white/5 py-3.5
              />
            </div>

            <button
              type="submit"
              disabled={loading}
              className="w-full rounded-xl bg-brand-accent hover:bg-brand-accent-hove
            >
              {loading ? (
                <>
                  <Loader2 className="h-4 w-4 animate-spin text-white" />
                  <span>Submitting request...</span>
                </>
              ) : (
                <>
                  <MessageSquare className="h-4 w-4" />
                  <span>Submit Inquiry</span>
                </>
              )}
            </button>
          </form>
        </div>

        {/* Right Column: Info cards */}
```

```
          <div className="space-y-6 flex flex-col justify-center">

            <div className="bg-[#1A1A1A]/60 backdrop-blur-sm rounded-2xl p-6 border bor
              <div className="h-12 w-12 rounded-xl bg-brand-accent/10 flex items-center
                <Phone className="h-5.5 w-5.5" />
              </div>
              <div>
                <span className="block text-[8px] text-[#8a8a8a] font-bold uppercase tr
                <span className="text-sm font-bold text-white">+1 800-BOOKME-NOW</span>
              </div>
            </div>

            <div className="bg-[#1A1A1A]/60 backdrop-blur-sm rounded-2xl p-6 border bor
              <div className="h-12 w-12 rounded-xl bg-brand-accent/10 flex items-center
                <Mail className="h-5.5 w-5.5" />
              </div>
              <div>
                <span className="block text-[8px] text-[#8a8a8a] font-bold uppercase tr
                <span className="text-sm font-bold text-white">concierge@bookme.com</sp
              </div>
            </div>

            <div className="bg-[#1A1A1A]/60 backdrop-blur-sm rounded-2xl p-6 border bor
              <div className="h-12 w-12 rounded-xl bg-brand-accent/10 flex items-center
                <MapPin className="h-5.5 w-5.5" />
              </div>
              <div>
                <span className="block text-[8px] text-[#8a8a8a] font-bold uppercase tr
                <span className="text-sm font-bold text-white">750 Luxury Promenade, Ma
              </div>
```

Function: `handleContactSubmit`

Inputs/Parameters: `e: React.FormEvent` - Standard form submit event.

Outputs/Returns: `void` - Updates local feedback states.

Internal Logic & Purpose:

Intercepts the public contact form submit. Simulates submission delay, triggers visual confirmation cards, and resets inputs.

Actual Function Source Code:

```
  const handleContactSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setSentMsg('');

    try {
      // Simulate sending customer inquiry
      await new Promise(resolve => setTimeout(resolve, 1500));
      setSentMsg('Inquiry submitted. Our concierge team will reach out to you within
      setName('');
      setEmail('');
      setMessage('');
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/admin/bookings/route.ts`
- **Line Count:** 196 lines

Purpose & Summary

Administrative reservation desk handler. Fetches all bookings, processes check-ins, check-outs, and mid-stay prorated refunds.

Architectural Role & Integration

Restricted to type 'staff' and role 'ADMIN' sessions. Operates inside transactional blocks to prevent double-booking.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **2** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**, **PUT**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const reservations = await prisma.reservation.findMany({
      include: {
        guest: {
          select: {
            id: true,
            fullName: true,
            email: true,
            phone: true
          }
        },
        room: {
          include: {
            roomType: true,
            resort: true
          }
        },
        payments: true
      },
      orderBy: {
        createdAt: 'desc'
      }
    });

    return NextResponse.json(reservations);
  } catch (error: any) {
    console.error('Admin Fetch Bookings API Error:', error);
    return NextResponse.json({ error: error.message || 'Internal server error' }, { s
  }
}
```

Function: `PUT`

Inputs/Parameters: `req: Request` - Encapsulates JSON body containing `reservationId` and `action` ('CHECK_IN' | 'CHECK_OUT' | 'CANCEL_MID_STAY').

Outputs/Returns: `NextResponse` - Confirmation message or `400/401/404` errors.

Internal Logic & Purpose:

Runs RBAC staff credentials checking. Modifies reservation states via isolated DB transactions (`prisma.$transaction`): - **CHECK_IN:** Shifts reservation status to `CONFIRMED` and Room status to `OCCUPIED`. - **CHECK_OUT:** Releases Room status to `DIRTY` to prompt housekeeping assignments. - **CANCEL_MID_STAY:** Restricts action to confirmed, ongoing dates. Calculates elapsed nights stayed (min 1). Computes prorated pricing, adjusts original payment log, releases Room status to `DIRTY`, and creates a new refund record indicating `REFUNDED` status, preserving financial audit trials.

Actual Function Source Code:

```
export async function PUT(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { reservationId, action } = await req.json();

    if (!reservationId || !action) {
      return NextResponse.json({ error: 'Missing reservationId or action parameter.'
    }

    const reservation = await prisma.reservation.findUnique({
      where: { id: reservationId },
      include: { room: { include: { roomType: true } }, payments: true }
    });

    if (!reservation) {
      return NextResponse.json({ error: 'Reservation not found.' }, { status: 404 });
    }

    // Action 1: CHECK_IN
    if (action === 'CHECK_IN') {
      if (reservation.status === 'CANCELED') {
        return NextResponse.json({ error: 'Cannot check-in a canceled reservation.' }
      }

      await prisma.$transaction(async (tx) => {
        // Update reservation to confirmed if pending
        await tx.reservation.update({
          where: { id: reservationId },
          data: { status: 'CONFIRMED' }
        });

        // Set room status to OCCUPIED
        await tx.room.update({
          where: { id: reservation.roomId },
          data: { status: 'OCCUPIED' }
        });
      });

      return NextResponse.json({ message: 'Guest checked in successfully.' });
    }

    // Action 2: CHECK_OUT
    if (action === 'CHECK_OUT') {
      await prisma.$transaction(async (tx) => {
        // Set room status to DIRTY (so housekeeping will turnover)
        await tx.room.update({
          where: { id: reservation.roomId },
          data: { status: 'DIRTY' }
        });
      });

      return NextResponse.json({ message: 'Guest checked out successfully. Room relea
    }

    // Action 3: CANCEL_MID_STAY
    if (action === 'CANCEL_MID_STAY') {
```

```
        if (reservation.status !== 'CONFIRMED') {
          return NextResponse.json({ error: 'Only confirmed and active stays can be can
        }

        const checkInTime = new Date(reservation.checkIn).getTime();
        const checkOutTime = new Date(reservation.checkOut).getTime();
        const now = new Date();
        const nowTime = now.getTime();

        if (nowTime < checkInTime || nowTime > checkOutTime) {
          return NextResponse.json({ error: 'Current date is outside the reservation st
        }

        // Calculate nights stayed
        const msPerDay = 1000 * 60 * 60 * 24;
        const totalNights = Math.ceil(Math.abs(checkOutTime - checkInTime) / msPerDay);

        // Nights stayed is days elapsed from check-in to today, minimum 1 night
        let nightsStayed = Math.ceil(Math.abs(nowTime - checkInTime) / msPerDay);
        nightsStayed = Math.max(1, Math.min(nightsStayed, totalNights));

        const remainingNights = totalNights - nightsStayed;

        if (remainingNights <= 0) {
          return NextResponse.json({ error: 'No remaining nights left to refund. Please
        }

        // Prorate calculations
        const totalAmountNum = Number(reservation.totalAmount);
        const costPerNight = totalAmountNum / totalNights;
        const proratedTotal = costPerNight * nightsStayed;
        const refundAmount = totalAmountNum - proratedTotal;

        const newCheckOutDate = new Date(checkInTime + (nightsStayed * msPerDay));

        // Update DB in a transaction
        await prisma.$transaction(async (tx) => {
          // 1. Update reservation dates, status and amount
          await tx.reservation.update({
            where: { id: reservationId },
            data: {
              checkOut: newCheckOutDate,
              totalAmount: proratedTotal,
              status: 'CANCELED' // Set status to CANCELED representing truncated stay
            }
          });

          // 2. Set room status to DIRTY
          await tx.room.update({
            where: { id: reservation.roomId },
            data: { status: 'DIRTY' }
          });

          // 3. Process refund payments
          const primaryPayment = reservation.payments.find(p => p.status === 'COMPLETED
          if (primaryPayment) {
            // Adjust original payment to the prorated amount
            await tx.payment.update({
              where: { id: primaryPayment.id },
              data: { amount: proratedTotal }
            });
```

```
            // Create a refund transaction log
            await tx.payment.create({
              data: {
                reservationId,
                guestId: reservation.guestId,
                amount: refundAmount,
                method: primaryPayment.method,
                status: 'REFUNDED',
                paidAt: new Date()
              }
            });
          }
        });

        return NextResponse.json({
          message: 'Mid-stay cancellation completed successfully.',
          nightsStayed,
          remainingNights,
          proratedTotal,
          refundAmount
        });
      }

      return NextResponse.json({ error: 'Invalid action parameter.' }, { status: 400 })
    } catch (error: any) {
      console.error('Admin Perform Action API Error:', error);
      return NextResponse.json({ error: error.message || 'Internal server error' }, { s
    }
  }
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/admin/departments/route.ts`
- **Line Count:** 71 lines

Purpose & Summary

CRUD actions for administration department structures.

Architectural Role & Integration

Supports listing, creating, and deleting property divisions (e.g. Spa, Food, Front Desk) for staff allocation.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **3** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**, **POST**, **DELETE**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const departments = await prisma.department.findMany({
      include: {
        staffs: true
      },
      orderBy: { name: 'asc' }
    });

    return NextResponse.json(departments);
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

Function: `POST`

Inputs/Parameters: `req: Request` - JSON payload mapping input variables.

Outputs/Returns: `NextResponse` - Created model details or error.

Internal Logic & Purpose:

Verifies permissions. Hashes passwords (using bcryptjs) and creates a new Department, Staff, Guest, or RoomAssignment record in PostgreSQL.

Actual Function Source Code:

```
export async function POST(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { name, managerName } = await req.json();
    if (!name || !managerName) {
      return NextResponse.json({ error: 'Missing required fields' }, { status: 400 })
    }

    const department = await prisma.department.create({
      data: { name, managerName }
    });

    return NextResponse.json(department);
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

Function: `DELETE`

Inputs/Parameters: `req: Request` - Target record ID to remove.

Outputs/Returns: `NextResponse` - Confirmation indicators.

Internal Logic & Purpose:

Verifies permissions. Executes deletion of the target department, staff, or reservation record.

Actual Function Source Code:

```
export async function DELETE(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { searchParams } = new URL(req.url);
    const id = searchParams.get('id');

    if (!id) {
      return NextResponse.json({ error: 'Missing department ID' }, { status: 400 });
    }

    await prisma.department.delete({
      where: { id }
    });

    return NextResponse.json({ message: 'Department deleted successfully' });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/admin/staff/route.ts`
- **Line Count:** 88 lines

Purpose & Summary

CRUD actions for staff registry.

Architectural Role & Integration

Allows admins to create new staff accounts with hashes (bcryptjs) and assign them to departments and shifts.

File Structure & Dependencies

- **Imports:** Tracks **5** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **3** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**, **POST**, **DELETE**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const staffList = await prisma.staff.findMany({
      include: {
        department: true
      },
      orderBy: { fullName: 'asc' }
    });

    return NextResponse.json(staffList);
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

Function: `POST`

Inputs/Parameters: `req: Request` - JSON payload mapping input variables.

Outputs/Returns: `NextResponse` - Created model details or error.

Internal Logic & Purpose:

Verifies permissions. Hashes passwords (using bcryptjs) and creates a new Department, Staff, Guest, or RoomAssignment record in PostgreSQL.

Actual Function Source Code:

```
export async function POST(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { fullName, email, password, role, shift, departmentId } = await req.json()
    if (!fullName || !email || !password || !role || !shift || !departmentId) {
      return NextResponse.json({ error: 'Missing required fields' }, { status: 400 })
    }

    const existingStaff = await prisma.staff.findUnique({
      where: { email }
    });
    if (existingStaff) {
      return NextResponse.json({ error: 'Email already registered' }, { status: 400 }
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    const newStaff = await prisma.staff.create({
      data: {
        fullName,
        email,
        password: hashedPassword,
        role,
        shift,
        departmentId
      }
    });

    return NextResponse.json(newStaff);
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

Function: `DELETE`

Inputs/Parameters: `req: Request` - Target record ID to remove.

Outputs/Returns: `NextResponse` - Confirmation indicators.

Internal Logic & Purpose:

Verifies permissions. Executes deletion of the target department, staff, or reservation record.

Actual Function Source Code:

```
export async function DELETE(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { searchParams } = new URL(req.url);
    const id = searchParams.get('id');

    if (!id) {
      return NextResponse.json({ error: 'Missing staff ID' }, { status: 400 });
    }

    await prisma.staff.delete({
      where: { id }
    });

    return NextResponse.json({ message: 'Staff deleted successfully' });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/auth/[...nextauth]/route.ts`
- **Line Count:** 7 lines

Purpose & Summary

Authentication gateway router mapping credentials endpoints.

Architectural Role & Integration

Next-Auth wrapper executing authorization logic from src/lib/auth.ts.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Source Code Implementation

```
import NextAuth from "next-auth";
import { authOptions } from "@/lib/auth";

const handler = NextAuth(authOptions);

export { handler as GET, handler as POST };
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/book/options/route.ts`
- **Line Count:** 13 lines

Purpose & Summary

API endpoint listing available property add-on services.

Architectural Role & Integration

Fetched during the booking details step to bundle spa, private dining, or tour excursions.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const roomTypes = await prisma.roomType.findMany();
    const services = await prisma.service.findMany();
    return NextResponse.json({ roomTypes, services });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/book/route.ts`
- **Line Count:** 196 lines

Purpose & Summary

Primary booking creator. Computes costs, verifies conflicts, and creates reservations.

Architectural Role & Integration

Requires guest authentication. Automatically blocks conflicting schedules and calculates totals based on nights stayed.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **2** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **POST**, **DELETE**

Functions & APIs Specifications

Function: `POST`

Inputs/Parameters: `req: Request` - JSON payload mapping input variables.

Outputs/Returns: `NextResponse` - Created model details or error.

Internal Logic & Purpose:

> Verifies permissions. Hashes passwords (using bcryptjs) and creates a new Department, Staff, Guest, or RoomAssignment record in PostgreSQL.

Actual Function Source Code:

```
export async function POST(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'guest') {
      return NextResponse.json({ error: 'Please sign in as a Guest to book a room.' }
    }

    const guestId = (session.user as any).id;
    const { roomTypeId, checkIn, checkOut, numGuests, serviceIds } = await req.json()

    if (!roomTypeId || !checkIn || !checkOut || !numGuests) {
      return NextResponse.json({ error: 'Missing required parameters.' }, { status: 4
    }

    const checkInDate = new Date(checkIn);
    const checkOutDate = new Date(checkOut);

    if (checkInDate >= checkOutDate) {
      return NextResponse.json({ error: 'Check-out date must be after check-in date.'
    }

    // 1. Find all rooms of this type
    const rooms = await prisma.room.findMany({
      where: { roomTypeId },
    });

    if (rooms.length === 0) {
      return NextResponse.json({ error: 'No rooms available for this room type.' }, {
    }

    // 2. Search for a room without scheduling conflicts
    let selectedRoom = null;
    for (const r of rooms) {
      const conflicts = await prisma.reservation.findMany({
        where: {
          roomId: r.id,
          status: { in: ['CONFIRMED', 'PENDING'] },
          NOT: {
            OR: [
              { checkOut: { lte: checkInDate } },
              { checkIn: { gte: checkOutDate } },
            ],
          },
        },
      });

      if (conflicts.length === 0) {
        selectedRoom = r;
        break;
      }
    }

    if (!selectedRoom) {
      return NextResponse.json({ error: 'All rooms of this category are occupied duri
    }

    // Calculate nights
    const diffTime = Math.abs(checkOutDate.getTime() - checkInDate.getTime());
    const nights = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
```

```
    // Get room type details
    const roomType = await prisma.roomType.findUnique({ where: { id: roomTypeId } });
    if (!roomType) return NextResponse.json({ error: 'Room type not found.' }, { stat

    // Calculate base stay price
    let basePriceNum = Number(roomType.basePrice);
    let totalStayPrice = basePriceNum * nights;

    // Retrieve selected services
    const services = await prisma.service.findMany({
      where: { id: { in: serviceIds || [] } },
    });

    let addOnTotal = 0;
    const servicesPayload = services.map(s => {
      const sub = Number(s.price) * nights;
      addOnTotal += sub;
      return {
        serviceId: s.id,
        quantity: nights,
        subtotal: sub,
      };
    });

    const totalAmount = totalStayPrice + addOnTotal;

    // Create the PENDING reservation
    const reservation = await prisma.reservation.create({
      data: {
        guestId,
        roomId: selectedRoom.id,
        checkIn: checkInDate,
        checkOut: checkOutDate,
        numGuests: parseInt(numGuests),
        status: 'PENDING',
        totalAmount,
        reservationServices: {
          create: servicesPayload,
        },
      },
    });

    return NextResponse.json({
      message: 'Draft reservation created successfully',
      reservationId: reservation.id,
    });
  } catch (error: any) {
    console.error('Reservation API Error:', error);
    return NextResponse.json({ error: error.message || 'Internal server error' }, { s
  }
}
```

Function: `DELETE`

Inputs/Parameters: `req: Request` - Target record ID to remove.

Outputs/Returns: `NextResponse` - Confirmation indicators.

Internal Logic & Purpose:

Verifies permissions. Executes deletion of the target department, staff, or reservation record.

Actual Function Source Code:

```
export async function DELETE(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session) {
      return NextResponse.json({ error: 'Unauthorized access.' }, { status: 401 });
    }

    const userType = (session.user as any).type;
    const userRole = (session.user as any).role;

    if (userType !== 'guest' && userType !== 'staff') {
      return NextResponse.json({ error: 'Unauthorized access.' }, { status: 401 });
    }

    const { searchParams } = new URL(req.url);
    const reservationId = searchParams.get('id');

    if (!reservationId) {
      return NextResponse.json({ error: 'Missing reservation identifier.' }, { status
    }

    const reservation = await prisma.reservation.findUnique({
      where: { id: reservationId },
      include: { payments: true }
    });

    if (!reservation) {
      return NextResponse.json({ error: 'Reservation not found.' }, { status: 404 });
    }

    // Guest users can only delete/cancel their own reservations
    if (userType === 'guest' && reservation.guestId !== (session.user as any).id) {
      return NextResponse.json({ error: 'Forbidden: You do not own this reservation.'
    }

    if (reservation.status === 'CANCELED') {
      return NextResponse.json({ error: 'Reservation is already canceled.' }, { statu
    }

    // Check 7-day cancellation boundary only for guests and only if booking is CONFI
    if (userType === 'guest' && reservation.status === 'CONFIRMED') {
      const checkInTime = new Date(reservation.checkIn).getTime();
      const currentTime = Date.now();
      const timeDiff = checkInTime - currentTime;
      const sevenDaysInMs = 7 * 24 * 60 * 60 * 1000;

      if (timeDiff < sevenDaysInMs) {
        return NextResponse.json({
          error: 'Cancellation window expired. Paid bookings can only be canceled at
        }, { status: 400 });
      }
    }

    // Update reservation status and associated payments to REFUNDED
    const updated = await prisma.$transaction(async (tx) => {
      const res = await tx.reservation.update({
        where: { id: reservationId },
        data: { status: 'CANCELED' }
      });
```

```
        // Update associated payments to REFUNDED
        await tx.payment.updateMany({
          where: { reservationId },
          data: { status: 'REFUNDED' }
        });

        return res;
      });

      return NextResponse.json({
        message: 'Reservation canceled and refunded successfully.',
        reservation: updated
      });
    } catch (error: any) {
      console.error('Cancel Reservation API Error:', error);
      return NextResponse.json({ error: error.message || 'Internal server error' }, { s
    }
  }
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/checkout/[id]/route.ts`
- **Line Count:** 32 lines

Purpose & Summary

Invoicing API fetching pricing summaries for specific reservations.

Architectural Role & Integration

Called during the payment page load to fetch total amounts, assigned room numbers, and dates.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET(req: Request, { params }: { params: Promise<{ id: string }>
  try {
    const { id } = await params;
    const res = await prisma.reservation.findUnique({
      where: { id },
      include: {
        room: {
          include: { roomType: true }
        }
      }
    });

    if (!res) {
      return NextResponse.json({ error: 'Reservation not found.' }, { status: 404 });
    }

    return NextResponse.json({
      id: res.id,
      checkIn: res.checkIn,
      checkOut: res.checkOut,
      totalAmount: res.totalAmount,
      roomNum: res.room.roomNum,
      roomTypeName: res.room.roomType.name,
    });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/dashboard/admin/route.ts`
- **Line Count:** 67 lines

Purpose & Summary

Admin dashboard stats compiler fetching financial metrics and housekeeping queues.

Architectural Role & Integration

Restricted to Admin staff. Aggregates room statuses, staff shifts, and total billing amounts.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    // 1. Calculate occupancy & stats
    const totalRooms = await prisma.room.count();
    const occupiedRooms = await prisma.room.count({ where: { status: 'OCCUPIED' } });
    const dirtyRooms = await prisma.room.count({ where: { status: 'DIRTY' } });
    const maintenanceRooms = await prisma.room.count({ where: { status: 'MAINTENANCE'
    const availableRooms = await prisma.room.count({ where: { status: 'AVAILABLE' } }

    // 2. Financials
    const payments = await prisma.payment.findMany({
      include: { guest: true },
      orderBy: { paidAt: 'desc' },
      take: 20
    });

    const totalRevenue = payments
      .filter(p => p.status === 'COMPLETED')
      .reduce((sum, p) => sum + Number(p.amount), 0);

    // 3. Room listings
    const rooms = await prisma.room.findMany({
      include: { roomType: true },
      orderBy: { roomNum: 'asc' }
    });

    // 4. Staff listings
    const staffList = await prisma.staff.findMany({
      include: { department: true }
    });

    // 5. Active reservations
    const reservations = await prisma.reservation.findMany({
      include: { guest: true, room: true },
      orderBy: { createdAt: 'desc' },
      take: 10
    });

    return NextResponse.json({
      stats: {
        totalRooms,
        occupiedRooms,
        dirtyRooms,
        maintenanceRooms,
        availableRooms,
        totalRevenue,
      },
      payments,
      rooms,
      staffList,
      reservations
    });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
```

```
    }
  }
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/dashboard/guest/route.ts`
- **Line Count:** 31 lines

Purpose & Summary

Guest dashboard loader retrieving reservation history.

Architectural Role & Integration

Fetches past stays, pending check-ins, and service receipts for the authenticated guest.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'guest') {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const guestId = (session.user as any).id;

    const reservations = await prisma.reservation.findMany({
      where: { guestId },
      include: {
        room: {
          include: { roomType: true }
        },
        payments: true
      },
      orderBy: { createdAt: 'desc' }
    });

    return NextResponse.json({ reservations });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/dashboard/staff/route.ts`
- **Line Count:** 33 lines

Purpose & Summary

Staff dashboard task compiler.

Architectural Role & Integration

Fetches task allocations (RoomAssignments) for cleaning and maintenance assigned to the active staff user.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff') {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const staffId = (session.user as any).id;

    const assignments = await prisma.roomAssignment.findMany({
      where: { staffId },
      include: {
        room: {
          include: { roomType: true }
        },
        reservation: {
          include: { guest: true }
        }
      },
      orderBy: { id: 'desc' }
    });

    return NextResponse.json({ assignments });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/guest/room-types/route.ts`
- **Line Count:** 12 lines

Purpose & Summary

Public endpoint listing room types and descriptions.

Architectural Role & Integration

Provides data for interactive room selection tabs on the landing page and browse list.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const roomTypes = await prisma.roomType.findMany();
    return NextResponse.json(roomTypes);
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/housekeeping/route.ts`
- **Line Count:** 137 lines

Purpose & Summary

Housekeeping operations center. Wires task assignments and completions.

Architectural Role & Integration

POST (Admin assigns a task to staff, locking room as DIRTY). PUT (Staff completes a task, releasing room as AVAILABLE).

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **3** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**, **POST**, **PUT**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET() {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff') {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const rooms = await prisma.room.findMany({
      include: { roomType: true },
      orderBy: { roomNum: 'asc' }
    });

    const staffList = await prisma.staff.findMany({
      select: { id: true, fullName: true, role: true }
    });

    return NextResponse.json({ rooms, staffList });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

Function: `POST`

Inputs/Parameters: `req: Request` - JSON payload mapping input variables.

Outputs/Returns: `NextResponse` - Created model details or error.

Internal Logic & Purpose:

Verifies permissions. Hashes passwords (using bcryptjs) and creates a new Department, Staff, Guest, or RoomAssignment record in PostgreSQL.

Actual Function Source Code:

```
export async function POST(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff' || (session.user as any).r
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { roomId, staffId, taskType } = await req.json();

    if (!roomId || !staffId || !taskType) {
      return NextResponse.json({ error: 'Missing required parameters' }, { status: 40
    }

    // Find if there is an active reservation for this room to link it
    const activeRes = await prisma.reservation.findFirst({
      where: { roomId, status: 'CONFIRMED' },
      orderBy: { checkIn: 'desc' }
    });

    let finalReservationId = activeRes?.id;
    if (!finalReservationId) {
      const roomRes = await prisma.reservation.findFirst({ where: { roomId } });
      finalReservationId = roomRes?.id;
    }
    if (!finalReservationId) {
      const anyRes = await prisma.reservation.findFirst();
      finalReservationId = anyRes?.id;
    }

    if (!finalReservationId) {
      return NextResponse.json({ error: 'No reservations exist in the database. House
    }

    // Create RoomAssignment
    const assignment = await prisma.roomAssignment.create({
      data: {
        roomId,
        staffId,
        taskType,
        status: 'PENDING',
        reservationId: finalReservationId
      }
    });

    // Update Room status to DIRTY or MAINTENANCE if relevant to cleanliness or repai
    let nextStatus = null;
    if (taskType === 'Repair') {
      nextStatus = 'MAINTENANCE';
    } else if (taskType === 'Turnover Cleaning' || taskType === 'Deep Sweep') {
      nextStatus = 'DIRTY';
    }

    if (nextStatus) {
      await prisma.room.update({
        where: { id: roomId },
        data: { status: nextStatus as any }
      });
    }

    return NextResponse.json({ message: 'Housekeeping task assigned successfully', as
```

```
    } catch (error: any) {
      console.error('Assign task error:', error);
      return NextResponse.json({ error: error.message }, { status: 500 });
    }
  }
```

Function: `PUT`

Inputs/Parameters: `req: Request` - Encapsulates JSON body containing `reservationId` and `action` ('CHECK_IN' | 'CHECK_OUT' | 'CANCEL_MID_STAY').

Outputs/Returns: `NextResponse` - Confirmation message or `400/401/404` errors.

Internal Logic & Purpose:

Runs RBAC staff credentials checking. Modifies reservation states via isolated DB transactions (`prisma.$transaction`): - **CHECK_IN:** Shifts reservation status to `CONFIRMED` and Room status to `OCCUPIED`. - **CHECK_OUT:** Releases Room status to `DIRTY` to prompt housekeeping assignments. - **CANCEL_MID_STAY:** Restricts action to confirmed, ongoing dates. Calculates elapsed nights stayed (min 1). Computes prorated pricing, adjusts original payment log, releases Room status to `DIRTY`, and creates a new refund record indicating `REFUNDED` status, preserving financial audit trials.

Actual Function Source Code:

```
export async function PUT(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || (session.user as any).type !== 'staff') {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { assignmentId } = await req.json();

    if (!assignmentId) {
      return NextResponse.json({ error: 'Missing assignmentId' }, { status: 400 });
    }

    // Find the assignment
    const assignment = await prisma.roomAssignment.findUnique({
      where: { id: assignmentId }
    });

    if (!assignment) {
      return NextResponse.json({ error: 'Assignment not found' }, { status: 404 });
    }

    // Update RoomAssignment status to COMPLETED
    await prisma.roomAssignment.update({
      where: { id: assignmentId },
      data: { status: 'COMPLETED' }
    });

    // Reset Room status to AVAILABLE
    await prisma.room.update({
      where: { id: assignment.roomId },
      data: { status: 'AVAILABLE' }
    });

    return NextResponse.json({ message: 'Task completed. Room released to AVAILABLE.'
  } catch (error: any) {
    console.error('Complete task error:', error);
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/resorts/route.ts`
- **Line Count:** 98 lines

Purpose & Summary

Retrieves lists of resorts with sorting, filtering, and pagination.

Architectural Role & Integration

Supports the landing page map coordinates display and the search browser grid.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **GET**

Functions & APIs Specifications

Function: `GET`

Inputs/Parameters: None (Reads cookie session automatically).

Outputs/Returns: `NextResponse` - JSON array of bookings or `401` error.

Internal Logic & Purpose:

> Verifies session token role structure. Queries all `reservation` records including deep maps for `guest`, `room`, `roomType`, `resort`, and `payments`, ordering entries by `createdAt` descending.

Actual Function Source Code:

```
export async function GET(req: Request) {
  const { searchParams } = new URL(req.url);
  const page = parseInt(searchParams.get('page') || '1');
  const limit = parseInt(searchParams.get('limit') || '6');
  const query = searchParams.get('query') || '';
  const type = searchParams.get('type') || 'all';

  const offset = (page - 1) * limit;

  // Build filter options
  const filter: any = {};

  if (query) {
    filter.OR = [
      { name: { contains: query, mode: 'insensitive' } },
      { location: { contains: query, mode: 'insensitive' } },
      { description: { contains: query, mode: 'insensitive' } }
    ];
  }

  // Filter based on keywords in location/name
  if (type !== 'all') {
    let keywords: string[] = [];
    if (type === 'tropical') keywords = ['Bali', 'Hawaii', 'Fiji', 'Maldives'];
    else if (type === 'alpine') keywords = ['Alps', 'Aspen', 'Swiss'];
    else if (type === 'coastal') keywords = ['Amalfi', 'Santorini', 'Bahamas'];
    else if (type === 'forest') keywords = ['Kyoto', 'Forest', 'Eco'];

    if (keywords.length > 0) {
      filter.OR = keywords.map(kw => ({
        name: { contains: kw, mode: 'insensitive' }
      }));
    }
  }

  // Unique cache key based on query parameters
  const cacheKey = `resorts:${page}:${limit}:${query}:${type}`;

  try {
    const result = await cache.getOrSet(
      cacheKey,
      async () => {
        const [resorts, total] = await Promise.all([
          prisma.resort.findMany({
            where: filter,
            skip: offset,
            take: limit,
            orderBy: { rating: 'desc' },
            include: {
              rooms: {
                include: {
                  roomType: true
                }
              }
            }
          }),
          prisma.resort.count({ where: filter })
        ]);

        const serializedResorts = resorts.map(resort => ({
```

```
            ...resort,
            latitude: Number(resort.latitude),
            longitude: Number(resort.longitude),
            rooms: resort.rooms.map(room => ({
              ...room,
              roomType: {
                ...room.roomType,
                basePrice: Number(room.roomType.basePrice)
              }
            }))
          }));

          return {
            resorts: serializedResorts,
            total,
            page,
            totalPages: Math.ceil(total / limit)
          };
        },
        120 // Cache for 2 minutes — resort data rarely changes
      );

      // Add CDN cache headers for Vercel Edge Network
      return NextResponse.json(result, {
        headers: {
          'Cache-Control': 'public, s-maxage=60, stale-while-revalidate=300',
        },
      });
    } catch (error) {
      console.error('Error fetching resorts:', error);
      return NextResponse.json({ error: 'Failed to load resorts.' }, { status: 500 });
    }
  }
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/signup/route.ts`
- **Line Count:** 37 lines

Purpose & Summary

Guest registration gateway.

Architectural Role & Integration

Creates a new guest profile, hashes the password with bcrypt, and checks for unique email constraints.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **POST**

Functions & APIs Specifications

Function: `POST`

Inputs/Parameters: `req: Request` - JSON payload mapping input variables.

Outputs/Returns: `NextResponse` - Created model details or error.

Internal Logic & Purpose:

Verifies permissions. Hashes passwords (using bcryptjs) and creates a new Department, Staff, Guest, or RoomAssignment record in PostgreSQL.

Actual Function Source Code:

```
export async function POST(req: Request) {
  try {
        const { fullName, email, password } = await req.json();

    if (!fullName || !email || !password) {
      return NextResponse.json({ error: "Missing required fields" }, { status: 400 })
    }

    const existingGuest = await prisma.guest.findUnique({ where: { email } });
    if (existingGuest) {
      return NextResponse.json({ error: "Email already registered" }, { status: 400 }
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    const newGuest = await prisma.guest.create({
      data: {
        fullName,
        email,
        password: hashedPassword,
        phone: "",
        nationality: "",
        idProofNum: "",
      },
    });

    return NextResponse.json({ message: "Guest created successfully", guestId: newGue
  } catch (error: any) {
    console.error("Signup error:", error);
    return NextResponse.json({ error: error.message || "Internal server error" }, { s
  }
}
```

File Documentation: route.ts

- **Relative Path:** `src/app/api/webhooks/dodo/route.ts`
- **Line Count:** 114 lines

Purpose & Summary

Simulated payment settlement webhook.

Architectural Role & Integration

Validates payment states, confirms reservations, logs transactional history, and fires email receipts.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Route Handlers Supported

- Supported HTTP Methods: **POST**

Functions & APIs Specifications

Function: `POST`

Inputs/Parameters: `req: Request` - JSON payload mapping input variables.

Outputs/Returns: `NextResponse` - Created model details or error.

Internal Logic & Purpose:

> Verifies permissions. Hashes passwords (using bcryptjs) and creates a new Department, Staff, Guest, or RoomAssignment record in PostgreSQL.

Actual Function Source Code:

```
export async function POST(req: Request) {
  try {
    const { reservationId, status, method } = await req.json();

    if (!reservationId || !status) {
      return NextResponse.json({ error: 'Missing reservationId or status.' }, { statu
    }

    // Retrieve reservation details
    const reservation = await prisma.reservation.findUnique({
      where: { id: reservationId },
      include: {
        guest: true,
        room: {
          include: { roomType: true }
        },
        reservationServices: {
          include: { service: true }
        }
      }
    });

    if (!reservation) {
      return NextResponse.json({ error: 'Reservation not found.' }, { status: 404 });
    }

    if (status === 'COMPLETED') {
      // 1. Update Reservation status to CONFIRMED
      await prisma.reservation.update({
        where: { id: reservationId },
        data: { status: 'CONFIRMED' },
      });

      // 2. Create Payment record
      await prisma.payment.create({
        data: {
          reservationId,
          guestId: reservation.guestId,
          amount: reservation.totalAmount,
          method: method || 'Simulated Card',
          status: 'COMPLETED',
          paidAt: new Date(),
        },
      });

      // 3. Assemble and dispatch Nodemailer HTML receipt email
      const checkInFormatted = new Date(reservation.checkIn).toLocaleDateString();
      const checkOutFormatted = new Date(reservation.checkOut).toLocaleDateString();
      const servicesHtml = reservation.reservationServices.map(rs =>
        `<li>${rs.service.name} (${rs.service.category}): \${Number(rs.subtotal).toFi
      ).join('');

      const htmlBody = `
        <div style="font-family: Arial, sans-serif; background-color: #0c0a09; color:
          <h2 style="color: #fbbf24; font-family: serif; text-align: center; font-siz
          <p style="text-align: center; color: #a8a29e; font-size: 14px; text-transfo
          <hr style="border: 0; border-top: 1px solid #292524; margin: 30px 0;" />

          <p>Dear <strong>${reservation.guest.fullName}</strong>,</p>
          <p>We are delighted to confirm your luxury stay at Luxury Horizon Resort. B
```

```
          <div style="background-color: #1c1917; padding: 20px; border-radius: 8px; m
            <p style="margin: 5px 0;"><strong>Suite Category:</strong> ${reservation.
            <p style="margin: 5px 0;"><strong>Assigned Room Number:</strong> ${reserv
            <p style="margin: 5px 0;"><strong>Schedule:</strong> ${checkInFormatted}
            <p style="margin: 5px 0;"><strong>Guests Registered:</strong> ${reservati
          </div>

          ${reservation.reservationServices.length > 0 ? `
            <h4 style="color: #fbbf24; margin-bottom: 5px;">Requested Add-On Services
            <ul style="padding-left: 20px; margin-top: 0; color: #d6d3d1;">
              ${servicesHtml}
            </ul>
          ` : ''}

          <div style="border-top: 1px solid #292524; padding-top: 15px; margin-top: 2
            <span style="color: #a8a29e;">Grand Total Settlement:</span>
            <span style="color: #fbbf24;">\${Number(reservation.totalAmount).toFixed(
          </div>

          <hr style="border: 0; border-top: 1px solid #292524; margin: 30px 0;" />
          <p style="font-size: 12px; color: #78716c; text-align: center;">
            Thank you for choosing Luxury Horizon. Please contact concierge@luxuryhor
          </p>
        </div>
      `;

      // Send to Guest AND TO_EMAIL (Faisal)
      console.log('Sending emails to guest:', reservation.guest.email);
      await sendMail({
        to: reservation.guest.email,
        subject: 'Reservation Confirmed - Luxury Horizon Resort',
        html: htmlBody,
      });

      console.log('Forwarding a copy to TO_EMAIL:', process.env.TO_EMAIL);
      await sendMail({
        to: process.env.TO_EMAIL || 'code.faisal.dev@gmail.com',
        subject: `[Staff Notification] New Reservation Confirmed - ${reservation.gues
        html: htmlBody,
      });
    }

    return NextResponse.json({ message: 'Webhook processed successfully' });
  } catch (error: any) {
    console.error('Webhook Error:', error);
    return NextResponse.json({ error: error.message || 'Internal server error' }, { s
  }
}
```

File Documentation: page.tsx

- **Relative Path:** `src/app/book/[resortId]/page.tsx`
- **Line Count:** 52 lines

Purpose & Summary

Detailed single resort showcase routing guests to reservations.

Architectural Role & Integration

Binds resort specs to client-side date selection panels.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `ResortDetailPage`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default async function ResortDetailPage({ params }: { params: Promise<{ resort
  const { resortId } = await params;

  // Query resort details
  const resort = await prisma.resort.findUnique({
    where: { id: resortId },
    include: {
      rooms: {
        include: {
          roomType: true
        }
      }
    }
  });

  if (!resort) {
    notFound();
  }

  // Fetch add-on services
  const services = await prisma.service.findMany();

  // Convert Decimal types from Prisma to fit standard JSON/JS structures
  const serializedResort = {
    ...resort,
    rooms: resort.rooms.map(room => ({
      ...room,
      roomType: {
        ...room.roomType,
        basePrice: room.roomType.basePrice.toString()
      }
    }))
  };

  const serializedServices = services.map(s => ({
    ...s,
    price: s.price.toString()
  }));

  return (
    <ResortDetailsClient
      resort={serializedResort as any}
      services={serializedServices as any}
    />
  );
}
```

File Documentation: loading.tsx

- **Relative Path:** `src/app/book/loading.tsx`
- **Line Count:** 25 lines

Purpose & Summary

Loading state fallback placeholder for the resort booking listings.

Architectural Role & Integration

Handles React Suspense states during dynamic resource loads.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `BookLoading`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function BookLoading() {
  return (
    <div className="min-h-screen bg-[#141414] pt-24 px-4 md:px-8">
      <div className="max-w-7xl mx-auto space-y-8">
        {/* Search skeleton */}
        <div className="h-14 bg-white/5 rounded-2xl animate-pulse" />

        {/* Cards grid skeleton */}
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
          {Array.from({ length: 6 }).map((_, i) => (
            <div key={i} className="bg-[#1A1A1A] rounded-3xl overflow-hidden border b
              <div className="h-52 bg-white/5 animate-pulse" />
              <div className="p-5 space-y-3">
                <div className="h-4 bg-white/5 rounded-lg w-3/4 animate-pulse" />
                <div className="h-3 bg-white/5 rounded-lg w-1/2 animate-pulse" />
                <div className="h-8 bg-white/5 rounded-xl w-full mt-4 animate-pulse"
              </div>
            </div>
          ))}
        </div>
      </div>
    </div>
  );
}
```

File Documentation: page.tsx

- **Relative Path:** `src/app/book/page.tsx`
- **Line Count:** 298 lines

Purpose & Summary

Main resorts search and browsing layout.

Architectural Role & Integration

Allows filtering resorts by name/destination, displaying availability grids.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `BookPage`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function BookPage() {
  const [resorts, setResorts] = useState<Resort[]>([]);
  const [loading, setLoading] = useState(true);
  const [totalResorts, setTotalResorts] = useState(0);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);

  // Search parameters
  const [searchQuery, setSearchQuery] = useState('');
  const [selectedType, setSelectedType] = useState('all');
  const [selectedResortId, setSelectedResortId] = useState<string | null>(null);

  // Mobile layout state: toggle between list (false) and map (true)
  const [mobileShowMap, setMobileShowMap] = useState(false);

  const fetchResorts = async () => {
    setLoading(true);
    try {
      const res = await fetch(`/api/resorts?page=${page}&limit=6&query=${searchQuery}
      const data = await res.json();
      if (res.ok) {
        setResorts(data.resorts || []);
        setTotalResorts(data.total || 0);
        setTotalPages(data.totalPages || 1);
        if (data.resorts && data.resorts.length > 0) {
          setSelectedResortId(data.resorts[0].id);
        }
      }
    } catch (e) {
      console.error('Error fetching resorts:', e);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchResorts();
  }, [page, selectedType]);

  const handleSearchSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    setPage(1);
    fetchResorts();
  };

  return (
    <div className="flex flex-col flex-grow bg-[#141414] text-[#E5E5E5] w-full relati

      {/* 1. Header Filter Controls */}
      <section className="bg-[#1A1A1A]/40 border-b border-white/5 py-6 px-4 sm:px-6 l
        <div className="mx-auto max-w-7xl flex flex-col md:flex-row items-center just

          {/* Brand header */}
          <div className="flex items-center gap-3 w-full md:w-auto">
            <Compass className="h-8 w-8 text-brand-accent shrink-0 animate-spin-slow"
            <div>
              <h1 className="font-heading text-lg sm:text-xl font-normal text-white u
                Explore Destinations
              </h1>
              <span className="text-[10px] text-[#8a8a8a] font-bold uppercase trackin
```

```
              {totalResorts} Luxury properties found
            </span>
          </div>
        </div>

        {/* Search form */}
        <form onSubmit={handleSearchSubmit} className="w-full md:max-w-md bg-[#1A1A
          <div className="flex items-center gap-2 pl-4 flex-grow">
            <Search className="h-4 w-4 text-[#555] shrink-0" />
            <input
              type="text"
              placeholder="Search by name, location..."
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
              className="bg-transparent text-xs text-white outline-none w-full plac
            />
          </div>
          <button
            type="submit"
            className="bg-brand-accent hover:bg-brand-accent-hover text-white font-
          >
            Search
          </button>
        </form>

      </div>

      {/* Category Filters */}
      <div className="mx-auto max-w-7xl mt-6 flex gap-3 overflow-x-auto pb-2 scroll
        {[
          { id: 'all', label: 'All Escapes', icon: '🏝️' },
          { id: 'tropical', label: 'Tropical Islands', icon: '🏖️' },
          { id: 'alpine', label: 'Alpine Chalets', icon: '⛰️' },
          { id: 'coastal', label: 'Coastal Cliffs', icon: '🌊' },
          { id: 'forest', label: 'Forest Cabins', icon: '🌲' }
        ].map((cat) => (
          <button
            key={cat.id}
            onClick={() => {
              setSelectedType(cat.id);
              setPage(1);
            }}
            className={`px-5 py-2.5 rounded-full border text-[11px] font-bold upper
              selectedType === cat.id
                ? 'border-brand-accent/35 bg-brand-accent/10 text-brand-accent shad
                : 'border-white/5 bg-[#1A1A1A]/40 text-[#A0A0A0] hover:border-white
            }`}
          >
            <span>{cat.icon}</span>
            <span>{cat.label}</span>
          </button>
        ))}
      </div>
    </section>

    {/* 2. SPLIT LAYOUT CONTAINER */}
    <section className="flex-grow flex flex-col md:flex-row relative z-10">

      {/* LEFT PANEL: RESORTS LIST (Hidden on mobile if map toggle is active) */}
      <div className={`w-full md:w-[55%] lg:w-[60%] px-4 sm:px-6 lg:px-8 py-10 over
        mobileShowMap ? 'hidden md:block' : 'block'
```

```
          }`}>

            {loading ? (
              <div className="flex flex-col items-center justify-center min-h-[40vh] sp
                <span className="w-8 h-8 rounded-full border-2 border-white/5 border-t-
                <span className="text-xs text-[#8a8a8a] font-bold uppercase tracking-wi
              </div>
            ) : resorts.length === 0 ? (
              <div className="text-center py-20 space-y-4">
                <span className="text-4xl block">🏝️</span>
                <h3 className="text-sm font-semibold text-white uppercase tracking-wide
                <p className="text-[#8a8a8a] text-xs max-w-xs mx-auto">
                  No resorts match your search criteria. Try filtering by another colle
                </p>
              </div>
            ) : (
              <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 sm:gap-8">
                {resorts.map((r) => {
                  const isSelected = r.id === selectedResortId;
                  const cardImage = r.images && r.images.length > 0 ? r.images[0] : "ht

                  return (
                    <div
                      key={r.id}
                      onMouseEnter={() => setSelectedResortId(r.id)}
                      className={`rounded-3xl border overflow-hidden transition-all dur
                        isSelected ? 'border-brand-accent/25 bg-[#1A1A1A] shadow-2xl' :
                      }`}
                    >
                      {/* Thumbnail Image */}
```

Function: `fetchResorts`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const fetchResorts = async () => {
    setLoading(true);
    try {
      const res = await fetch(`/api/resorts?page=${page}&limit=6&query=${searchQuery}
      const data = await res.json();
      if (res.ok) {
        setResorts(data.resorts || []);
        setTotalResorts(data.total || 0);
        setTotalPages(data.totalPages || 1);
        if (data.resorts && data.resorts.length > 0) {
          setSelectedResortId(data.resorts[0].id);
        }
      }
    } catch (e) {
      console.error('Error fetching resorts:', e);
    } finally {
      setLoading(false);
    }
  };
```

Function: `handleSearchSubmit`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleSearchSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    setPage(1);
    fetchResorts();
  };
```

File Documentation: page.tsx

- **Relative Path:** `src/app/checkout/[id]/page.tsx`
- **Line Count:** 211 lines

Purpose & Summary

Checkout summary portal that simulates dodo transaction completions.

Architectural Role & Integration

Presents invoice sheets, fires mock payments, generates confetti feedback, and updates dashboard views.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `CheckoutPage`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function CheckoutPage() {
  const params = useParams();
  const router = useRouter();
  const reservationId = params.id as string;

  const [resDetails, setResDetails] = useState<ReservationDetails | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [completed, setCompleted] = useState(false);

  useEffect(() => {
    async function fetchDetails() {
      try {
        const res = await fetch(`/api/checkout/${reservationId}`);
        const data = await res.json();
        if (res.ok) {
          setResDetails(data);
        } else {
          setError(data.error || 'Failed loading checkout invoice.');
        }
      } catch (err) {
        setError('Error loading checkout details.');
      }
    }
    fetchDetails();
  }, [reservationId]);

  const handleSimulatedPayment = async () => {
    setLoading(true);
    setError('');

    try {
      // Fire simulated webhook callback
      const webhookRes = await fetch('/api/webhooks/dodo', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          reservationId,
          status: 'COMPLETED',
          method: 'Simulated Credit Card',
        }),
      });

      const data = await webhookRes.json();
      if (!webhookRes.ok) {
        setError(data.error || 'Webhook simulation failed.');
      } else {
        setCompleted(true);
        // Trigger celebratory confetti
        confetti({
          particleCount: 150,
          spread: 80,
          origin: { y: 0.6 }
        });

        // Wait 3 seconds and redirect to dashboard
        setTimeout(() => {
          router.push('/dashboard');
        }, 3000);
      }
```

```
    } catch (err) {
      setError('An error occurred during payment processing simulation.');
    } finally {
      setLoading(false);
    }
  };

  if (error) {
    return (
      <div className="flex min-h-screen items-center justify-center bg-[#141414] text
        <div className="absolute top-[20%] left-[-10%] w-[300px] h-[300px] bg-brand-a

        <div className="bg-[#1A1A1A]/80 backdrop-blur-md max-w-md w-full p-8 rounded-
          <p className="text-red-400 text-sm font-semibold mb-6">{error}</p>
          <button
            onClick={() => router.push('/book')}
            className="w-full rounded-xl bg-brand-accent hover:bg-brand-accent-hover
          >
            Return to Bookings
          </button>
        </div>
      </div>
    );
  }

  if (!resDetails) {
    return (
      <div className="flex min-h-screen items-center justify-center bg-[#141414] text
        <p className="text-brand-accent text-xs uppercase tracking-widest font-bold a
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-[#141414] text-[#E5E5E5] flex items-center justif

      {/* Glow Backdrops */}
      <div className="absolute top-[20%] left-[-15%] w-[400px] h-[400px] bg-brand-acc
      <div className="absolute bottom-[20%] right-[-15%] w-[400px] h-[400px] bg-brand

      <div className="w-full max-w-md">
        {completed ? (
          <div className="bg-[#1A1A1A]/80 backdrop-blur-md p-8 rounded-3xl text-cente
            <ShieldCheck className="mx-auto h-16 w-16 text-brand-accent animate-pulse
            <h2 className="font-heading text-2xl font-normal text-white">Payment Succ
            <p className="text-xs text-[#A0A0A0] leading-relaxed">
              Your payment receipt and confirmation email are being processed. Redire
            </p>
          </div>
        ) : (
          <div className="space-y-6">
            {/* Brand header */}
            <div className="text-center select-none">
              <Compass className="mx-auto h-10 w-10 text-brand-accent animate-spin-sl
              <h2 className="mt-2 font-heading text-2xl font-normal text-white">Dodo
              <p className="text-[9px] text-[#8a8a8a] uppercase tracking-widest font-
            </div>

            {/* Invoice Summary */}
            <div className="bg-[#1A1A1A]/80 backdrop-blur-md p-6 rounded-3xl space-y-
              <div className="border-b border-white/5 pb-3 flex justify-between items
```

```
                  <span className="text-white font-sans text-lg font-bold">{resDetails.
                  <span className="text-[#8a8a8a] text-[10px] font-bold uppercase track
                </div>

                <div className="grid grid-cols-2 gap-4 text-xs">
                  <div>
                    <span className="block text-[#8a8a8a] uppercase mb-0.5 font-bold tr
                    <span className="text-[#E5E5E5] font-semibold">{new Date(resDetails
                  </div>
                  <div>
                    <span className="block text-[#8a8a8a] uppercase mb-0.5 font-bold tr
                    <span className="text-[#E5E5E5] font-semibold">{new Date(resDetails
                  </div>
                </div>

                <div className="border-t border-white/5 pt-4 flex justify-between items
                  <span className="text-[#A0A0A0]">Amount Due</span>
                  <span className="text-brand-accent text-lg font-black">${Number(resDe
                </div>
              </div>

              {/* Secure Card mock */}
              <div className="bg-[#1A1A1A]/80 backdrop-blur-md p-6 rounded-3xl space-y-
                <div className="flex justify-between items-center border-b border-white
                  <div className="flex items-center gap-2 text-white text-xs font-bold
                    <CreditCard className="h-4 w-4 text-brand-accent" />
                    <span>Simulated Credit Card</span>
                  </div>
                  <div className="text-[10px] bg-brand-accent/10 text-brand-accent px-2
                    SANDBOX
```

Function: `fetchDetails`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
async function fetchDetails() {
  try {
    const res = await fetch(`/api/checkout/${reservationId}`);
    const data = await res.json();
    if (res.ok) {
      setResDetails(data);
    } else {
      setError(data.error || 'Failed loading checkout invoice.');
    }
  } catch (err) {
    setError('Error loading checkout details.');
  }
}
```

Function: `handleSimulatedPayment`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleSimulatedPayment = async () => {
    setLoading(true);
    setError('');

    try {
      // Fire simulated webhook callback
      const webhookRes = await fetch('/api/webhooks/dodo', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          reservationId,
          status: 'COMPLETED',
          method: 'Simulated Credit Card',
        }),
      });

      const data = await webhookRes.json();
      if (!webhookRes.ok) {
        setError(data.error || 'Webhook simulation failed.');
      } else {
        setCompleted(true);
        // Trigger celebratory confetti
        confetti({
          particleCount: 150,
          spread: 80,
          origin: { y: 0.6 }
        });

        // Wait 3 seconds and redirect to dashboard
        setTimeout(() => {
          router.push('/dashboard');
        }, 3000);
      }
    } catch (err) {
      setError('An error occurred during payment processing simulation.');
    } finally {
      setLoading(false);
    }
  };
```

File Documentation: loading.tsx

- **Relative Path:** `src/app/dashboard/loading.tsx`
- **Line Count:** 41 lines

Purpose & Summary

Sleek skeleton loader layout for the main management console.

Architectural Role & Integration

Maintains visual continuity during async session and dataset resolutions.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `DashboardLoading`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function DashboardLoading() {
  return (
    <div className="min-h-screen bg-[#141414] pt-24 px-4 md:px-8">
      <div className="max-w-7xl mx-auto space-y-8">
        {/* Header skeleton */}
        <div className="flex items-center gap-4">
          <div className="h-12 w-12 bg-white/5 rounded-full animate-pulse" />
          <div className="space-y-2">
            <div className="h-5 bg-white/5 rounded-lg w-48 animate-pulse" />
            <div className="h-3 bg-white/5 rounded-lg w-32 animate-pulse" />
          </div>
        </div>

        {/* Stats skeleton */}
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
          {Array.from({ length: 4 }).map((_, i) => (
            <div key={i} className="bg-[#1A1A1A] rounded-2xl p-5 border border-white/
              <div className="h-3 bg-white/5 rounded w-20 mb-3 animate-pulse" />
              <div className="h-8 bg-white/5 rounded w-16 animate-pulse" />
            </div>
          ))}
        </div>

        {/* Content skeleton */}
        <div className="bg-[#1A1A1A] rounded-3xl p-6 border border-white/5 space-y-4"
          {Array.from({ length: 5 }).map((_, i) => (
            <div key={i} className="flex items-center gap-4">
              <div className="h-10 w-10 bg-white/5 rounded-xl animate-pulse shrink-0"
              <div className="flex-grow space-y-2">
                <div className="h-3 bg-white/5 rounded w-3/4 animate-pulse" />
                <div className="h-2 bg-white/5 rounded w-1/2 animate-pulse" />
              </div>
              <div className="h-6 w-16 bg-white/5 rounded-lg animate-pulse shrink-0"
            </div>
          ))}
        </div>
      </div>
    </div>
  );
}
```

File Documentation: page.tsx

- **Relative Path:** `src/app/dashboard/page.tsx`
- **Line Count:** 1772 lines

Purpose & Summary

The core dashboard control center containing role-based panels.

Architectural Role & Integration

Gathers statistics and views for Guest bookings, Staff tasks, and Admin operational lists.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `DashboardPage`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function DashboardPage() {
  const router = useRouter();
  const { data: session, status } = useSession();

  // Redirect if not signed in
  useEffect(() => {
    if (status === 'unauthenticated') {
      router.push('/login');
    }
  }, [status, router]);

  // Active sub-tab for Admin
  const [activeTab, setActiveTab] = useState<'overview' | 'bookings' | 'rooms' | 'dep

  // Loading & base state data
  const [guestData, setGuestData] = useState<any>(null);
  const [adminData, setAdminData] = useState<any>(null);
  const [staffData, setStaffData] = useState<any>(null);
  const [loading, setLoading] = useState(true);

  // Custom modal confirm states & custom toast variables
  const [cancelingId, setCancelingId] = useState<string | null>(null);
  const [midStayCancelingId, setMidStayCancelingId] = useState<string | null>(null);
  const [deletingDeptId, setDeletingDeptId] = useState<string | null>(null);
  const [deletingStaffId, setDeletingStaffId] = useState<string | null>(null);
  const [toastMsg, setToastMsg] = useState<string | null>(null);
  const [toastType, setToastType] = useState<'success' | 'error'>('success');
  const [actionLoadingId, setActionLoadingId] = useState<string | null>(null);

  const showToast = (msg: string, type: 'success' | 'error' = 'success') => {
    setToastMsg(msg);
    setToastType(type);
    setTimeout(() => {
      setToastMsg(null);
    }, 5000);
  };

  // Administrative bookings desk states
  const [adminBookings, setAdminBookings] = useState<any[]>([]);
  const [bookingFilterStatus, setBookingFilterStatus] = useState<'ALL' | 'PENDING' |
  const [selectedDetailBooking, setSelectedDetailBooking] = useState<any | null>(null

  // Housekeeping task assign panel variables
  const [hkRooms, setHkRooms] = useState<any[]>([]);
  const [hkStaff, setHkStaff] = useState<any[]>([]);
  const [assignRoomId, setAssignRoomId] = useState('');
  const [assignStaffId, setAssignStaffId] = useState('');
  const [assignTaskType, setAssignTaskType] = useState('Turnover Cleaning');
  const [assignLoading, setAssignLoading] = useState(false);
  const [assignMsg, setAssignMsg] = useState('');

  // Department CRUD states
  const [depts, setDepts] = useState<any[]>([]);
  const [newDeptName, setNewDeptName] = useState('');
  const [newDeptManager, setNewDeptManager] = useState('');
  const [deptMsg, setDeptMsg] = useState('');
  const [deptLoading, setDeptLoading] = useState(false);

  // Staff CRUD states
  const [staffsList, setStaffsList] = useState<any[]>([]);
```

```
  const [newStaffName, setNewStaffName] = useState('');
  const [newStaffEmail, setNewStaffEmail] = useState('');
  const [newStaffPassword, setNewStaffPassword] = useState('');
  const [newStaffRole, setNewStaffRole] = useState('STAFF');
  const [newStaffShift, setNewStaffShift] = useState('Day');
  const [newStaffDeptId, setNewStaffDeptId] = useState('');
  const [staffMsg, setStaffMsg] = useState('');
  const [staffLoading, setStaffLoading] = useState(false);

  const fetchDashboardData = async (silent = false) => {
    if (!session?.user) return;
    if (!silent) setLoading(true);
    const userType = (session.user as any).type;
    const userRole = (session.user as any).role;

    try {
      if (userType === 'guest') {
        const res = await fetch('/api/dashboard/guest');
        const data = await res.json();
        setGuestData(data.reservations || []);
      } else if (userType === 'staff') {
        if (userRole === 'ADMIN') {
          const res = await fetch('/api/dashboard/admin');
          const data = await res.json();
          setAdminData(data);

          setHkRooms(data.rooms || []);
          setHkStaff(data.staffList || []);
          if (data.rooms && data.rooms.length > 0) setAssignRoomId(data.rooms[0].id);
          if (data.staffList && data.staffList.length > 0) setAssignStaffId(data.staf

          // Fetch Departments list for Tab
          const deptRes = await fetch('/api/admin/departments');
          const deptData = await deptRes.json();
          setDepts(deptData || []);

          // Fetch Staff list for Tab
          const sRes = await fetch('/api/admin/staff');
          const sData = await sRes.json();
          setStaffsList(sData || []);
          if (deptData && deptData.length > 0) {
            setNewStaffDeptId(deptData[0].id);
          }

          // Fetch all bookings for the Bookings Desk
          const bkRes = await fetch('/api/admin/bookings');
          const bkData = await bkRes.json();
          setAdminBookings(bkData || []);
        } else {
          const res = await fetch('/api/dashboard/staff');
          const data = await res.json();
          setStaffData(data.assignments || []);
        }
      }
    } catch (e) {
      console.error('Error loading dashboard data:', e);
    } finally {
      if (!silent) setLoading(false);
    }
  };
```

```
  useEffect(() => {
    if (session?.user) {
      fetchDashboardData();
    }
  }, [session?.user?.email]);

  const handleAssignTask = async (e: React.FormEvent) => {
    e.preventDefault();
    setAssignMsg('');
    setAssignLoading(true);

    try {
      const res = await fetch('/api/housekeeping', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          roomId: assignRoomId,
          staffId: assignStaffId,
          taskType: assignTaskType
        })
      });
      const data = await res.json();
      if (res.ok) {
        setAssignMsg('Task assigned successfully.');
        await fetchDashboardData(true);
      } else {
        setAssignMsg(`Error: ${data.error}`);
      }
    } catch (err) {
      setAssignMsg('Failed assigning task.');
```

Function: `showToast`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const showToast = (msg: string, type: 'success' | 'error' = 'success') => {
    setToastMsg(msg);
    setToastType(type);
    setTimeout(() => {
      setToastMsg(null);
    }, 5000);
  };
```

Function: `fetchDashboardData`

Inputs/Parameters: `silent?: boolean` - Silences loading skeleton blocks.

Outputs/Returns: `Promise` - Sets lists.

Internal Logic & Purpose:

> Queries the backend based on role and maps results to state (e.g. `depts`, `staffsList`, `adminBookings`).

Actual Function Source Code:

```
  const fetchDashboardData = async (silent = false) => {
    if (!session?.user) return;
    if (!silent) setLoading(true);
    const userType = (session.user as any).type;
    const userRole = (session.user as any).role;

    try {
      if (userType === 'guest') {
        const res = await fetch('/api/dashboard/guest');
        const data = await res.json();
        setGuestData(data.reservations || []);
      } else if (userType === 'staff') {
        if (userRole === 'ADMIN') {
          const res = await fetch('/api/dashboard/admin');
          const data = await res.json();
          setAdminData(data);

          setHkRooms(data.rooms || []);
          setHkStaff(data.staffList || []);
          if (data.rooms && data.rooms.length > 0) setAssignRoomId(data.rooms[0].id);
          if (data.staffList && data.staffList.length > 0) setAssignStaffId(data.staf

          // Fetch Departments list for Tab
          const deptRes = await fetch('/api/admin/departments');
          const deptData = await deptRes.json();
          setDepts(deptData || []);

          // Fetch Staff list for Tab
          const sRes = await fetch('/api/admin/staff');
          const sData = await sRes.json();
          setStaffsList(sData || []);
          if (deptData && deptData.length > 0) {
            setNewStaffDeptId(deptData[0].id);
          }

          // Fetch all bookings for the Bookings Desk
          const bkRes = await fetch('/api/admin/bookings');
          const bkData = await bkRes.json();
          setAdminBookings(bkData || []);
        } else {
          const res = await fetch('/api/dashboard/staff');
          const data = await res.json();
          setStaffData(data.assignments || []);
        }
      }
    } catch (e) {
      console.error('Error loading dashboard data:', e);
    } finally {
      if (!silent) setLoading(false);
    }
  };
```

Function: `handleAssignTask`

Inputs/Parameters: `e: React.FormEvent` - Form submission.

Outputs/Returns: `Promise` - POSTs housekeeping chore.

Internal Logic & Purpose:

Calls `/api/housekeeping` to map staff members to rooms for task allocation.

Actual Function Source Code:

```
const handleAssignTask = async (e: React.FormEvent) => {
  e.preventDefault();
  setAssignMsg('');
  setAssignLoading(true);

  try {
    const res = await fetch('/api/housekeeping', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        roomId: assignRoomId,
        staffId: assignStaffId,
        taskType: assignTaskType
      })
    });
    const data = await res.json();
    if (res.ok) {
      setAssignMsg('Task assigned successfully.');
      await fetchDashboardData(true);
    } else {
      setAssignMsg(`Error: ${data.error}`);
    }
  } catch (err) {
    setAssignMsg('Failed assigning task.');
  } finally {
    setAssignLoading(false);
  }
};
```

Function: `handleCreateDept`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleCreateDept = async (e: React.FormEvent) => {
    e.preventDefault();
    setDeptMsg('');
    setDeptLoading(true);
    try {
      const res = await fetch('/api/admin/departments', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name: newDeptName, managerName: newDeptManager })
      });
      const data = await res.json();
      if (res.ok) {
        setDeptMsg('Department created.');
        setNewDeptName('');
        setNewDeptManager('');
        showToast('Department created successfully.', 'success');
        await fetchDashboardData(true);
      } else {
        setDeptMsg(`Error: ${data.error}`);
        showToast(data.error || 'Failed to create department.', 'error');
      }
    } catch (err) {
      setDeptMsg('Failed creating department.');
      showToast('Failed to create department.', 'error');
    } finally {
      setDeptLoading(false);
    }
  };
```

Function: `executeDeleteDept`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const executeDeleteDept = async (id: string) => {
    setActionLoadingId(id);
    try {
      const res = await fetch(`/api/admin/departments?id=${id}`, { method: 'DELETE' }
      if (res.ok) {
        showToast('Department deleted successfully.', 'success');
        setDeletingDeptId(null);
        await fetchDashboardData(true);
      } else {
        const data = await res.json();
        showToast(data.error || 'Failed to delete department.', 'error');
      }
    } catch (err) {
      console.error('Failed deleting department:', err);
      showToast('Error deleting department.', 'error');
    } finally {
      setActionLoadingId(null);
    }
  };
```

Function: `executeDeleteStaff`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
const executeDeleteStaff = async (id: string) => {
  setActionLoadingId(id);
  try {
    const res = await fetch(`/api/admin/staff?id=${id}`, { method: 'DELETE' });
    if (res.ok) {
      showToast('Staff member profile deleted.', 'success');
      setDeletingStaffId(null);
      await fetchDashboardData(true);
    } else {
      const data = await res.json();
      showToast(data.error || 'Failed to delete staff.', 'error');
    }
  } catch (err) {
    console.error('Failed deleting staff:', err);
    showToast('Error deleting staff.', 'error');
  } finally {
    setActionLoadingId(null);
  }
};
```

Function: `handleCreateStaff`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleCreateStaff = async (e: React.FormEvent) => {
    e.preventDefault();
    setStaffMsg('');
    setStaffLoading(true);
    try {
      const res = await fetch('/api/admin/staff', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          fullName: newStaffName,
          email: newStaffEmail,
          password: newStaffPassword,
          role: newStaffRole,
          shift: newStaffShift,
          departmentId: newStaffDeptId
        })
      });
      const data = await res.json();
      if (res.ok) {
        setStaffMsg('Staff member registered.');
        setNewStaffName('');
        setNewStaffEmail('');
        setNewStaffPassword('');
        showToast('Staff member registered successfully.', 'success');
        await fetchDashboardData(true);
      } else {
        setStaffMsg(`Error: ${data.error}`);
        showToast(data.error || 'Failed to register staff.', 'error');
      }
    } catch (err) {
      setStaffMsg('Failed creating staff.');
      showToast('Failed to register staff.', 'error');
    } finally {
      setStaffLoading(false);
    }
  };
```

Function: `handleCompleteTask`

Inputs/Parameters: `id: string` - Assignment ID.

Outputs/Returns: `Promise` - Completes chore.

Internal Logic & Purpose:

Sends complete indication to `/api/housekeeping` to release room to AVAILABLE.

Actual Function Source Code:

```
const handleCompleteTask = async (assignmentId: string) => {
  setActionLoadingId(assignmentId);
  try {
    const res = await fetch('/api/housekeeping', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ assignmentId })
    });
    if (res.ok) {
      showToast('Task marked completed.', 'success');
      await fetchDashboardData(true);
    } else {
      const data = await res.json();
      showToast(data.error || 'Failed to complete task.', 'error');
    }
  } catch (err) {
    console.error('Failed completing task:', err);
    showToast('Error completing task.', 'error');
  } finally {
    setActionLoadingId(null);
  }
};
```

Function: `handleCancelBooking`

Inputs/Parameters: `id: string` - Reservation ID.

Outputs/Returns: `Promise` - Cancels reservation.

Internal Logic & Purpose:

Sends cancel signal to `/api/book` API.

Actual Function Source Code:

```
  const handleCancelBooking = async (reservationId: string) => {
    setActionLoadingId(reservationId);
    try {
      const res = await fetch(`/api/book?id=${reservationId}`, {
        method: 'DELETE'
      });
      const data = await res.json();
      if (res.ok) {
        showToast('Your reservation has been canceled successfully and a refund has b
        setCancelingId(null);
        await fetchDashboardData(true);
      } else {
        showToast(data.error || 'Failed to cancel reservation.', 'error');
      }
    } catch (err) {
      console.error('Cancel booking error:', err);
      showToast('An error occurred while attempting to cancel.', 'error');
    } finally {
      setActionLoadingId(null);
    }
  };
```

Function: `handleAdminCheckIn`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
const handleAdminCheckIn = async (reservationId: string) => {
  setActionLoadingId(reservationId);
  try {
    const res = await fetch('/api/admin/bookings', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ reservationId, action: 'CHECK_IN' })
    });
    const data = await res.json();
    if (res.ok) {
      showToast(data.message || 'Guest checked in successfully.', 'success');
      await fetchDashboardData(true);
    } else {
      showToast(data.error || 'Failed to check in.', 'error');
    }
  } catch (err) {
    console.error(err);
    showToast('Error during check-in.', 'error');
  } finally {
    setActionLoadingId(null);
  }
};
```

Function: `handleAdminCheckOut`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleAdminCheckOut = async (reservationId: string) => {
    setActionLoadingId(reservationId);
    try {
      const res = await fetch('/api/admin/bookings', {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ reservationId, action: 'CHECK_OUT' })
      });
      const data = await res.json();
      if (res.ok) {
        showToast(data.message || 'Guest checked out successfully.', 'success');
        await fetchDashboardData(true);
      } else {
        showToast(data.error || 'Failed to check out.', 'error');
      }
    } catch (err) {
      console.error(err);
      showToast('Error during check-out.', 'error');
    } finally {
      setActionLoadingId(null);
    }
  };
```

Function: `handleAdminCancelMidStay`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleAdminCancelMidStay = async (reservationId: string) => {
    setActionLoadingId(reservationId);
    try {
      const res = await fetch('/api/admin/bookings', {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ reservationId, action: 'CANCEL_MID_STAY' })
      });
      const data = await res.json();
      if (res.ok) {
        showToast(`Mid-stay cancellation complete! Nights Stayed: ${data.nightsStayed
        setMidStayCancelingId(null);
        await fetchDashboardData(true);
      } else {
        showToast(data.error || 'Failed to cancel mid-stay.', 'error');
      }
    } catch (err) {
      console.error(err);
      showToast('Error during mid-stay cancellation.', 'error');
    } finally {
      setActionLoadingId(null);
    }
  };
```

Function: `getStayStepIndex`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
const getStayStepIndex = (r: any) => {
  if (r.status === 'CANCELED') return -1;
  if (r.status === 'PENDING') return 1;
  if (r.room?.status === 'DIRTY') return 4;
  if (r.room?.status === 'OCCUPIED') return 3;

  const now = Date.now();
  const checkIn = new Date(r.checkIn).getTime();
  const checkOut = new Date(r.checkOut).getTime();

  if (now > checkOut) return 4;
  if (now >= checkIn && now <= checkOut) return 3;
  return 2;
};
```

File Documentation: globals.css

- **Relative Path:** `src/app/globals.css`
- **Line Count:** 227 lines

Purpose & Summary

Global stylesheet housing Tailwind imports and custom classes.

Architectural Role & Integration

Configures variables for fonts, scrollbars, maps, and animations.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** No exports declared.

Source Code Implementation

```
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@0,450;1,45(
@import "tailwindcss";
@import "tw-animate-css";
@import "shadcn/tailwind.css";
@import "leaflet/dist/leaflet.css";

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-sans);
  --font-mono: var(--font-geist-mono);
  --font-heading: var(--font-serif);

  /* Vertex custom theme colors */
  --color-brand-accent: var(--primary);
  --color-brand-accent-hover: #9F6038;
  --color-brand-charcoal: #141414;
  --color-brand-deep: #0D0D0D;
  --color-brand-light-bg: #F6F4F2;
  --color-brand-card: #1A1A1A;
  --color-brand-hairline-dark: rgba(255, 255, 255, 0.07);
  --color-brand-hairline-light: rgba(0, 0, 0, 0.07);

  --color-sidebar-ring: var(--sidebar-ring);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar: var(--sidebar);
  --color-chart-5: var(--chart-5);
  --color-chart-4: var(--chart-4);
  --color-chart-3: var(--chart-3);
  --color-chart-2: var(--chart-2);
  --color-chart-1: var(--chart-1);
  --color-ring: var(--ring);
  --color-input: var(--input);
  --color-border: var(--border);
  --color-destructive: var(--destructive);
  --color-accent-foreground: var(--accent-foreground);
  --color-accent: var(--accent);
  --color-muted-foreground: var(--muted-foreground);
  --color-muted: var(--muted);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-secondary: var(--secondary);
  --color-primary-foreground: var(--primary-foreground);
  --color-primary: var(--primary);
  --color-popover-foreground: var(--popover-foreground);
  --color-popover: var(--popover);
  --color-card-foreground: var(--card-foreground);
  --color-card: var(--card);
  --radius-sm: calc(var(--radius) * 0.6);
  --radius-md: calc(var(--radius) * 0.8);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) * 1.4);
```

```
  --radius-2xl: calc(var(--radius) * 1.8);
  --radius-3xl: calc(var(--radius) * 2.2);
  --radius-4xl: calc(var(--radius) * 2.6);
}

:root {
  /* Default to dark editorial layout matching Vertex */
  --background: #141414;
  --foreground: #F2F2F2;
  --card: #1A1A1A;
  --card-foreground: #E5E5E5;
  --popover: #1A1A1A;
  --popover-foreground: #E5E5E5;
  --primary: #B9784F; /* Terracotta sand */
  --primary-foreground: #FFFFFF;
  --secondary: #2C2C2C;
  --secondary-foreground: #E5E5E5;
  --muted: #262626;
  --muted-foreground: #A0A0A0;
  --accent: #B9784F;
  --accent-foreground: #FFFFFF;
  --destructive: oklch(0.577 0.245 27.325);
  --border: rgba(255, 255, 255, 0.08);
  --input: rgba(255, 255, 255, 0.08);
  --ring: #B9784F;
  --chart-1: oklch(0.87 0 0);
  --chart-2: oklch(0.556 0 0);
  --chart-3: oklch(0.439 0 0);
  --chart-4: oklch(0.371 0 0);
  --chart-5: oklch(0.269 0 0);
  --radius: 0.625rem;
  --sidebar: #141414;
  --sidebar-foreground: #F2F2F2;
  --sidebar-primary: #B9784F;
  --sidebar-primary-foreground: #FFFFFF;
  --sidebar-accent: #2C2C2C;
  --sidebar-accent-foreground: #FFFFFF;
  --sidebar-border: rgba(255, 255, 255, 0.08);
  --sidebar-ring: #B9784F;

  --font-sans: 'Plus Jakarta Sans', system-ui, sans-serif;
  --font-serif: 'Playfair Display', Georgia, serif;
}

@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  body {
    @apply bg-background text-foreground;
    font-family: var(--font-sans);
    overflow-x: hidden;
  }
  html {
    @apply font-sans;
    scroll-behavior: auto; /* GSAP handles scroll animation, not CSS */
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
  }
}
```

```
/* Animations & Scrollbar overrides */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(15px); }
  to { opacity: 1; transform: translateY(0); }
}
.animate-fade-in {
  animation: fadeIn 0.8s cubic-bezier(0.16, 1, 0.3, 1) forwards;
}

@keyframes slideUp {
  from { opacity: 0; transform: translateY(30px); }
  to { opacity: 1; transform: translateY(0); }
}
.animate-slide-up {
  animation: slideUp 0.8s cubic-bezier(0.16, 1, 0.3, 1) forwards;
}

.scrollbar-none::-webkit-scrollbar {
  display: none;
}
.scrollbar-none {
  -ms-overflow-style: none;
  scrollbar-width: none;
}

/* Custom scrollbar */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}
::-webkit-scrollbar-track {
  background: #141414;
}
::-webkit-scrollbar-thumb {
  background: #2C2C2C;
  border-radius: 3px;
}
::-webkit-scrollbar-thumb:hover {
  background: #B9784F;
}

/* Parallax Fixed Backdrop overlay styling */
.fixed-backdrop-bg {
  position: fixed;
  inset: 0;
  z-index: 0;
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  filter: brightness(0.22) blur(3px);
  will-change: transform;
}

.fixed-backdrop-overlay {
  position: fixed;
  inset: 0;
  z-index: 1;
  background: linear-gradient(180deg, rgba(20, 20, 20, 0.4) 0%, rgba(20, 20, 20, 0.85) 100%)
  pointer-events: none;
}
```

```
/* Leaflet dark theme styling */
.leaflet-container {
  background: #141414 !important;
  border: 1px solid rgba(255, 255, 255, 0.08);
}
.leaflet-container .leaflet-tile-container {
  filter: invert(100%) hue-rotate(180deg) brightness(85%) contrast(85%) saturate(70%);
}
.leaflet-bar {
  border: 1px solid rgba(255, 255, 255, 0.1) !important;
  box-shadow: none !important;
}
.leaflet-bar a {
  background-color: #1A1A1A !important;
  color: #E5E5E5 !important;
  border-bottom: 1px solid rgba(255, 255, 255, 0.08) !important;
  transition: all 0.2s ease;
}
.leaflet-bar a:hover {
  background-color: #2C2C2C !important;
  color: #B9784F !important;
}
.leaflet-control-attribution {
  background: rgba(26, 26, 26, 0.8) !important;
  color: #8a8a8a !important;
  font-size: 9px !important;
  border-top-left-radius: 8px;
}
.custom-map-marker {
  background: transparent !important;
  border: none !important;
}

/* Modal animations */
@keyframes modalFadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
@keyframes modalZoomIn {
  from { opacity: 0; transform: scale(0.95); }
  to { opacity: 1; transform: scale(1); }
}

.animate-modal-fade {
  animation: modalFadeIn 0.3s cubic-bezier(0.16, 1, 0.3, 1) forwards;
}
.animate-modal-zoom {
  animation: modalZoomIn 0.4s cubic-bezier(0.16, 1, 0.3, 1) forwards;
}
```

File Documentation: layout.tsx

- **Relative Path:** `src/app/layout.tsx`
- **Line Count:** 92 lines

Purpose & Summary

HTML layout wrapper nesting children inside global session providers.

Architectural Role & Integration

Integrates headers, footers, mobile docking bars, and fonts.

File Structure & Dependencies

- **Imports:** Tracks **5** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **3** export indicators for external module integration.

Functions & APIs Specifications

Function: `RootLayout`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} h-full antialiased`}
      suppressHydrationWarning
    >
      <head>
        {/* DNS prefetch for external resources */}
        <link rel="dns-prefetch" href="https://fonts.googleapis.com" />
        <link rel="dns-prefetch" href="https://fonts.gstatic.com" />
        <link rel="dns-prefetch" href="https://tile.openstreetmap.org" />

        {/* Preconnect for critical third-party origins */}
        <link rel="preconnect" href="https://fonts.googleapis.com" crossOrigin="" />
        <link rel="preconnect" href="https://fonts.gstatic.com" crossOrigin="" />
      </head>
      <body className="min-h-full flex flex-col" suppressHydrationWarning>
        <SessionProviderWrapper>
          <Navbar />
          {children}
        </SessionProviderWrapper>
      </body>
    </html>
  );
}
```

File Documentation: loading.tsx

- **Relative Path:** `src/app/loading.tsx`
- **Line Count:** 13 lines

Purpose & Summary

Top-level application suspense loading layout.

Architectural Role & Integration

Displays pre-loader indicators on initial page mounts.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Loading`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function Loading() {
  return (
    <div className="min-h-screen bg-[#141414] flex items-center justify-center">
      <div className="flex flex-col items-center gap-4">
        <div className="w-10 h-10 border-2 border-[#B9784F] border-t-transparent roun
        <p className="text-[#8a8a8a] text-xs uppercase tracking-widest font-bold anim
          Loading Experience...
        </p>
      </div>
    </div>
  );
}
```

File Documentation: page.tsx

- **Relative Path:** `src/app/login/page.tsx`
- **Line Count:** 209 lines

Purpose & Summary

Role-aware entry login form.

Architectural Role & Integration

Triggers Next-Auth credentials login with fields for guests vs. staff.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `LoginPage`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function LoginPage() {
  const router = useRouter();
  const [isSignUp, setIsSignUp] = useState(false);
  const [roleType, setRoleType] = useState('GUEST'); // GUEST or STAFF

  // Form Fields
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [fullName, setFullName] = useState('');
  const [showPassword, setShowPassword] = useState(false);

  // States
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    if (isSignUp) {
      // Sign Up Guest Flow
      try {
        const res = await fetch('/api/signup', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ fullName, email, password }),
        });
        const data = await res.json();
        if (!res.ok) {
          setError(data.error || 'Registration failed');
          setLoading(false);
          return;
        }

        // Auto log in after sign up
        const loginRes = await signIn('credentials', {
          redirect: false,
          email,
          password,
          roleType: 'GUEST',
        });

        if (loginRes?.error) {
          setError('Sign up success, but login failed. Please sign in manually.');
        } else {
          router.push('/dashboard');
        }
      } catch (err: any) {
        setError('An unexpected error occurred during sign up.');
      } finally {
        setLoading(false);
      }
    } else {
      // Sign In Flow
      try {
        const loginRes = await signIn('credentials', {
          redirect: false,
          email,
          password,
```

```
          roleType,
        });

        if (loginRes?.error) {
          setError('Invalid email, password, or login type.');
        } else {
          router.push('/dashboard');
        }
      } catch (err) {
        setError('An error occurred during sign in.');
      } finally {
        setLoading(false);
      }
    }
  };

  return (
    <div className="flex min-h-[85vh] items-center justify-center bg-[#141414] text-[

      {/* Glow Backdrops */}
      <div className="absolute top-[10%] left-[-15%] w-[400px] h-[400px] bg-brand-acc
      <div className="absolute bottom-[10%] right-[-15%] w-[400px] h-[400px] bg-brand

      <div className="w-full max-w-md space-y-8 bg-[#1A1A1A]/80 backdrop-blur-md roun

        <div className="text-center select-none">
          <Compass className="mx-auto h-12 w-12 text-brand-accent animate-spin-slow"
          <h2 className="mt-4 font-heading text-3xl font-normal tracking-tight text-w
            {isSignUp ? 'Join Luxury Horizon' : 'Welcome Back'}
          </h2>
          <p className="mt-2 text-xs text-[#A0A0A0] font-medium">
            {isSignUp ? 'Create your profile to start booking' : 'Access your resort
          </p>
        </div>

        {error && (
          <div className="rounded-xl bg-red-950/40 border border-red-500/20 p-3.5 tex
            {error}
          </div>
        )}

        <form className="mt-8 space-y-6" onSubmit={handleSubmit}>

          {/* Sign In Role Type Switch */}
          {!isSignUp && (
            <div className="grid grid-cols-2 gap-2 rounded-xl bg-[#141414]/50 p-1 bor
              <button
                type="button"
                onClick={() => setRoleType('GUEST')}
                className={`py-2 text-xs font-semibold rounded-lg transition-all curs
              >
                Guest Portal
              </button>
              <button
                type="button"
                onClick={() => setRoleType('STAFF')}
                className={`py-2 text-xs font-semibold rounded-lg transition-all curs
              >
                Staff Portal
              </button>
            </div>
```

```
            )}

            <div className="space-y-4">
              {isSignUp && (
                <div>
                  <label className="block text-[10px] font-bold uppercase tracking-wide
                  <div className="relative">
                    <User className="absolute left-3.5 top-3.5 h-4 w-4 text-stone-500"
                    <input
                      type="text"
                      required
                      value={fullName}
                      onChange={(e) => setFullName(e.target.value)}
                      placeholder="John Doe"
                      className="w-full rounded-xl bg-white/5 border border-white/5 foc
                    />
                  </div>
                </div>
              )}

              <div>
                <label className="block text-[10px] font-bold uppercase tracking-wider
                <div className="relative">
                  <Mail className="absolute left-3.5 top-3.5 h-4 w-4 text-stone-500" />
                  <input
                    type="email"
                    required
                    value={email}
                    onChange={(e) => setEmail(e.target.value)}
                    placeholder="name@example.com"
```

Function: `handleSubmit`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    if (isSignUp) {
      // Sign Up Guest Flow
      try {
        const res = await fetch('/api/signup', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ fullName, email, password }),
        });
        const data = await res.json();
        if (!res.ok) {
          setError(data.error || 'Registration failed');
          setLoading(false);
          return;
        }

        // Auto log in after sign up
        const loginRes = await signIn('credentials', {
          redirect: false,
          email,
          password,
          roleType: 'GUEST',
        });

        if (loginRes?.error) {
          setError('Sign up success, but login failed. Please sign in manually.');
        } else {
          router.push('/dashboard');
        }
      } catch (err: any) {
        setError('An unexpected error occurred during sign up.');
      } finally {
        setLoading(false);
      }
    } else {
      // Sign In Flow
      try {
        const loginRes = await signIn('credentials', {
          redirect: false,
          email,
          password,
          roleType,
        });

        if (loginRes?.error) {
          setError('Invalid email, password, or login type.');
        } else {
          router.push('/dashboard');
        }
      } catch (err) {
        setError('An error occurred during sign in.');
      } finally {
        setLoading(false);
      }
```

```
    }
};
```

File Documentation: page.tsx

- **Relative Path:** `src/app/page.tsx`
- **Line Count:** 1806 lines

Purpose & Summary

Luxury-oriented interactive landing page with Leaflet maps and GSAP animations.

Architectural Role & Integration

Entry point showcase displaying property listings and booking check selectors.

File Structure & Dependencies

- **Imports:** Tracks **11** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `HomePage`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function HomePage() {
  const router = useRouter();
  const { data: session } = useSession();

  // Search parameters states
  const [searchPlace, setSearchPlace] = useState('');
  const [isDestDropdownOpen, setIsDestDropdownOpen] = useState(false);

  // Date Picker States
  const [checkInDate, setCheckInDate] = useState<Date | undefined>(new Date(2026, 5,
  const [checkOutDate, setCheckOutDate] = useState<Date | undefined>(new Date(2026, 5

  // Guest Selector States
  const [adults, setAdults] = useState(2);
  const [children, setChildren] = useState(0);
  const [rooms, setRooms] = useState(1);
  const [isGuestDropdownOpen, setIsGuestDropdownOpen] = useState(false);

  // Other States
  const [resorts, setResorts] = useState<Resort[]>([]);
  const [loading, setLoading] = useState(true);
  const [email, setEmail] = useState('');
  const [isSubscribed, setIsSubscribed] = useState(false);
  const [subscribing, setSubscribing] = useState(false);
  const [showFloatingHeader, setShowFloatingHeader] = useState(false);
  const [favorites, setFavorites] = useState<string[]>([]);
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);
  const [selectedCard, setSelectedCard] = useState<{ src: string; title: string; subt

  // Interactive Calculator States
  const [calcRate, setCalcRate] = useState(380);
  const [calcNights, setCalcNights] = useState(5);
  const [calcGuests, setCalcGuests] = useState(2);

  // New dining/spa interactive states
  const [activeDiningMenu, setActiveDiningMenu] = useState<'breakfast' | 'tea' | 'din
  const [activeSpaService, setActiveSpaService] = useState<'massage' | 'spring' | 'yo

  // Dynamic Background State
  const [activeBgImage, setActiveBgImage] = useState('https://images.unsplash.com/pho

  // Refs for closing dropdowns
  const destRef = useRef<HTMLDivElement>(null);
  const guestRef = useRef<HTMLDivElement>(null);
  const destScrollRef = useRef<HTMLDivElement>(null);

  // GSAP animation container refs
  const rootRef = useRef<HTMLDivElement>(null);
  const heroPinRef = useRef<HTMLDivElement>(null);
  const bookingBarRef = useRef<HTMLDivElement>(null);
  const staysSectionRef = useRef<HTMLDivElement>(null);
  const diningSectionRef = useRef<HTMLDivElement>(null);
  const spaSectionRef = useRef<HTMLDivElement>(null);
  const excursionsSectionRef = useRef<HTMLDivElement>(null);
  const statsSectionRef = useRef<HTMLDivElement>(null);
  const featuresSectionRef = useRef<HTMLDivElement>(null);
  const destinationSectionRef = useRef<HTMLDivElement>(null);
  const testimonialSectionRef = useRef<HTMLDivElement>(null);
  const calcSectionRef = useRef<HTMLDivElement>(null);
  const newsletterSectionRef = useRef<HTMLDivElement>(null);
```

```
  // Leaflet Map Refs
  const mapContainerRef = useRef<HTMLDivElement>(null);
  const mapInstanceRef = useRef<any>(null);

  // Fetch resorts
  useEffect(() => {
    async function getResorts() {
      try {
        const res = await fetch('/api/resorts?page=1&limit=12');
        const data = await res.json();
        if (res.ok) setResorts(data.resorts || []);
      } catch (e) {
        console.error(e);
      } finally {
        setLoading(false);
      }
    }
    getResorts();
  }, []);

  // Handle outside clicks
  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
      if (destRef.current && !destRef.current.contains(event.target as Node)) {
        setIsDestDropdownOpen(false);
      }
      if (guestRef.current && !guestRef.current.contains(event.target as Node)) {
        setIsGuestDropdownOpen(false);
      }
    }
    document.addEventListener('mousedown', handleClickOutside);
    return () => document.removeEventListener('mousedown', handleClickOutside);
  }, []);

  // Sticky navigation threshold
  useEffect(() => {
    const handleScroll = () => {
      setShowFloatingHeader(window.scrollY > 150);
    };
    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  // Initialize Leaflet Map (Client-side only)
  useEffect(() => {
    if (loading) return;

    let active = true;
    let localMap: any = null;

    async function initMap() {
      if (!mapContainerRef.current) return;

      const L = await import('leaflet');
      if (!active) return;
      if (mapInstanceRef.current) return;

      // Initialize map container with global zoom-out
      localMap = L.map(mapContainerRef.current, {
        zoomControl: true,
```

```
        scrollWheelZoom: false, // Prevents scroll hijacking on snap scroll
      }).setView([20, 0], 2);

      L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        attribution: '&copy; OpenStreetMap'
      }).addTo(localMap);

      mapInstanceRef.current = localMap;

      // Add pins for current resorts
      resorts.forEach((resort) => {
        const coords = getCoords(resort.name || resort.location);
        if (coords) {
          const marker = L.marker(coords, {
            icon: L.divIcon({
              className: 'custom-map-marker',
              html: `<div class="w-4 h-4 rounded-full bg-brand-accent border-2 border
              iconSize: [16, 16],
              iconAnchor: [8, 8]
            })
          }).addTo(localMap);

          marker.bindPopup(`
            <div class="text-left font-sans p-1">
              <span class="block text-xs font-bold text-[#141414]">${resort.name}</sp
              <span class="block text-[9px] text-[#666] mt-0.5">${resort.location}</s
            </div>
          `);
        }
      });
```

Function: `getResorts`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
async function getResorts() {
  try {
    const res = await fetch('/api/resorts?page=1&limit=12');
    const data = await res.json();
    if (res.ok) setResorts(data.resorts || []);
  } catch (e) {
    console.error(e);
  } finally {
    setLoading(false);
  }
}
```

Function: `handleClickOutside`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function handleClickOutside(event: MouseEvent) {
  if (destRef.current && !destRef.current.contains(event.target as Node)) {
    setIsDestDropdownOpen(false);
  }
  if (guestRef.current && !guestRef.current.contains(event.target as Node)) {
    setIsGuestDropdownOpen(false);
  }
}
```

Function: `handleScroll`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
    const handleScroll = () => {
      setShowFloatingHeader(window.scrollY > 150);
    };
```

Function: `initMap`

Inputs/Parameters: None.

Outputs/Returns: `void` - Instantiates Leaflet.

Internal Logic & Purpose:

> Generates vector maps inside container. Maps resort locations to coordinates and binds descriptions to custom pin overlays.

Actual Function Source Code:

```
async function initMap() {
  if (!mapContainerRef.current) return;

  const L = await import('leaflet');
  if (!active) return;
  if (mapInstanceRef.current) return;

  // Initialize map container with global zoom-out
  localMap = L.map(mapContainerRef.current, {
    zoomControl: true,
    scrollWheelZoom: false, // Prevents scroll hijacking on snap scroll
  }).setView([20, 0], 2);

  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '&copy; OpenStreetMap'
  }).addTo(localMap);

  mapInstanceRef.current = localMap;

  // Add pins for current resorts
  resorts.forEach((resort) => {
    const coords = getCoords(resort.name || resort.location);
    if (coords) {
      const marker = L.marker(coords, {
        icon: L.divIcon({
          className: 'custom-map-marker',
          html: `<div class="w-4 h-4 rounded-full bg-brand-accent border-2 border
          iconSize: [16, 16],
          iconAnchor: [8, 8]
        })
      }).addTo(localMap);

      marker.bindPopup(`
        <div class="text-left font-sans p-1">
          <span class="block text-xs font-bold text-[#141414]">${resort.name}</sp
          <span class="block text-[9px] text-[#666] mt-0.5">${resort.location}</s
        </div>
      `);
    }
  });
}
```

Function: `playHeroAnimations`

Inputs/Parameters: None.

Outputs/Returns: `void` - Animates hero.

Internal Logic & Purpose:

Configures GSAP timelines for text reveals and page scrolling triggers.

Actual Function Source Code:

```
        const playHeroAnimations = () => {
          gsap.fromTo('.hero-animate-tag',
            { opacity: 0, y: 30 },
            { opacity: 1, y: 0, duration: 0.8, ease: 'power3.out' }
          );
          gsap.fromTo('.hero-animate-title',
            { opacity: 0, y: 40 },
            { opacity: 1, y: 0, duration: 1, delay: 0.15, ease: 'power3.out' }
          );
          gsap.fromTo('.hero-animate-desc',
            { opacity: 0, y: 30 },
            { opacity: 1, y: 0, duration: 0.8, delay: 0.3, ease: 'power3.out' }
          );
          gsap.fromTo('.hero-animate-cta',
            { opacity: 0, y: 30 },
            { opacity: 1, y: 0, duration: 0.8, delay: 0.45, ease: 'power3.out' }
          );
          gsap.fromTo('.hero-animate-card',
            { opacity: 0, scale: 0.8, rotateY: 30 },
            { opacity: 1, scale: 1, rotateY: 0, duration: 1.2, delay: 0.3, stagger: 0
          );
        };
```

Function: `onMouseMove`

Inputs/Parameters: `e: MouseEvent` - Cursor movement event.

Outputs/Returns: `void` - Triggers tilt animations.

Internal Logic & Purpose:

> Alters page image dimensions using perspective transform ratios.

Actual Function Source Code:

```
            const onMouseMove = (e: MouseEvent) => {
              const rect = hero.getBoundingClientRect();
              const x = (e.clientX - rect.left - rect.width / 2) / (rect.width / 2);
              const y = (e.clientY - rect.top - rect.height / 2) / (rect.height / 2);

              gsap.to('.floating-card-1', {
                x: x * 30,
                y: y * 30,
                rotateY: x * 15,
                rotateX: -y * 15,
                duration: 0.8,
                ease: 'power2.out',
                overwrite: 'auto'
              });

              gsap.to('.floating-card-2', {
                x: x * -20,
                y: y * -20,
                rotateY: x * 25,
                rotateX: -y * 25,
                duration: 1,
                ease: 'power2.out',
                overwrite: 'auto'
              });

              gsap.to('.floating-card-3', {
                x: x * 40,
                y: y * -30,
                rotateY: x * -12,
                rotateX: -y * 12,
                duration: 0.9,
                ease: 'power2.out',
                overwrite: 'auto'
              });

              gsap.to('.floating-card-4', {
                x: x * -25,
                y: y * 20,
                rotateY: x * -15,
                rotateX: -y * 15,
                duration: 0.85,
                ease: 'power2.out',
                overwrite: 'auto'
              });

              gsap.to('.floating-card-5', {
                x: x * 20,
                y: y * -15,
                rotateY: x * 18,
                rotateX: -y * 18,
                duration: 0.95,
                ease: 'power2.out',
                overwrite: 'auto'
              });

              gsap.to('.hero-parallax-text', {
                x: x * 12,
                y: y * 12,
                duration: 0.7,
                ease: 'power2.out',
```

```
            overwrite: 'auto'
          });
        };
```

Function: `handleSearch`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();
    const query = searchPlace.trim();
    const checkInParam = checkInDate ? `&checkIn=${checkInDate.toISOString()}` : '';
    const checkOutParam = checkOutDate ? `&checkOut=${checkOutDate.toISOString()}` :
    const guestsParam = `&guests=${adults + children}&rooms=${rooms}`;

    let url = '/resorts';
    if (query) {
      url += `?query=${encodeURIComponent(query)}${checkInParam}${checkOutParam}${gue
    } else {
      url += `?${checkInParam.slice(1)}${checkOutParam}${guestsParam}`;
    }
    router.push(url);
  };
```

Function: `handleSubscribe`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleSubscribe = (e: React.FormEvent) => {
    e.preventDefault();
    if (!email.trim() || !email.includes('@')) return;
    setSubscribing(true);
    setTimeout(() => {
      setSubscribing(false);
      setIsSubscribed(true);
      setEmail('');
    }, 1200);
  };
```

Function: `toggleFavorite`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const toggleFavorite = (id: string, e: React.MouseEvent) => {
    e.stopPropagation();
    setFavorites(prev =>
      prev.includes(id) ? prev.filter(favId => favId !== id) : [...prev, id]
    );
  };
```

Function: `getFormattedGuests`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const getFormattedGuests = () => {
    const total = adults + children;
    return `${total} guest${total > 1 ? 's' : ''}, ${rooms} room${rooms > 1 ? 's' : '
  };
```

Function: `scrollCarousel`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const scrollCarousel = (ref: React.RefObject<HTMLDivElement | null>, direction: 'le
    if (ref.current) {
      const { scrollLeft, clientWidth } = ref.current;
      const offset = direction === 'left' ? -clientWidth / 2 : clientWidth / 2;
      ref.current.scrollTo({ left: scrollLeft + offset, behavior: 'smooth' });
    }
  };
```

Function: `getSuggestions`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const getSuggestions = () => {
    const matchedResorts = resorts
      .filter(r => r.name.toLowerCase().includes(searchPlace.toLowerCase()) || r.loca
      .map(r => ({ name: r.name, type: 'Resort' }));

    const matchedLocations = resorts
      .filter(r => r.location.toLowerCase().includes(searchPlace.toLowerCase()))
      .map(r => ({ name: r.location, type: 'Location' }));

    const all = [...popularDestinations, ...matchedResorts, ...matchedLocations];
    const unique = Array.from(new Map(all.map(item => [item.name, item])).values());

    if (!searchPlace.trim()) {
      return popularDestinations;
    }
    return unique.filter(item => item.name.toLowerCase().includes(searchPlace.toLower
  };
```

Function: `matchClassification`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
const matchClassification = () => {
  if (calcRate < 250) return 'Standard Pavilion';
  if (calcRate < 500) return 'Executive Lagoon Suite';
  if (calcRate < 1000) return 'Premium Overwater Villa';
  return 'Royal Ocean Sanctuary';
};
```

File Documentation: page.tsx

- **Relative Path:** `src/app/resorts/page.tsx`
- **Line Count:** 341 lines

Purpose & Summary

Standalone resorts browser browse container.

Architectural Role & Integration

Shows listings, reviews, ratings, and locations.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `ResortsBrowseContent`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function ResortsBrowseContent() {
  const router = useRouter();
  const searchParams = useSearchParams();

  // Pagination states
  const [resorts, setResorts] = useState<Resort[]>([]);
  const [loading, setLoading] = useState(true);
  const [total, setTotal] = useState(0);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);

  const [searchQuery, setSearchQuery] = useState('');
  const [selectedCategory, setSelectedCategory] = useState('all');
  const [showAllRooms, setShowAllRooms] = useState(false);

  // Sync state from query parameters on mount or query change
  useEffect(() => {
    const urlQuery = searchParams.get('query') || '';
    setSearchQuery(urlQuery);
  }, [searchParams]);

  // Load resorts whenever the search params, page, or category changes
  useEffect(() => {
    const loadAllResorts = async () => {
      setLoading(true);
      try {
        const urlQuery = searchParams.get('query') || '';
        const res = await fetch(`/api/resorts?page=${page}&limit=9&query=${encodeURIC
        const data = await res.json();
        if (res.ok) {
          setResorts(data.resorts || []);
          setTotal(data.total || 0);
          setTotalPages(data.totalPages || 1);
        }
      } catch (e) {
        console.error('Error browsing resorts:', e);
      } finally {
        setLoading(false);
      }
    };

    loadAllResorts();
  }, [searchParams, page, selectedCategory]);

  const handleSearchSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    setPage(1);
    const params = new URLSearchParams(searchParams.toString());
    if (searchQuery.trim()) {
      params.set('query', searchQuery.trim());
    } else {
      params.delete('query');
    }
    router.replace(`/resorts?${params.toString()}`);
  };

  return (
    <div className="w-full min-h-screen bg-[#141414] text-[#E5E5E5] pb-24 pt-24 relat

      {/* Glow Backdrops */}
```

```
        <div className="absolute top-[10%] left-[-15%] w-[500px] h-[500px] bg-brand-acc
        <div className="absolute bottom-[30%] right-[-15%] w-[500px] h-[500px] bg-brand

        {/* 1. Header Hero section */}
        <section className="py-16 px-4 sm:px-8 text-center space-y-6 relative z-10">
          <div className="inline-flex items-center gap-1.5 rounded-full bg-brand-accent
            <span>Global Resort Collection</span>
          </div>
          <h1 className="font-heading text-4xl sm:text-6xl font-normal tracking-tight t
            Browse All Resorts & Rooms
          </h1>
          <p className="mx-auto max-w-xl text-[#A0A0A0] text-xs sm:text-sm font-medium
            Explore our collection of premium properties, map accommodations, compare p
          </p>

          {/* Dynamic Search Parameters Bar */}
          <form onSubmit={handleSearchSubmit} className="mx-auto max-w-2xl bg-[#1A1A1A]
            <div className="flex items-center gap-2 pl-4 flex-grow">
              <Search className="h-4.5 w-4.5 text-[#555] shrink-0" />
              <input
                type="text"
                placeholder="Search resorts, locations, regions..."
                value={searchQuery}
                onChange={(e) => setSearchQuery(e.target.value)}
                className="bg-transparent text-xs text-white outline-none w-full placeh
              />
            </div>
            <button
              type="submit"
              className="bg-brand-accent hover:bg-brand-accent-hover text-white font-bo
            >
              Search Collection
            </button>
          </form>
        </section>

        {/* 2. MAIN LAYOUT AND FILTERS */}
        <section className="mx-auto max-w-7xl px-4 sm:px-6 lg:px-8 py-6 space-y-10 rela

          {/* Category Filters */}
          <div className="flex flex-col md:flex-row gap-4 items-center justify-between
            <div className="flex gap-2 overflow-x-auto pb-2 scrollbar-none justify-star
              {[
                { id: 'all', label: 'All Collections', icon: '🏝️' },
                { id: 'tropical', label: 'Tropical Beach', icon: '🏄' },
                { id: 'alpine', label: 'Alpine Peaks', icon: '⛰️' },
                { id: 'coastal', label: 'Coastal Cliffs', icon: '🌊' },
                { id: 'forest', label: 'Forest Eco', icon: '🌲' }
              ].map((cat) => (
                <button
                  key={cat.id}
                  onClick={() => {
                    setSelectedCategory(cat.id);
                    setPage(1);
                  }}
                  className={`px-5 py-2.5 rounded-full border text-[11px] font-bold upp
                    selectedCategory === cat.id
                      ? 'border-brand-accent/35 bg-brand-accent/10 text-brand-accent sh
                      : 'border-white/5 bg-[#1A1A1A]/40 text-[#A0A0A0] hover:border-whi
                  }`}
                >
```

```
                <span>{cat.icon}</span>
                <span>{cat.label}</span>
              </button>
            ))}
          </div>

          <button
            onClick={() => setShowAllRooms(!showAllRooms)}
            className={`px-6 py-2.5 rounded-full border text-[11px] font-bold upperca
              showAllRooms
                ? 'bg-brand-accent text-white shadow-brand-accent/10'
                : 'bg-transparent text-brand-accent hover:bg-brand-accent/5'
            }`}
          >
            <Layers className="h-4 w-4" />
            <span>{showAllRooms ? 'Hide Rooms Globally' : 'Show Rooms Globally'}</spa
          </button>
        </div>

        {/* Results Info */}
        <div className="flex justify-between items-center text-xs text-[#8a8a8a] uppe
          <span>Found {total} properties</span>
          <span>Showing page {page} of {totalPages}</span>
        </div>

        {/* Resorts Grid */}
        {loading ? (
          <div className="flex flex-col items-center justify-center py-24 space-y-3">
            <Loader2 className="h-8 w-8 text-brand-accent animate-spin" />
            <span className="text-xs text-[#8a8a8a] font-bold uppercase">Loading prop
```

Function: `loadAllResorts`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
const loadAllResorts = async () => {
  setLoading(true);
  try {
    const urlQuery = searchParams.get('query') || '';
    const res = await fetch(`/api/resorts?page=${page}&limit=9&query=${encodeURIC
    const data = await res.json();
    if (res.ok) {
      setResorts(data.resorts || []);
      setTotal(data.total || 0);
      setTotalPages(data.totalPages || 1);
    }
  } catch (e) {
    console.error('Error browsing resorts:', e);
  } finally {
    setLoading(false);
  }
};
```

Function: `handleSearchSubmit`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleSearchSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    setPage(1);
    const params = new URLSearchParams(searchParams.toString());
    if (searchQuery.trim()) {
      params.set('query', searchQuery.trim());
    } else {
      params.delete('query');
    }
    router.replace(`/resorts?${params.toString()}`);
  };
```

Function: `ResortsBrowsePage`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function ResortsBrowsePage() {
  return (
    <Suspense fallback={
      <div className="flex flex-col items-center justify-center min-h-screen space-y-
        <Loader2 className="h-8 w-8 text-brand-accent animate-spin" />
        <span className="text-xs text-[#8a8a8a] font-bold uppercase tracking-wider an
          Loading Search Collection...
        </span>
      </div>
    }>
      <ResortsBrowseContent />
    </Suspense>
  );
}
```

File Documentation: AnimateOnScroll.tsx

- **Relative Path:** `src/components/AnimateOnScroll.tsx`
- **Line Count:** 81 lines

Purpose & Summary

GSAP-powered scroll entry animation wrapper.

Architectural Role & Integration

Injects entry transitions (fades, slides, zooms) on target items.

File Structure & Dependencies

- **Imports:** Tracks **1** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `AnimateOnScroll`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function AnimateOnScroll({
  children,
  variant = 'fade-up',
  delay = 0,
  duration = 800,
  className = ''
}: AnimateOnScrollProps) {
  const domRef = useRef<HTMLDivElement>(null);
  const [isVisible, setVisible] = useState(false);

  useEffect(() => {
    const observer = new IntersectionObserver(entries => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
          setVisible(true);
          observer.unobserve(entry.target);
        }
      });
    }, {
      threshold: 0.1
    });

    if (domRef.current) {
      observer.observe(domRef.current);
    }

    return () => {
      if (domRef.current) {
        observer.unobserve(domRef.current);
      }
    };
  }, []);

  const getVariantStyles = () => {
    switch (variant) {
      case 'fade-up':
        return isVisible
          ? 'translate-y-0 opacity-100'
          : 'translate-y-10 opacity-0';
      case 'scale-in':
        return isVisible
          ? 'scale-100 opacity-100'
          : 'scale-95 opacity-0';
      case 'rotate-in':
        return isVisible
          ? 'rotate-0 translate-y-0 opacity-100'
          : '-rotate-2 translate-y-8 opacity-0';
      case 'fade-in':
        return isVisible
          ? 'opacity-100'
          : 'opacity-0';
      default:
        return '';
    }
  };

  return (
    <div
      ref={domRef}
      className={`transition-all ease-[cubic-bezier(0.16,1,0.3,1)] ${getVariantStyles
```

```
        style={{
          transitionDelay: `${delay}ms`,
          transitionDuration: `${duration}ms`
        }}
      >
        {children}
      </div>
    );
  }
```

Function: `getVariantStyles`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const getVariantStyles = () => {
    switch (variant) {
      case 'fade-up':
        return isVisible
          ? 'translate-y-0 opacity-100'
          : 'translate-y-10 opacity-0';
      case 'scale-in':
        return isVisible
          ? 'scale-100 opacity-100'
          : 'scale-95 opacity-0';
      case 'rotate-in':
        return isVisible
          ? 'rotate-0 translate-y-0 opacity-100'
          : '-rotate-2 translate-y-8 opacity-0';
      case 'fade-in':
        return isVisible
          ? 'opacity-100'
          : 'opacity-0';
      default:
        return '';
    }
  };
```

File Documentation: Footer.tsx

- **Relative Path:** `src/components/Footer.tsx`
- **Line Count:** 214 lines

Purpose & Summary

Global interactive footer layout.

Architectural Role & Integration

Embeds newsletter signups, navigation links, and back-to-top micro-interactions.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Footer`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function Footer() {
  const pathname = usePathname();
  const [showScrollTop, setShowScrollTop] = useState(false);
  const [footerEmail, setFooterEmail] = useState('');
  const [footerSubscribed, setFooterSubscribed] = useState(false);
  const [footerSubscribing, setFooterSubscribing] = useState(false);

  const handleFooterSubscribe = (e: React.FormEvent) => {
    e.preventDefault();
    if (!footerEmail.trim() || !footerEmail.includes('@')) return;
    setFooterSubscribing(true);
    setTimeout(() => {
      setFooterSubscribing(false);
      setFooterSubscribed(true);
      setFooterEmail('');
    }, 1200);
  };

  useEffect(() => {
    const handleScroll = () => {
      if (window.scrollY > 400) {
        setShowScrollTop(true);
      } else {
        setShowScrollTop(false);
      }
    };
    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  const scrollToTop = () => {
    window.scrollTo({ top: 0, behavior: 'smooth' });
  };

  // Only hide on dashboard routes
  if (pathname.startsWith('/dashboard')) return null;

  const year = new Date().getFullYear();

  return (
    <footer className="relative w-full bg-[#0a0a0a] text-stone-400 border-t border-st
      {/* Main footer grid */}
      <div className="mx-auto max-w-7xl px-6 md:px-10 lg:px-16 py-14 md:py-20">
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-10 lg:gap-

          {/* Column 1: Logo & Tagline */}
          <div className="space-y-5">
            <Link href="/" className="inline-block group">
              <span className="font-sans text-2xl font-black tracking-tight text-whit
                ESKAP<span className="text-orange-500 italic">INN</span>
              </span>
            </Link>
            <p className="text-[13px] text-stone-500 leading-relaxed max-w-xs font-me
              Curating extraordinary stays across 30+ destinations worldwide. From ov
            </p>
            {/* Social icons */}
            <div className="flex items-center gap-3 pt-2">
              {[
                { label: 'Instagram', path: 'M16 11.37A4 4 0 1 1 12.63 8 4 4 0 0 1 16
                { label: 'Facebook', path: 'M9 8h-3v4h3v12h5v-12h3.642l.358-4h-4v-1.6
```

```
                { label: 'X', path: 'M18.244 2.25h3.308l-7.227 8.26 8.502 11.24H16.17
                { label: 'YouTube', path: 'M22.54 6.42a2.78 2.78 0 0 0-1.94-2C18.88 4
              ].map((social, i) => (
                <a key={i} href="#" aria-label={social.label}
                  className="w-9 h-9 rounded-full bg-stone-900 border border-stone-80
                  <svg className="h-3.5 w-3.5" fill="currentColor" viewBox="0 0 24 24
                    {social.rect && <rect x="2" y="2" width="20" height="20" rx="5" r
                    <path d={social.path} />
                  </svg>
                </a>
              ))}
            </div>
          </div>

          {/* Column 2: Quick Links */}
          <div className="space-y-5">
            <h4 className="text-[12px] font-bold text-white uppercase tracking-[0.15e
            <ul className="space-y-3">
              {[
                { label: 'All Destinations', href: '/resorts' },
                { label: 'Hotels & Resorts', href: '/resorts' },
                { label: 'About Us', href: '/about' },
                { label: 'Sign In', href: '/login' },
                { label: 'Dashboard', href: '/dashboard' },
                { label: 'Contact Support', href: '/about' },
              ].map((link, i) => (
                <li key={i}>
                  <Link href={link.href}
                    className="text-[13px] text-stone-500 hover:text-white transition
                    <span className="w-1.5 h-1.5 rounded-full bg-stone-800 group-hove
                    <span className="relative pb-0.5 overflow-hidden">
                      {link.label}
                      <span className="absolute bottom-0 left-0 w-0 h-0.5 bg-orange-5
                    </span>
                  </Link>
                </li>
              ))}
            </ul>
          </div>

          {/* Column 3: Top Destinations */}
          <div className="space-y-5">
            <h4 className="text-[12px] font-bold text-white uppercase tracking-[0.15e
            <ul className="space-y-3">
              {[
                'Maldives', 'Bali, Indonesia', 'Santorini, Greece',
                'Dubai, UAE', 'Phuket, Thailand', 'Bora Bora',
                'Amalfi Coast, Italy', 'Maui, Hawaii',
              ].map((dest, i) => (
                <li key={i}>
                  <Link href={`/resorts?query=${encodeURIComponent(dest.split(',')[0]
                    className="text-[13px] text-stone-500 hover:text-white transition
                    <MapPin className="h-3 w-3 text-stone-700 group-hover:text-orange
                    <span className="relative pb-0.5 overflow-hidden">
                      {dest}
                      <span className="absolute bottom-0 left-0 w-0 h-0.5 bg-orange-5
                    </span>
                  </Link>
                </li>
              ))}
            </ul>
```

```
            </div>

            {/* Column 4: Contact & Newsletter */}
            <div className="space-y-5">
              <h4 className="text-[12px] font-bold text-white uppercase tracking-[0.15e
              <ul className="space-y-3 text-[13px] font-semibold">
                <li className="flex items-center gap-2.5 hover:text-stone-300 transitio
                  <div className="w-8 h-8 rounded-lg bg-stone-900 border border-stone-8
                    <MapPin className="h-3.5 w-3.5 text-orange-500" />
                  </div>
                  <span>750 Luxury Promenade, Maldives</span>
                </li>
                <li className="flex items-center gap-2.5 hover:text-stone-300 transitio
                  <div className="w-8 h-8 rounded-lg bg-stone-900 border border-stone-8
                    <Phone className="h-3.5 w-3.5 text-orange-500" />
                  </div>
                  <span>+1 800-ESKAP-INN</span>
                </li>
                <li className="flex items-center gap-2.5 hover:text-stone-300 transitio
                  <div className="w-8 h-8 rounded-lg bg-stone-900 border border-stone-8
                    <Mail className="h-3.5 w-3.5 text-orange-500" />
                  </div>
                  <span>concierge@eskapinn.com</span>
                </li>
              </ul>

              {/* Mini newsletter */}
              <div className="pt-3">
                <span className="text-[11px] text-stone-500 font-bold uppercase trackin
                {footerSubscribed ? (
```

Function: `handleFooterSubscribe`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleFooterSubscribe = (e: React.FormEvent) => {
    e.preventDefault();
    if (!footerEmail.trim() || !footerEmail.includes('@')) return;
    setFooterSubscribing(true);
    setTimeout(() => {
      setFooterSubscribing(false);
      setFooterSubscribed(true);
      setFooterEmail('');
    }, 1200);
  };
```

Function: `handleScroll`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
    const handleScroll = () => {
      if (window.scrollY > 400) {
        setShowScrollTop(true);
      } else {
        setShowScrollTop(false);
      }
    };
```

Function: `scrollToTop`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
const scrollToTop = () => {
  window.scrollTo({ top: 0, behavior: 'smooth' });
};
```

File Documentation: MobileDock.tsx

- **Relative Path:** `src/components/MobileDock.tsx`
- **Line Count:** 42 lines

Purpose & Summary

Fixed bottom dock navigation for mobile screens.

Architectural Role & Integration

Adapts header navigation into floating action nodes for smaller layouts.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `MobileDock`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function MobileDock() {
  const pathname = usePathname();

  const navItems = [
    { label: 'Home', href: '/', icon: Home },
    { label: 'Explore', href: '/book', icon: Search },
    { label: 'Bookings', href: '/dashboard', icon: Calendar },
    { label: 'Profile', href: '/login', icon: User }
  ];

  return (
    <div className="fixed bottom-6 left-1/2 -translate-x-1/2 z-50 flex items-center j
      {navItems.map((item) => {
        const Icon = item.icon;
        const isActive = pathname === item.href;

        return (
          <Link
            key={item.href}
            href={item.href}
            className={`flex flex-col items-center gap-1 transition-all duration-300
              isActive ? 'text-amber-400 scale-110' : 'text-stone-400 hover:text-ston
            }`}
          >
            <Icon className="h-5 w-5" />
            <span className="text-[9px] font-semibold uppercase tracking-wider font-s
            {isActive && (
              <span className="absolute -bottom-1 left-1/2 -translate-x-1/2 w-1 h-1 b
            )}
          </Link>
        );
      })}
    </div>
  );
}
```

File Documentation: Navbar.tsx

- **Relative Path:** `src/components/Navbar.tsx`
- **Line Count:** 137 lines

Purpose & Summary

Global application header navigation.

Architectural Role & Integration

Adapts user paths based on active sessions (Login/Logout buttons, Dashboard links).

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Navbar`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function Navbar() {
  const router = useRouter();
  const pathname = usePathname();
  const { data: session } = useSession();
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);
  const [showFloatingHeader, setShowFloatingHeader] = useState(false);

  useEffect(() => {
    const handleScroll = () => {
      setShowFloatingHeader(window.scrollY > 150);
    };
    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  const isActive = (path: string) => {
    if (path === '/resorts' && pathname?.startsWith('/resorts')) return true;
    return pathname === path;
  };

  const linkStyle = (path: string) => {
    return `transition-colors uppercase tracking-[0.2em] text-[11px] font-semibold ${
      isActive(path) ? 'text-brand-accent' : 'text-[#A0A0A0] hover:text-white'
    }`;
  };

  return (
    <nav className={`fixed top-0 left-0 right-0 z-50 transition-all duration-500 bord
      showFloatingHeader || pathname !== '/'
        ? 'bg-[#141414]/90 backdrop-blur-md py-4 px-4 md:px-16 border-white/10 shadow
        : 'bg-transparent py-6 px-4 md:px-16 border-transparent'
    }`}>
      <div className="flex items-center justify-between">
        {/* Brand logo */}
        <div className="flex items-center gap-2 cursor-pointer select-none" onClick={
          <span className="font-heading text-lg md:text-2xl font-semibold tracking-wi
            BOOKME<span className="text-brand-accent">.COM</span>
          </span>
        </div>

        {/* Center menu links */}
        <div className="hidden md:flex items-center gap-10">
          <Link href="/book" className={linkStyle('/book')}>
            Stays Map
          </Link>
          <Link href="/resorts" className={linkStyle('/resorts')}>
            Destinations
          </Link>
          <Link href="/about" className={linkStyle('/about')}>
            About & Experiences
          </Link>
        </div>

        {/* Right action button */}
        <div className="flex items-center gap-4">
          <div className="hidden md:block">
            {session ? (
              <div className="flex items-center gap-4">
                <button
                  onClick={() => router.push('/dashboard')}
```

```
                  className="bg-brand-accent hover:bg-brand-accent-hover text-white t
                >
                  Dashboard
                </button>
                <button
                  onClick={() => signOut({ callbackUrl: '/' })}
                  className="text-[#A0A0A0] hover:text-white text-[11px] font-bold up
                >
                  Logout
                </button>
              </div>
            ) : (
              <button
                onClick={() => router.push('/login')}
                className="bg-brand-accent hover:bg-brand-accent-hover text-white tex
              >
                Sign In
              </button>
            )}
          </div>

          {/* Mobile hamburger */}
          <button
            onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
            className="md:hidden flex flex-col items-center justify-center w-9 h-9 ga
            aria-label="Toggle menu"
          >
            <span className={`w-5 h-[1.5px] bg-white transition-all duration-300 ${mo
            <span className={`w-5 h-[1.5px] bg-white transition-all duration-300 ${mo
            <span className={`w-5 h-[1.5px] bg-white transition-all duration-300 ${mo
          </button>
        </div>
      </div>

      {/* Mobile dropdown menu */}
      <div className={`md:hidden overflow-hidden transition-all duration-400 ease-in-
        <div className="flex flex-col gap-4 py-4 border-t border-white/10">
          <Link href="/book" onClick={() => setMobileMenuOpen(false)} className="text
          <Link href="/resorts" onClick={() => setMobileMenuOpen(false)} className="t
          <Link href="/about" onClick={() => setMobileMenuOpen(false)} className="tex
          <div className="pt-2 border-t border-white/5 flex flex-col gap-3">
            {session ? (
              <>
                <button
                  onClick={() => { setMobileMenuOpen(false); router.push('/dashboard'
                  className="w-full bg-brand-accent hover:bg-brand-accent-hover text-
                >
                  Dashboard
                </button>
                <button
                  onClick={() => { setMobileMenuOpen(false); signOut({ callbackUrl: '
                  className="w-full border border-white/10 hover:border-white/20 text
                >
                  Logout
                </button>
              </>
            ) : (
              <button
                onClick={() => { setMobileMenuOpen(false); router.push('/login'); }}
                className="w-full bg-brand-accent hover:bg-brand-accent-hover text-wh
              >
```

```
                Sign In
              </button>
            )}
          </div>
        </div>
      </div>
    </nav>
  );
}
```

Function: `handleScroll`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleScroll = () => {
    setShowFloatingHeader(window.scrollY > 150);
  };
```

Function: `isActive`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
const isActive = (path: string) => {
  if (path === '/resorts' && pathname?.startsWith('/resorts')) return true;
  return pathname === path;
};
```

Function: `linkStyle`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
const linkStyle = (path: string) => {
  return `transition-colors uppercase tracking-[0.2em] text-[11px] font-semibold ${
    isActive(path) ? 'text-brand-accent' : 'text-[#A0A0A0] hover:text-white'
  }`;
};
```

File Documentation: ResortDetailsClient.tsx

- **Relative Path:** `src/components/ResortDetailsClient.tsx`
- **Line Count:** 450 lines

Purpose & Summary

Client-side date and service booking selector.

Architectural Role & Integration

Handles check-in/out calendars, guest count adjustments, add-ons calculation, and booking submissions.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `ResortDetailsClient`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function ResortDetailsClient({ resort, services }: ResortDetailsClient
  const router = useRouter();
  const { data: session } = useSession();

  // Extract unique room types available at this resort
  const roomTypesMap: { [key: string]: RoomType } = {};
  resort.rooms.forEach(r => {
    roomTypesMap[r.roomType.id] = r.roomType;
  });
  const availableRoomTypes = Object.values(roomTypesMap);

  // Active States
  const [selectedRoomTypeId, setSelectedRoomTypeId] = useState(
    availableRoomTypes.length > 0 ? availableRoomTypes[0].id : ''
  );
  const [activeImageIdx, setActiveImageIdx] = useState(0);

  // Reservation states
  const [checkIn, setCheckIn] = useState('');
  const [checkOut, setCheckOut] = useState('');
  const [numGuests, setNumGuests] = useState('2');
  const [selectedServices, setSelectedServices] = useState<string[]>([]);

  // Calculation outputs
  const [nights, setNights] = useState(0);
  const [basePriceTotal, setBasePriceTotal] = useState(0);
  const [servicesTotal, setServicesTotal] = useState(0);
  const [grandTotal, setGrandTotal] = useState(0);

  // Submit states
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  // Calculate live estimate when checkIn, checkOut, selectedServices, or selectedRoo
  useEffect(() => {
    if (!checkIn || !checkOut) {
      setNights(0);
      return;
    }

    const start = new Date(checkIn);
    const end = new Date(checkOut);
    if (start >= end) {
      setNights(0);
      return;
    }

    const diff = Math.abs(end.getTime() - start.getTime());
    const calcNights = Math.ceil(diff / (1000 * 60 * 60 * 24));
    setNights(calcNights);

    const selectedType = availableRoomTypes.find(t => t.id === selectedRoomTypeId);
    const basePrice = selectedType ? Number(selectedType.basePrice) : 0;
    const baseSum = basePrice * calcNights;
    setBasePriceTotal(baseSum);

    let servSum = 0;
    services.forEach(s => {
      if (selectedServices.includes(s.id)) {
        servSum += Number(s.price) * calcNights;
```

```
      }
    });
    setServicesTotal(servSum);

    setGrandTotal(baseSum + servSum);
  }, [checkIn, checkOut, selectedRoomTypeId, selectedServices]);

  const handleServiceToggle = (id: string) => {
    setSelectedServices(prev =>
      prev.includes(id) ? prev.filter(sid => sid !== id) : [...prev, id]
    );
  };

  const handleBookingSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError('');

    if (!session) {
      router.push('/login?callbackUrl=' + encodeURIComponent(window.location.pathname
      return;
    }

    if ((session.user as any).type !== 'guest') {
      setError('Staff accounts cannot place guest room reservations.');
      return;
    }

    if (nights === 0) {
      setError('Please select valid check-in and check-out dates.');
      return;
    }

    setLoading(true);

    try {
      const res = await fetch('/api/book', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          roomTypeId: selectedRoomTypeId,
          checkIn,
          checkOut,
          numGuests,
          serviceIds: selectedServices
        })
      });

      const data = await res.json();
      if (!res.ok) {
        setError(data.error || 'Failed creating draft reservation.');
      } else {
        router.push(`/checkout/${data.reservationId}`);
      }
    } catch (err) {
      setError('An unexpected connection error occurred.');
    } finally {
      setLoading(false);
    }
  };

  return (
```

```
    <div className="mx-auto max-w-7xl px-4 py-12 sm:px-6 lg:px-8 space-y-12 animate-f

      {/* Back button */}
      <button
        onClick={() => router.push('/book')}
        className="inline-flex items-center gap-2 text-xs font-semibold uppercase tra
      >
        <ArrowLeft className="h-4 w-4 text-brand-accent" />
        <span>Return to Listings</span>
      </button>

      {/* Main Grid */}
      <div className="grid grid-cols-1 lg:grid-cols-12 gap-10">

        {/* Left Column (Resort Detail & Images Slider) */}
        <div className="lg:col-span-7 space-y-8">

          {/* Images Slider */}
          <div className="relative rounded-3xl overflow-hidden aspect-[16/10] bg-[#14
            <img
              src={resort.images[activeImageIdx] || 'https://images.unsplash.com/phot
              alt={resort.name}
              onError={(e) => {
                e.currentTarget.src = 'https://images.unsplash.com/photo-157189634984
              }}
              className="w-full h-full object-cover"
            />
            <div className="absolute inset-0 bg-gradient-to-t from-[#141414] via-tran

            {/* Thumbnail dots selector inside the image container */}
```

Function: `handleServiceToggle`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleServiceToggle = (id: string) => {
    setSelectedServices(prev =>
      prev.includes(id) ? prev.filter(sid => sid !== id) : [...prev, id]
    );
  };
```

Function: `handleBookingSubmit`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleBookingSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError('');

    if (!session) {
      router.push('/login?callbackUrl=' + encodeURIComponent(window.location.pathname
      return;
    }

    if ((session.user as any).type !== 'guest') {
      setError('Staff accounts cannot place guest room reservations.');
      return;
    }

    if (nights === 0) {
      setError('Please select valid check-in and check-out dates.');
      return;
    }

    setLoading(true);

    try {
      const res = await fetch('/api/book', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          roomTypeId: selectedRoomTypeId,
          checkIn,
          checkOut,
          numGuests,
          serviceIds: selectedServices
        })
      });

      const data = await res.json();
      if (!res.ok) {
        setError(data.error || 'Failed creating draft reservation.');
      } else {
        router.push(`/checkout/${data.reservationId}`);
      }
    } catch (err) {
      setError('An unexpected connection error occurred.');
    } finally {
      setLoading(false);
    }
  };
```

File Documentation: ResortMap.tsx

- **Relative Path:** `src/components/ResortMap.tsx`
- **Line Count:** 181 lines

Purpose & Summary

Visual interactive maps showing resort markers.

Architectural Role & Integration

Utilizes Leaflet to place pin indicators on geographic coordinates.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `ResortMap`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function ResortMap({ resorts, selectedResortId, onMarkerClick }: Resor
  const mapContainerRef = useRef<HTMLDivElement>(null);
  const mapRef = useRef<L.Map | null>(null);
  const markersRef = useRef<{ [key: string]: L.Marker }>({});

  useEffect(() => {
    if (!mapContainerRef.current) return;

    // Create map instance if it doesn't exist
    if (!mapRef.current) {
      mapRef.current = L.map(mapContainerRef.current, {
        zoomControl: false,
      }).setView([20.0, 0.0], 2);

      // Add zoom control at bottom-right
      L.control.zoom({ position: 'bottomright' }).addTo(mapRef.current);

      // Add elegant dark theme tiles (CartoDB Dark Matter)
      L.tileLayer('https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png', {
        attribution: '&copy; OpenStreetMap contributors &copy; CARTO',
        subdomains: 'abcd',
        maxZoom: 20
      }).addTo(mapRef.current);
    }

    const map = mapRef.current;

    // Clear existing markers
    Object.values(markersRef.current).forEach(marker => marker.remove());
    markersRef.current = {};

    // Custom glowing icon
    const defaultIcon = L.divIcon({
      className: 'custom-map-pin',
      html: `<div style="background-color: #fbbf24; border: 2px solid #0c0a09; width:
      iconSize: [16, 16],
      iconAnchor: [8, 8]
    });

    const activeIcon = L.divIcon({
      className: 'custom-map-pin-active',
      html: `<div style="background-color: #f59e0b; border: 2.5px solid #ffffff; widt
      iconSize: [22, 22],
      iconAnchor: [11, 11]
    });

    // Add markers for all resorts
    resorts.forEach(r => {
      const lat = Number(r.latitude);
      const lng = Number(r.longitude);
      if (isNaN(lat) || isNaN(lng)) return;

      const isSelected = r.id === selectedResortId;
      const marker = L.marker([lat, lng], {
        icon: isSelected ? activeIcon : defaultIcon
      }).addTo(map);

      marker.on('click', () => {
        onMarkerClick(r.id);
      });
```

```
      marker.bindTooltip(`<b>${r.name}</b><br/>⭐ ${r.rating}`, {
        direction: 'top',
        className: 'custom-map-tooltip'
      });

      markersRef.current[r.id] = marker;
    });

    return () => {
      // Map stays initialized
    };
  }, [resorts]);

  // Handle flyTo when selectedResortId changes
  useEffect(() => {
    if (!mapRef.current || !selectedResortId) return;
    const map = mapRef.current;

    // Recalculate dimensions in case the container was hidden or resized
    map.invalidateSize();

    const selectedResort = resorts.find(r => r.id === selectedResortId);
    const lat = selectedResort ? Number(selectedResort.latitude) : NaN;
    const lng = selectedResort ? Number(selectedResort.longitude) : NaN;
    if (!isNaN(lat) && !isNaN(lng)) {
      const container = map.getContainer();
      if (container && container.clientWidth > 0 && container.clientHeight > 0) {
        map.flyTo([lat, lng], 12, {
          duration: 1.5,
          easeLinearity: 0.25
        });
      } else {
        map.setView([lat, lng], 12);
      }

      // Update markers icons
      Object.entries(markersRef.current).forEach(([id, marker]) => {
        const isSelected = id === selectedResortId;
        const defaultIcon = L.divIcon({
          className: 'custom-map-pin',
          html: `<div style="background-color: #fbbf24; border: 2px solid #0c0a09; wi
          iconSize: [16, 16],
          iconAnchor: [8, 8]
        });

        const activeIcon = L.divIcon({
          className: 'custom-map-pin-active',
          html: `<div style="background-color: #f59e0b; border: 2.5px solid #ffffff;
          iconSize: [22, 22],
          iconAnchor: [11, 11]
        });
        marker.setIcon(isSelected ? activeIcon : defaultIcon);
      });
    }
  }, [selectedResortId, resorts]);

  // Clean up fully on unmount
  useEffect(() => {
    return () => {
      if (mapRef.current) {
```

```
        mapRef.current.remove();
        mapRef.current = null;
      }
    };
  }, []);

  return (
    <div className="relative w-full h-full rounded-3xl overflow-hidden border border-
      <div ref={mapContainerRef} className="w-full h-full" style={{ minHeight: '450px
      <style jsx global>{`
        .custom-map-tooltip {
          background-color: #1c1917 !important;
          color: #f5f5f4 !important;
          border: 1px solid #78350f !important;
          border-radius: 8px !important;
          font-family: var(--font-sans), sans-serif !important;
          font-size: 11px !important;
          padding: 6px 10px !important;
          box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1) !important;
        }
        .leaflet-container {
          background: #0c0a09 !important;
        }
        .leaflet-bar {
          border: 1px solid rgba(120, 53, 4, 0.2) !important;
        }
        .leaflet-bar a {
          background-color: #1c1917 !important;
          color: #fbbf24 !important;
          border-bottom: 1px solid rgba(120, 53, 4, 0.2) !important;
```

File Documentation: SessionProviderWrapper.tsx

- **Relative Path:** `src/components/SessionProviderWrapper.tsx`
- **Line Count:** 8 lines

Purpose & Summary

Next-Auth context provider.

Architectural Role & Integration

Wraps Client Components to allow access to user session details (`useSession` hook).

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `SessionProviderWrapper`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function SessionProviderWrapper({ children }: { children: React.ReactN
  return <SessionProvider>{children}</SessionProvider>;
}
```

File Documentation: TiltCard.tsx

- **Relative Path:** `src/components/TiltCard.tsx`
- **Line Count:** 71 lines

Purpose & Summary

21st.dev mouse-tilt 3D card wrapper.

Architectural Role & Integration

Animates card perspectives on hover to add micro-interactivity.

File Structure & Dependencies

- **Imports:** Tracks **1** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `TiltCard`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
export default function TiltCard({ children, className = '' }: { children: React.React
  const cardRef = useRef<HTMLDivElement>(null);
  const [transform, setTransform] = useState('perspective(1000px) rotateX(0deg) rotat
  const [spotlightPos, setSpotlightPos] = useState({ x: 0, y: 0 });
  const [isHovered, setIsHovered] = useState(false);

  const handleMouseMove = (e: React.MouseEvent<HTMLDivElement>) => {
    if (!cardRef.current) return;
    const card = cardRef.current;
    const box = card.getBoundingClientRect();
    const x = e.clientX - box.left;
    const y = e.clientY - box.top;

    // Relative coordinates from center
    const rx = x - box.width / 2;
    const ry = y - box.height / 2;

    // Calculate rotation angles (max 10 degrees for 3D tilt)
    const rotateX = -(ry / (box.height / 2)) * 10;
    const rotateY = (rx / (box.width / 2)) * 10;

    setTransform(`perspective(1000px) rotateX(${rotateX}deg) rotateY(${rotateY}deg) s
    setSpotlightPos({ x, y });
    setIsHovered(true);
  };

  const handleMouseLeave = () => {
    setTransform('perspective(1000px) rotateX(0deg) rotateY(0deg) scale3d(1, 1, 1)');
    setIsHovered(false);
  };

  return (
    <div
      ref={cardRef}
      onMouseMove={handleMouseMove}
      onMouseLeave={handleMouseLeave}
      style={{
        transform,
        transition: 'transform 0.2s cubic-bezier(0.25, 1, 0.5, 1), border-color 0.3s
        transformStyle: 'preserve-3d',
      }}
      className={`relative rounded-[28px] overflow-hidden border transition-all durat
    >
      {/* 3D Reflection overlay */}
      {isHovered && (
        <div
          className="absolute inset-0 pointer-events-none z-10 mix-blend-overlay opac
          style={{
            background: `radial-gradient(circle 250px at ${spotlightPos.x}px ${spotli
          }}
        />
      )}

      {/* 3D Border Highlight overlay */}
      {isHovered && (
        <div
          className="absolute inset-0 pointer-events-none z-10 opacity-40 transition-
          style={{
            background: `radial-gradient(circle 180px at ${spotlightPos.x}px ${spotli
          }}
```

```
          />
        )}

        {children}
      </div>
    );
  }
```

Function: `handleMouseMove`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleMouseMove = (e: React.MouseEvent<HTMLDivElement>) => {
    if (!cardRef.current) return;
    const card = cardRef.current;
    const box = card.getBoundingClientRect();
    const x = e.clientX - box.left;
    const y = e.clientY - box.top;

    // Relative coordinates from center
    const rx = x - box.width / 2;
    const ry = y - box.height / 2;

    // Calculate rotation angles (max 10 degrees for 3D tilt)
    const rotateX = -(ry / (box.height / 2)) * 10;
    const rotateY = (rx / (box.width / 2)) * 10;

    setTransform(`perspective(1000px) rotateX(${rotateX}deg) rotateY(${rotateY}deg) s
    setSpotlightPos({ x, y });
    setIsHovered(true);
  };
```

Function: `handleMouseLeave`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
  const handleMouseLeave = () => {
    setTransform('perspective(1000px) rotateX(0deg) rotateY(0deg) scale3d(1, 1, 1)');
    setIsHovered(false);
  };
```

File Documentation: button.tsx

- **Relative Path:** `src/components/ui/button.tsx`
- **Line Count:** 59 lines

Purpose & Summary

Primitive Button component supporting variants.

Architectural Role & Integration

Base visual design token for submit actions and links.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Button`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
import { Button as ButtonPrimitive } from "@base-ui/react/button"
```

File Documentation: calendar.tsx

- **Relative Path:** `src/components/ui/calendar.tsx`
- **Line Count:** 222 lines

Purpose & Summary

Custom styling wrapper for react-day-picker.

Architectural Role & Integration

Core calendar element used in booking sheets and checklists.

File Structure & Dependencies

- **Imports:** Tracks **5** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Calendar`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function Calendar({
  className,
  classNames,
  showOutsideDays = true,
  captionLayout = "label",
  buttonVariant = "ghost",
  locale,
  formatters,
  components,
  ...props
}: React.ComponentProps<typeof DayPicker> & {
  buttonVariant?: React.ComponentProps<typeof Button>["variant"]
}) {
  const defaultClassNames = getDefaultClassNames()

  return (
    <DayPicker
      showOutsideDays={showOutsideDays}
      className={cn(
        "group/calendar bg-background p-2 [--cell-radius:var(--radius-md)] [--cell-si
        String.raw`rtl:**:[.rdp-button\_next>svg]:rotate-180`,
        String.raw`rtl:**:[.rdp-button\_previous>svg]:rotate-180`,
        className
      )}
      captionLayout={captionLayout}
      locale={locale}
      formatters={{
        formatMonthDropdown: (date) =>
          date.toLocaleString(locale?.code, { month: "short" }),
        ...formatters,
      }}
      classNames={{
        root: cn("w-fit", defaultClassNames.root),
        months: cn(
          "relative flex flex-col gap-4 md:flex-row",
          defaultClassNames.months
        ),
        month: cn("flex w-full flex-col gap-4", defaultClassNames.month),
        nav: cn(
          "absolute inset-x-0 top-0 flex w-full items-center justify-between gap-1",
          defaultClassNames.nav
        ),
        button_previous: cn(
          buttonVariants({ variant: buttonVariant }),
          "size-(--cell-size) p-0 select-none aria-disabled:opacity-50",
          defaultClassNames.button_previous
        ),
        button_next: cn(
          buttonVariants({ variant: buttonVariant }),
          "size-(--cell-size) p-0 select-none aria-disabled:opacity-50",
          defaultClassNames.button_next
        ),
        month_caption: cn(
          "flex h-(--cell-size) w-full items-center justify-center px-(--cell-size)",
          defaultClassNames.month_caption
        ),
        dropdowns: cn(
          "flex h-(--cell-size) w-full items-center justify-center gap-1.5 text-sm fo
          defaultClassNames.dropdowns
        ),
```

```
          dropdown_root: cn(
            "relative rounded-(--cell-radius)",
            defaultClassNames.dropdown_root
          ),
          dropdown: cn(
            "absolute inset-0 bg-popover opacity-0",
            defaultClassNames.dropdown
          ),
          caption_label: cn(
            "font-medium select-none",
            captionLayout === "label"
              ? "text-sm"
              : "flex items-center gap-1 rounded-(--cell-radius) text-sm [&>svg]:size-3
            defaultClassNames.caption_label
          ),
          month_grid: cn("w-full border-collapse", defaultClassNames.month_grid),
          weekdays: cn("flex", defaultClassNames.weekdays),
          weekday: cn(
            "flex-1 rounded-(--cell-radius) text-[0.8rem] font-normal text-muted-foregr
            defaultClassNames.weekday
          ),
          week: cn("mt-2 flex w-full", defaultClassNames.week),
          week_number_header: cn(
            "w-(--cell-size) select-none",
            defaultClassNames.week_number_header
          ),
          week_number: cn(
            "text-[0.8rem] text-muted-foreground select-none",
            defaultClassNames.week_number
          ),
          day: cn(
            "group/day relative aspect-square h-full w-full rounded-(--cell-radius) p-0
            props.showWeekNumber
              ? "[&:nth-child(2)[data-selected=true]_button]:rounded-l-(--cell-radius)"
              : "[&:first-child[data-selected=true]_button]:rounded-l-(--cell-radius)",
            defaultClassNames.day
          ),
          range_start: cn(
            "relative isolate z-0 rounded-l-(--cell-radius) bg-muted after:absolute aft
            defaultClassNames.range_start
          ),
          range_middle: cn("rounded-none", defaultClassNames.range_middle),
          range_end: cn(
            "relative isolate z-0 rounded-r-(--cell-radius) bg-muted after:absolute aft
            defaultClassNames.range_end
          ),
          today: cn(
            "rounded-(--cell-radius) bg-muted text-foreground data-[selected=true]:roun
            defaultClassNames.today
          ),
          outside: cn(
            "text-muted-foreground aria-selected:text-muted-foreground",
            defaultClassNames.outside
          ),
          disabled: cn(
            "text-muted-foreground opacity-50",
            defaultClassNames.disabled
          ),
          hidden: cn("invisible", defaultClassNames.hidden),
          ...classNames,
        }}
```

```
        components={{
          Root: ({ className, rootRef, ...props }) => {
            return (
              <div
                data-slot="calendar"
                ref={rootRef}
                className={cn(className)}
                {...props}
              />
            )
          },
          Chevron: ({ className, orientation, ...props }) => {
            if (orientation === "left") {
              return (
                <ChevronLeftIcon className={cn("size-4", className)} {...props} />
              )
            }

            if (orientation === "right") {
              return (
                <ChevronRightIcon className={cn("size-4", className)} {...props} />
              )
            }

            return (
              <ChevronDownIcon className={cn("size-4", className)} {...props} />
            )
          },
          DayButton: ({ ...props }) => (
            <CalendarDayButton locale={locale} {...props} />
```

Function: `CalendarDayButton`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
            <CalendarDayButton locale={locale} {...props} />
```

File Documentation: card.tsx

- **Relative Path:** `src/components/ui/card.tsx`
- **Line Count:** 104 lines

Purpose & Summary

Bento grid container wrapper.

Architectural Role & Integration

Provides border radius, borders, and layouts for modular grids.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Card`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function Card({
  className,
  size = "default",
  ...props
}: React.ComponentProps<"div"> & { size?: "default" | "sm" }) {
  return (
    <div
      data-slot="card"
      data-size={size}
      className={cn(
        "group/card flex flex-col gap-(--card-spacing) overflow-hidden rounded-xl bg-
        className
      )}
      {...props}
    />
  )
}
```

Function: `CardHeader`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function CardHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-header"
      className={cn(
        "group/card-header @container/card-header grid auto-rows-min items-start gap-
        className
      )}
      {...props}
    />
  )
}
```

Function: `CardTitle`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function CardTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-title"
      className={cn(
        "font-heading text-base leading-snug font-medium group-data-[size=sm]/card:te
        className
      )}
      {...props}
    />
  )
}
```

Function: `CardDescription`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function CardDescription({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-description"
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  )
}
```

Function: `CardAction`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function CardAction({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-action"
      className={cn(
        "col-start-2 row-span-2 row-start-1 self-start justify-self-end",
        className
      )}
      {...props}
    />
  )
}
```

Function: `CardContent`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function CardContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-content"
      className={cn("px-(--card-spacing)", className)}
      {...props}
    />
  )
}
```

Function: `CardFooter`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function CardFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-footer"
      className={cn(
        "flex items-center rounded-b-xl border-t bg-muted/50 p-(--card-spacing)",
        className
      )}
      {...props}
    />
  )
}
```

File Documentation: dialog.tsx

- **Relative Path:** `src/components/ui/dialog.tsx`
- **Line Count:** 161 lines

Purpose & Summary

Interactive modal overlays using Radix primitives.

Architectural Role & Integration

Used for delete confirmations, new item creation fields, and warnings.

File Structure & Dependencies

- **Imports:** Tracks **5** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Dialog`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
import { Dialog as DialogPrimitive } from "@base-ui/react/dialog"
```

Function: `DialogTrigger`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogTrigger({ ...props }: DialogPrimitive.Trigger.Props) {
  return <DialogPrimitive.Trigger data-slot="dialog-trigger" {...props} />
}
```

Function: `DialogPortal`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogPortal({ ...props }: DialogPrimitive.Portal.Props) {
  return <DialogPrimitive.Portal data-slot="dialog-portal" {...props} />
}
```

Function: `DialogClose`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogClose({ ...props }: DialogPrimitive.Close.Props) {
  return <DialogPrimitive.Close data-slot="dialog-close" {...props} />
}
```

Function: `DialogOverlay`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogOverlay({
  className,
  ...props
}: DialogPrimitive.Backdrop.Props) {
  return (
    <DialogPrimitive.Backdrop
      data-slot="dialog-overlay"
      className={cn(
        "fixed inset-0 isolate z-50 bg-black/10 duration-100 supports-backdrop-filter
        className
      )}
      {...props}
    />
  )
}
```

Function: `DialogContent`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogContent({
  className,
  children,
  showCloseButton = true,
  ...props
}: DialogPrimitive.Popup.Props & {
  showCloseButton?: boolean
}) {
  return (
    <DialogPortal>
      <DialogOverlay />
      <DialogPrimitive.Popup
        data-slot="dialog-content"
        className={cn(
          "fixed top-1/2 left-1/2 z-50 grid w-full max-w-[calc(100%-2rem)] -translate
          className
        )}
        {...props}
      >
        {children}
        {showCloseButton && (
          <DialogPrimitive.Close
            data-slot="dialog-close"
            render={
              <Button
                variant="ghost"
                className="absolute top-2 right-2"
                size="icon-sm"
              />
            }
          >
            <XIcon
            />
            <span className="sr-only">Close</span>
          </DialogPrimitive.Close>
        )}
      </DialogPrimitive.Popup>
    </DialogPortal>
  )
}
```

Function: `DialogHeader`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="dialog-header"
      className={cn("flex flex-col gap-2", className)}
      {...props}
    />
  )
}
```

Function: `DialogFooter`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogFooter({
  className,
  showCloseButton = false,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  showCloseButton?: boolean
}) {
  return (
    <div
      data-slot="dialog-footer"
      className={cn(
        "-mx-4 -mb-4 flex flex-col-reverse gap-2 rounded-b-xl border-t bg-muted/50 p-
        className
      )}
      {...props}
    >
      {children}
      {showCloseButton && (
        <DialogPrimitive.Close render={<Button variant="outline" />}>
          Close
        </DialogPrimitive.Close>
      )}
    </div>
  )
}
```

Function: `DialogTitle`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogTitle({ className, ...props }: DialogPrimitive.Title.Props) {
  return (
    <DialogPrimitive.Title
      data-slot="dialog-title"
      className={cn(
        "font-heading text-base leading-none font-medium",
        className
      )}
      {...props}
    />
  )
}
```

Function: `DialogDescription`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DialogDescription({
  className,
  ...props
}: DialogPrimitive.Description.Props) {
  return (
    <DialogPrimitive.Description
      data-slot="dialog-description"
      className={cn(
        "text-sm text-muted-foreground *:[a]:underline *:[a]:underline-offset-3 *:[a]
        className
      )}
      {...props}
    />
  )
}
```

File Documentation: dropdown-menu.tsx

- **Relative Path:** `src/components/ui/dropdown-menu.tsx`
- **Line Count:** 269 lines

Purpose & Summary

Floating drop-down menus.

Architectural Role & Integration

Used for navigation settings, guest options, and filters.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `DropdownMenu`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenu({ ...props }: MenuPrimitive.Root.Props) {
  return <MenuPrimitive.Root data-slot="dropdown-menu" {...props} />
}
```

Function: `DropdownMenuPortal`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuPortal({ ...props }: MenuPrimitive.Portal.Props) {
  return <MenuPrimitive.Portal data-slot="dropdown-menu-portal" {...props} />
}
```

Function: `DropdownMenuTrigger`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuTrigger({ ...props }: MenuPrimitive.Trigger.Props) {
  return <MenuPrimitive.Trigger data-slot="dropdown-menu-trigger" {...props} />
}
```

Function: `DropdownMenuContent`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuContent({
  align = "start",
  alignOffset = 0,
  side = "bottom",
  sideOffset = 4,
  className,
  ...props
}: MenuPrimitive.Popup.Props &
```

Function: `DropdownMenuGroup`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuGroup({ ...props }: MenuPrimitive.Group.Props) {
  return <MenuPrimitive.Group data-slot="dropdown-menu-group" {...props} />
}
```

Function: `DropdownMenuLabel`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuLabel({
  className,
  inset,
  ...props
}: MenuPrimitive.GroupLabel.Props & {
  inset?: boolean
}) {
  return (
    <MenuPrimitive.GroupLabel
      data-slot="dropdown-menu-label"
      data-inset={inset}
      className={cn(
        "px-1.5 py-1 text-xs font-medium text-muted-foreground data-inset:pl-7",
        className
      )}
      {...props}
    />
  )
}
```

Function: `DropdownMenuItem`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}: MenuPrimitive.Item.Props & {
  inset?: boolean
  variant?: "default" | "destructive"
}) {
  return (
    <MenuPrimitive.Item
      data-slot="dropdown-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "group/dropdown-menu-item relative flex cursor-default items-center gap-1.5 r
        className
      )}
      {...props}
    />
  )
}
```

Function: `DropdownMenuSub`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuSub({ ...props }: MenuPrimitive.SubmenuRoot.Props) {
  return <MenuPrimitive.SubmenuRoot data-slot="dropdown-menu-sub" {...props} />
}
```

Function: `DropdownMenuSubTrigger`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: MenuPrimitive.SubmenuTrigger.Props & {
  inset?: boolean
}) {
  return (
    <MenuPrimitive.SubmenuTrigger
      data-slot="dropdown-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "flex cursor-default items-center gap-1.5 rounded-md px-1.5 py-1 text-sm outl
        className
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto" />
    </MenuPrimitive.SubmenuTrigger>
  )
}
```

Function: `DropdownMenuSubContent`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuSubContent({
  align = "start",
  alignOffset = -3,
  side = "right",
  sideOffset = 0,
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuContent>) {
  return (
    <DropdownMenuContent
      data-slot="dropdown-menu-sub-content"
      className={cn("w-auto min-w-[96px] rounded-lg bg-popover p-1 text-popover-foreg
      align={align}
      alignOffset={alignOffset}
      side={side}
      sideOffset={sideOffset}
      {...props}
    />
  )
}
```

Function: `DropdownMenuCheckboxItem`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuCheckboxItem({
  className,
  children,
  checked,
  inset,
  ...props
}: MenuPrimitive.CheckboxItem.Props & {
  inset?: boolean
}) {
  return (
    <MenuPrimitive.CheckboxItem
      data-slot="dropdown-menu-checkbox-item"
      data-inset={inset}
      className={cn(
        "relative flex cursor-default items-center gap-1.5 rounded-md py-1 pr-8 pl-1.
        className
      )}
      checked={checked}
      {...props}
    >
      <span
        className="pointer-events-none absolute right-2 flex items-center justify-cen
        data-slot="dropdown-menu-checkbox-item-indicator"
      >
        <MenuPrimitive.CheckboxItemIndicator>
          <CheckIcon
          />
        </MenuPrimitive.CheckboxItemIndicator>
      </span>
      {children}
    </MenuPrimitive.CheckboxItem>
  )
}
```

Function: `DropdownMenuRadioGroup`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuRadioGroup({ ...props }: MenuPrimitive.RadioGroup.Props) {
  return (
    <MenuPrimitive.RadioGroup
      data-slot="dropdown-menu-radio-group"
      {...props}
    />
  )
}
```

Function: `DropdownMenuRadioItem`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuRadioItem({
  className,
  children,
  inset,
  ...props
}: MenuPrimitive.RadioItem.Props & {
  inset?: boolean
}) {
  return (
    <MenuPrimitive.RadioItem
      data-slot="dropdown-menu-radio-item"
      data-inset={inset}
      className={cn(
        "relative flex cursor-default items-center gap-1.5 rounded-md py-1 pr-8 pl-1.
        className
      )}
      {...props}
    >
      <span
        className="pointer-events-none absolute right-2 flex items-center justify-cen
        data-slot="dropdown-menu-radio-item-indicator"
      >
        <MenuPrimitive.RadioItemIndicator>
          <CheckIcon
          />
        </MenuPrimitive.RadioItemIndicator>
      </span>
      {children}
    </MenuPrimitive.RadioItem>
  )
}
```

Function: `DropdownMenuSeparator`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuSeparator({
  className,
  ...props
}: MenuPrimitive.Separator.Props) {
  return (
    <MenuPrimitive.Separator
      data-slot="dropdown-menu-separator"
      className={cn("-mx-1 my-1 h-px bg-border", className)}
      {...props}
    />
  )
}
```

Function: `DropdownMenuShortcut`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function DropdownMenuShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="dropdown-menu-shortcut"
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground group-focus/dropdown-m
        className
      )}
      {...props}
    />
  )
}
```

File Documentation: input.tsx

- **Relative Path:** `src/components/ui/input.tsx`
- **Line Count:** 21 lines

Purpose & Summary

Base standard inputs.

Architectural Role & Integration

Governs forms for signup, login, and CRUD actions.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Input`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
import { Input as InputPrimitive } from "@base-ui/react/input"
```

File Documentation: popover.tsx

- **Relative Path:** `src/components/ui/popover.tsx`
- **Line Count:** 91 lines

Purpose & Summary

Radix-based floating popovers.

Architectural Role & Integration

Anchors calendar date pickers and guest counts selectors.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Popover`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
import { Popover as PopoverPrimitive } from "@base-ui/react/popover"
```

Function: `PopoverTrigger`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function PopoverTrigger({ ...props }: PopoverPrimitive.Trigger.Props) {
  return <PopoverPrimitive.Trigger data-slot="popover-trigger" {...props} />
}
```

Function: `PopoverContent`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function PopoverContent({
  className,
  align = "center",
  alignOffset = 0,
  side = "bottom",
  sideOffset = 4,
  ...props
}: PopoverPrimitive.Popup.Props &
```

Function: `PopoverHeader`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function PopoverHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="popover-header"
      className={cn("flex flex-col gap-0.5 text-sm", className)}
      {...props}
    />
  )
}
```

Function: `PopoverTitle`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function PopoverTitle({ className, ...props }: PopoverPrimitive.Title.Props) {
  return (
    <PopoverPrimitive.Title
      data-slot="popover-title"
      className={cn("font-medium", className)}
      {...props}
    />
  )
}
```

Function: `PopoverDescription`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function PopoverDescription({
  className,
  ...props
}: PopoverPrimitive.Description.Props) {
  return (
    <PopoverPrimitive.Description
      data-slot="popover-description"
      className={cn("text-muted-foreground", className)}
      {...props}
    />
  )
}
```

File Documentation: tabs.tsx

- **Relative Path:** `src/components/ui/tabs.tsx`
- **Line Count:** 83 lines

Purpose & Summary

Switchable panel sheets.

Architectural Role & Integration

Governs dashboard sections and public booking selections.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `Tabs`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
import { Tabs as TabsPrimitive } from "@base-ui/react/tabs"
```

Function: `TabsList`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function TabsList({
  className,
  variant = "default",
  ...props
}: TabsPrimitive.List.Props & VariantProps<typeof tabsListVariants>) {
  return (
    <TabsPrimitive.List
      data-slot="tabs-list"
      data-variant={variant}
      className={cn(tabsListVariants({ variant }), className)}
      {...props}
    />
  )
}
```

Function: `TabsTrigger`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function TabsTrigger({ className, ...props }: TabsPrimitive.Tab.Props) {
  return (
    <TabsPrimitive.Tab
      data-slot="tabs-trigger"
      className={cn(
        "relative inline-flex h-[calc(100%-1px)] flex-1 items-center justify-center g
        "group-data-[variant=line]/tabs-list:bg-transparent group-data-[variant=line]
        "data-active:bg-background data-active:text-foreground dark:data-active:borde
        "after:absolute after:bg-foreground after:opacity-0 after:transition-opacity
        className
      )}
      {...props}
    />
  )
}
```

Function: `TabsContent`

Inputs/Parameters: Varies by invocation context.

Outputs/Returns: NextResponse, JSX component tree, or model payload.

Internal Logic & Purpose:

> Executes state adjustments, database transactions, or responsive layout renders.

Actual Function Source Code:

```
function TabsContent({ className, ...props }: TabsPrimitive.Panel.Props) {
  return (
    <TabsPrimitive.Panel
      data-slot="tabs-content"
      className={cn("flex-1 text-sm outline-none", className)}
      {...props}
    />
  )
}
```

File Documentation: auth.ts

- **Relative Path:** `src/lib/auth.ts`
- **Line Count:** 81 lines

Purpose & Summary

Credentials callback, authorization rules, and session mappings.

Architectural Role & Integration

Core configuration instance exported for Next-Auth APIs and middleware rules.

File Structure & Dependencies

- **Imports:** Tracks **4** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Source Code Implementation

```
import CredentialsProvider from "next-auth/providers/credentials";
import { prisma } from "./db";
import bcrypt from "bcryptjs";
import { AuthOptions } from "next-auth";

export const authOptions: AuthOptions = {
  providers: [
    CredentialsProvider({
      name: "Credentials",
      credentials: {
        email: { label: "Email", type: "text" },
        password: { label: "Password", type: "password" },
        roleType: { label: "Role Type", type: "text" }
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) return null;

        const { email, password, roleType } = credentials;

        if (roleType === 'STAFF') {
          const staff = await prisma.staff.findUnique({
            where: { email },
            include: { department: true }
          });
          if (staff && bcrypt.compareSync(password, staff.password)) {
            return {
              id: staff.id,
              name: staff.fullName,
              email: staff.email,
              role: staff.role,
              type: 'staff',
              department: staff.department.name
            } as any;
          }
        } else {
          const guest = await prisma.guest.findUnique({
            where: { email }
          });
          if (guest && bcrypt.compareSync(password, guest.password)) {
            return {
              id: guest.id,
              name: guest.fullName,
              email: guest.email,
              role: 'GUEST',
              type: 'guest'
            } as any;
          }
        }
        return null;
      }
    })
  ],
  callbacks: {
    async jwt({ token, user }) {
      if (user) {
        token.id = user.id;
```

```
        token.role = (user as any).role;
        token.type = (user as any).type;
        token.department = (user as any).department;
      }
      return token;
    },
    async session({ session, token }) {
      if (session.user) {
        (session.user as any).id = token.id;
        (session.user as any).role = token.role;
        (session.user as any).type = token.type;
        (session.user as any).department = token.department;
      }
      return session;
    }
  },
  pages: {
    signIn: '/login',
  },
  session: {
    strategy: 'jwt',
  },
  secret: process.env.NEXTAUTH_SECRET,
};
```

File Documentation: cache.ts

- **Relative Path:** `src/lib/cache.ts`
- **Line Count:** 65 lines

Purpose & Summary

Memory storage cache preventing repeat db queries.

Architectural Role & Integration

Exposes clear, invalidate, and getOrSet actions to optimize resource loads.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** Declares **1** export indicators for external module integration.

Source Code Implementation

```
/**
 * Lightweight in-memory cache for API responses.
 * Avoids repeated database queries for data that rarely changes (e.g. resorts list).
 * Each entry auto-expires after its TTL.
 *
 * In serverless environments (Vercel), each cold-start gets a fresh cache.
 * Warm invocations reuse cached data, significantly reducing DB round-trips.
 */

interface CacheEntry<T> {
  data: T;
  expiresAt: number;
}

class MemoryCache {
  private store = new Map<string, CacheEntry<any>>();

  /**
   * Get cached data, or execute the fetcher and cache the result.
   * @param key - Unique cache key
   * @param fetcher - Async function that produces the data
   * @param ttlSeconds - Time-to-live in seconds (default: 60s)
   */
  async getOrSet<T>(key: string, fetcher: () => Promise<T>, ttlSeconds = 60): Promise<T> {
    const now = Date.now();
    const existing = this.store.get(key);

    if (existing && existing.expiresAt > now) {
      return existing.data as T;
    }

    const data = await fetcher();
    this.store.set(key, {
      data,
      expiresAt: now + ttlSeconds * 1000,
    });

    return data;
  }

  /** Invalidate a specific cache key */
  invalidate(key: string): void {
    this.store.delete(key);
  }

  /** Invalidate all keys matching a prefix */
  invalidatePrefix(prefix: string): void {
    for (const key of this.store.keys()) {
      if (key.startsWith(prefix)) {
        this.store.delete(key);
      }
    }
  }

  /** Clear the entire cache */
  clear(): void {
```

```
    this.store.clear();
  }
}

// Singleton instance persists across warm serverless invocations
const globalForCache = global as unknown as { __memoryCache: MemoryCache };
export const cache = globalForCache.__memoryCache || new MemoryCache();
if (process.env.NODE_ENV !== 'production') globalForCache.__memoryCache = cache;
```

File Documentation: db.ts

- **Relative Path:** `src/lib/db.ts`
- **Line Count:** 13 lines

Purpose & Summary

Global Prisma Client instance initializer.

Architectural Role & Integration

Acts as a database driver, managing connections securely to avoid pool leaks.

File Structure & Dependencies

- **Imports:** Tracks **1** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Source Code Implementation

```
import { PrismaClient } from '@prisma/client';

const globalForPrisma = global as unknown as { prisma: PrismaClient };

export const prisma =
  globalForPrisma.prisma ||
  new PrismaClient({
    // Disable verbose query logging in production for performance
    log: process.env.NODE_ENV === 'development' ? ['warn', 'error'] : ['error'],
  });

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

File Documentation: mailer.ts

- **Relative Path:** `src/lib/mailer.ts`
- **Line Count:** 30 lines

Purpose & Summary

Mailer configurations utilizing SMTP.

Architectural Role & Integration

Dispatches transactional emails for reservation settlements and updates.

File Structure & Dependencies

- **Imports:** Tracks **1** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **2** export indicators for external module integration.

Functions & APIs Specifications

Function: `sendMail`

Inputs/Parameters: `{ to, subject, html }` - Email params.

Outputs/Returns: `Promise` - Nodemailer dispatch log.

Internal Logic & Purpose:

Configures transport layer using app passwords. Dispatches HTML receipt letters to guests.

Actual Function Source Code:

```
export const sendMail = async ({ to, subject, html }: { to: string; subject: string;
  const mailOptions = {
    from: `"Luxury Horizon Resort" <${process.env.SMTP_USER}>`,
    to,
    subject,
    html,
  };

  try {
    const info = await transporter.sendMail(mailOptions);
    console.log('Email sent: %s', info.messageId);
    return info;
  } catch (error) {
    console.error('Error sending email:', error);
    throw error;
  }
};
```

File Documentation: utils.ts

- **Relative Path:** `src/lib/utils.ts`
- **Line Count:** 7 lines

Purpose & Summary

Helper tools resolving style collisions.

Architectural Role & Integration

Combines clsx and tailwind-merge (cn) to safely append CSS styles dynamically.

File Structure & Dependencies

- **Imports:** Tracks **2** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Functions & APIs Specifications

Function: `cn`

Inputs/Parameters: `...inputs: ClassValue[]` - Class names.

Outputs/Returns: `string` - Compiled clean CSS class string.

Internal Logic & Purpose:

> Combines class values via `clsx` and resolves overlaps via `tailwind-merge`.

Actual Function Source Code:

```
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

File Documentation: proxy.ts

- **Relative Path:** `src/proxy.ts`
- **Line Count:** 16 lines

Purpose & Summary

Next-Auth route protection middleware configuration.

Architectural Role & Integration

Standard Next.js middleware protecting secure folders from unauthenticated guests.

File Structure & Dependencies

- **Imports:** Tracks **1** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **2** export indicators for external module integration.

Source Code Implementation

```
import { withAuth } from "next-auth/middleware";

export default withAuth({
  pages: {
    signIn: "/login",
  },
});

export const config = {
  matcher: [
    "/dashboard/:path*",
    "/book/:path*",
    "/checkout/:path*"
  ],
};
```

File Documentation: components.json

- **Relative Path:** `components.json`
- **Line Count:** 26 lines

Purpose & Summary

Configuration file for UI component primitives (shadcn/ui).

Architectural Role & Integration

Defines directories for UI components, global CSS, tailwind configurations, and utility helpers.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** No exports declared.

Source Code Implementation

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "base-nova",
  "rsc": true,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "src/app/globals.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
  "iconLibrary": "lucide",
  "rtl": false,
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "menuColor": "default",
  "menuAccent": "subtle",
  "registries": {}
}
```

File Documentation: eslint.config.mjs

- **Relative Path:** `eslint.config.mjs`
- **Line Count:** 19 lines

Purpose & Summary

Static linting configuration mapping Next.js rules.

Architectural Role & Integration

Used during pre-build pipelines to enforce typescript and React syntax consistency.

File Structure & Dependencies

- **Imports:** Tracks **3** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Source Code Implementation

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
]);

export default eslintConfig;
```

File Documentation: next-env.d.ts

- **Relative Path:** `next-env.d.ts`
- **Line Count:** 7 lines

Purpose & Summary

Next.js compilation environment types.

Architectural Role & Integration

Ensures type safety for Next.js features such as image formats and global variables.

File Structure & Dependencies

- **Imports:** Tracks **1** import declarations referencing external dependencies or local utilities.
- **Exports:** No exports declared.

Source Code Implementation

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "./.next/types/routes.d.ts";

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.
```

File Documentation: next.config.ts

- **Relative Path:** `next.config.ts`
- **Line Count:** 78 lines

Purpose & Summary

Centralized compilation and request routing configuration.

Architectural Role & Integration

Manages asset compression, security headers (X-Frame-Options, CSP, etc.), and image remote CDN white-lists.

File Structure & Dependencies

- **Imports:** Tracks **1** import declarations referencing external dependencies or local utilities.
- **Exports:** Declares **1** export indicators for external module integration.

Source Code Implementation

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  // Enable React strict mode for better development checks
  reactStrictMode: true,

  // Optimize images for Vercel's CDN
  images: {
    formats: ['image/avif', 'image/webp'],
    minimumCacheTTL: 60 * 60 * 24 * 30, // 30 days
    deviceSizes: [640, 750, 828, 1080, 1200, 1920, 2048],
    imageSizes: [16, 32, 48, 64, 96, 128, 256, 384],
    remotePatterns: [
      { protocol: 'https', hostname: '**' },
    ],
  },

  // Enable gzip/brotli compression
  compress: true,

  // Powered-by header removal for security
  poweredByHeader: false,

  // Production source maps disabled for faster builds
  productionBrowserSourceMaps: false,

  // Optimize package imports for tree-shaking
  experimental: {
    optimizePackageImports: [
      'lucide-react',
      'date-fns',
      'gsap',
    ],
  },

  // Advanced HTTP headers for caching and security
  async headers() {
    return [

      {
        // Images - long cache (30 days)
        source: '/images/:path*',
        headers: [
          { key: 'Cache-Control', value: 'public, max-age=2592000, stale-while-revalidate=8
        ],
      },
      {
        // Fonts - immutable cache
        source: '/:path*.woff2',
        headers: [
          { key: 'Cache-Control', value: 'public, max-age=31536000, immutable' },
        ],
      },
      {
        // API routes - short cache with revalidation
        source: '/api/resorts',
```

```
        headers: [
          { key: 'Cache-Control', value: 'public, s-maxage=60, stale-while-revalidate=300' }
        ],
      },
      {
        // All pages - security headers
        source: '/:path*',
        headers: [
          { key: 'X-DNS-Prefetch-Control', value: 'on' },
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'X-Frame-Options', value: 'DENY' },
          { key: 'X-XSS-Protection', value: '1; mode=block' },
          { key: 'Referrer-Policy', value: 'origin-when-cross-origin' },
          { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=()' },
        ],
      },
    ];
  },
};

export default nextConfig;
```

File Documentation: package.json

- **Relative Path:** `package.json`
- **Line Count:** 52 lines

Purpose & Summary

Project dependencies, dev dependencies, build targets, and metadata registry.

Architectural Role & Integration

Configures next, react, prisma client generation (`postinstall` script), and run environments (dev server, builds).

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** No exports declared.

Source Code Implementation

```
{
  "name": "resort-booking",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev --turbopack",
    "build": "prisma generate && next build",
    "postinstall": "prisma generate",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "@base-ui/react": "^1.6.0",
    "@prisma/client": "^6.2.1",
    "bcryptjs": "^3.0.3",
    "canvas-confetti": "^1.9.4",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "date-fns": "^4.4.0",
    "gsap": "^3.15.0",
    "leaflet": "^1.9.4",
    "lucide-react": "^1.24.0",
    "next": "16.2.10",
    "next-auth": "^4.24.14",
    "nodemailer": "^7.0.13",
    "react": "19.2.4",
    "react-day-picker": "^10.0.1",
    "react-dom": "19.2.4",
    "shadcn": "^4.13.0",
    "tailwind-merge": "^3.6.0",
    "tw-animate-css": "^1.4.0"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/bcryptjs": "^2.4.6",
    "@types/canvas-confetti": "^1.9.0",
    "@types/leaflet": "^1.9.21",
    "@types/node": "^20",
    "@types/nodemailer": "^8.0.1",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.2.10",
    "prisma": "^6.2.1",
    "tailwindcss": "^4",
    "typescript": "^5"
  },
  "prisma": {
    "seed": "node prisma/seed_100.js"
  }
}
```

File Documentation: postcss.config.mjs

- **Relative Path:** `postcss.config.mjs`
- **Line Count:** 8 lines

Purpose & Summary

PostCSS configurations for compiling advanced CSS directives.

Architectural Role & Integration

Wires Tailwind CSS processors to parse utilities and browser prefix rules.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** Declares **1** export indicators for external module integration.

Source Code Implementation

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

export default config;
```

File Documentation: tsconfig.json

- **Relative Path:** `tsconfig.json`
- **Line Count:** 35 lines

Purpose & Summary

TypeScript compiler options configuration.

Architectural Role & Integration

Governs path aliases (`@/*`), strict parsing flags, and compiler target environments.

File Structure & Dependencies

- **Imports:** No imports declared in this file.
- **Exports:** No exports declared.

Source Code Implementation

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```